Universidad Rafael Landívar

Facultad de Ingeniería

Ingeniería en Informática y Sistemas

Ingeniería de Software I · Proyecto final 2026

Shapi

Documento de Diseño

Plataforma como servicio para publicar APIs, controlar su uso y cobrar por ellas

Héctor Jordin Adolfo García Coyoy	2427124
Ronaldo Emilio Méndez Mayorga	1563224
Dominique Guillermo Contreras Sierra	1508224
José Pablo Zúñiga de León	1507524

Quetzaltenango, Guatemala

25 de octubre de 2026

Índice

Seleccione un capítulo o sección para ir a su contenido. El PDF también incluye marcadores de navegación.

06 · Arquitectura

1. Estilo arquitectónico y justificación
2. Diagrama de contexto
3. Contenedores y procesos
4. Hosts y enrutamiento en el borde
5. Diagramas de secuencia
6. Dominios simulados
7. Despliegue
8. Observabilidad y estado de componentes
9. Estructura del repositorio

07 · Modelo de datos

1. Diagrama de clases del dominio
2. Modelo entidad-relación
3. Diseño físico (PostgreSQL 16)
4. Estructura de las llaves en Redis
5. Máquinas de estado de las entidades
6. Datos de siembra

08 · Compuerta de tráfico (plano de datos)

1. Contrato de entrada
2. Formato de la clave
3. Tubería de filtros
4. Contrato de errores
5. Cabeceras
6. CORS
7. Medición y consolidación
8. Rendimiento

09 · Cobros y suscripciones

1. Principios
2. Pasarela simulada
3. Máquina de estados de las suscripciones

4. Ciclos
5. Cambio de plan
6. Límites del plan de plataforma (RF-43)
7. Reversión de pagos (RF-24)
8. Quién paga a quién
9. Modo demostración

10 · Identidad y seguridad

1. Autenticación
2. Resolución de la organización (multi-tenancy)
3. Secretos y datos sensibles
4. Protección del origen (SSRF)
5. Encabezados y protección de los frontends
6. Correos (RF-46)
7. Bitácora
8. Amenazas consideradas

11 · Interfaz y mockups

1. Sistema visual oficial: variante 4, "Plano azul"
2. Formato de los mockups
3. Catálogo de pantallas
4. Estados que toda pantalla debe cubrir

12 · Registro de decisiones de arquitectura (ADR)

06 · Arquitectura

1. Estilo arquitectónico y justificación

La solución separa el **plano de control** del **plano de datos**. Se despliegan como procesos independientes y solo comparten PostgreSQL y Redis. Cada plano usa una arquitectura distinta porque sus atributos de calidad son distintos:

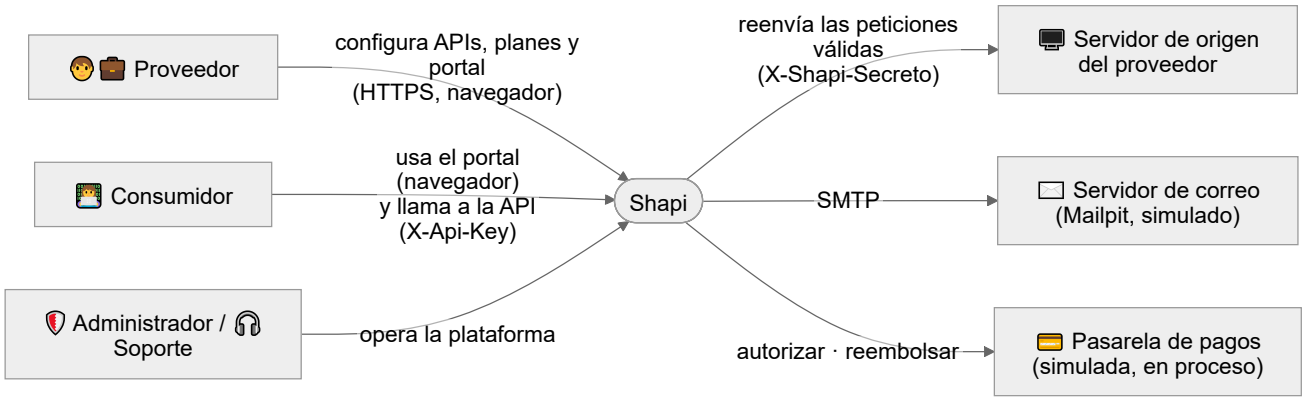
Plano	Qué prioriza	Estilo	Por qué
Datos (la compuerta)	Latencia de 15 ms o menos en el p95, disponibilidad y extensibilidad de las reglas	**Tuberías y filtros**	Cada regla es un filtro independiente que deja pasar la petición o la detiene. Una regla nueva es un filtro más (RNF-13). No consulta PostgreSQL, solo Redis (RNF-02).
Control (API de control y trabajador)	Correctitud, trazabilidad y reglas de negocio	**Tres capas**: presentación, lógica de negocio y acceso a datos	Separa la API HTTP de las reglas del dominio y de la persistencia, para poder probar las reglas sin infraestructura.

Alternativas que se descartaron (el detalle está en 12 · ADR-27):

- **Un monolito**: si la compuerta viviera en el mismo proceso que el panel, una caída o un despliegue del panel detendría el tráfico de los clientes (incumple RNF-04).
- **Microservicios por dominio**: para 4 personas y 6 semanas, cuestan más en operación de lo que aportan.
- **Una compuerta existente** (Kong, Tyk): el curso pide desarrollo *full stack* y la compuerta es el núcleo del producto. Con YARP se programa en C# y se reutiliza el modelo de dominio.

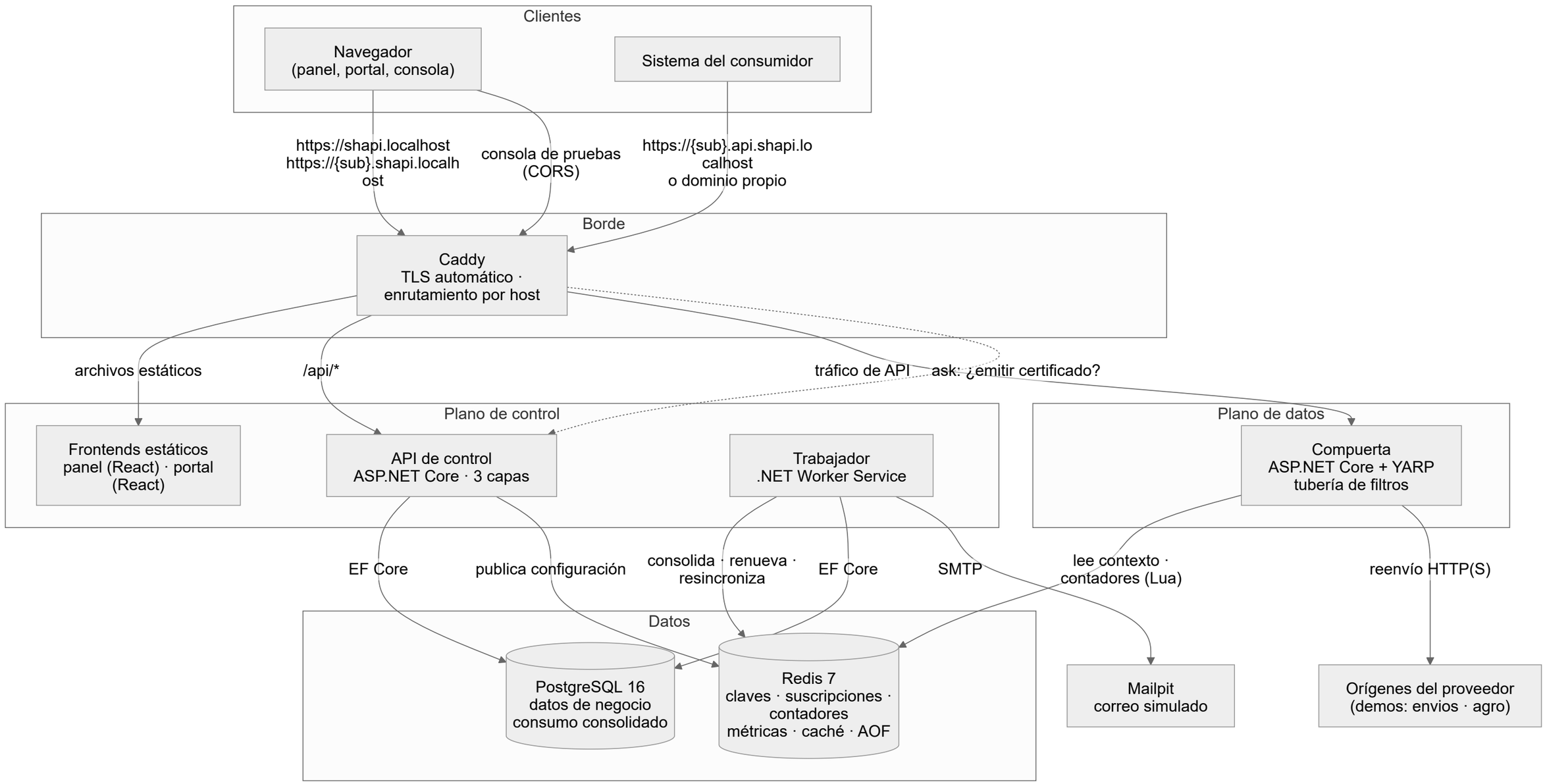
2. Diagrama de contexto

Diagrama 1 · Vista general



3. Contenedores y procesos

Diagrama 2 · Vista general



Proceso	Tecnología	Responsabilidad	Si se cae
Borde	Caddy 2	Termina TLS, emite certificados automáticamente (con la autoridad interna y *on-demand*), redirige HTTP a HTTPS, enruta por host y sirve los frontends	Se cae todo el sistema; es el único punto de entrada
Compuerta	.NET 10 + YARP	Tubería de filtros, reenvío al origen, cabeceras y métricas	Se detiene el tráfico de las APIs. El panel sigue funcionando
API de control	.NET 10, ASP.NET Core	API REST del panel, del portal y de administración; reglas de negocio; publica la configuración en Redis	No se puede administrar nada, pero **el tráfico sigue** (RNF-04)
Trabajador	.NET 10 Worker	Consolida métricas cada 10 s, cierra ciclos cada minuto, envía correos cada 5 s, verifica dominios pendientes cada 5 min y resincroniza Redis cada 5 min	Se pausan la consolidación y las renovaciones, **sin perder datos** (RNF-05)
Frontends	React 19 + TypeScript + Vite	`panel`: sitio público, panel del proveedor y administración. `portal`: portal de marca blanca, que identifica la API por el host	—
Redis	Redis 7.4, AOF `everysec`	Vista de lectura de la compuerta, contadores atómicos, métricas pendientes y caché de respuestas	Se detiene la compuerta. Al volver, el trabajador reconstruye la caché desde PostgreSQL
PostgreSQL	PostgreSQL 16	Fuente de verdad de los datos de negocio	Se detiene la administración; **la compuerta sigue funcionando**
Mailpit	Mailpit	Servidor SMTP y bandeja web de correo simulado	Los correos esperan en `correo_saliente` y B3 marca el correo como degradado

4. Hosts y enrutamiento en el borde

El dominio base se configura con `SHAPI_DOMINIO_BASE` y vale `shapi.localhost` en todos los ambientes, porque se usan dominios simulados (ADR-09).

Host	Destino	Contenido
`shapi.localhost`	Frontend `panel` (estático) + `/api/*` → API de control	Sitio público (`/`, `/registro`, `/entrar`, …), panel del proveedor (`/panel/*`) y administración (`/admin/*`)
`{sub}.shapi.localhost`	Frontend `portal` (estático) + `/api/portal/*` → API de control	Portal de marca blanca de la API con ese subdominio
`{sub}.api.shapi.localhost`	Compuerta	Tráfico de la API
Dominio propio verificado, por ejemplo `api.enviosxelaju.localhost`	Compuerta	Tráfico de la API. El certificado se emite *on-demand*
`correo.shapi.localhost`	Mailpit (puerto 8025)	Bandeja de correo simulado. Solo existe en los ambientes de desarrollo y de demostración

Subdominios reservados, que no se pueden usar como `{sub}`: `api`, `app`, `www`, `admin`, `panel`, `correo`, `mail`, `soporte`, `docs`, `estado`, `status`, `shapi`, `static`, `cdn`, `interno`.

La API de control expone además `/interno/*`, que **solo es accesible desde la red de Docker** porque Caddy no lo publica:

- `GET /interno/tls/autorizar?domain=…`: el *ask* de *on-demand TLS*. Responde 200 si el dominio es propio y está verificado, y 404 en cualquier otro caso.

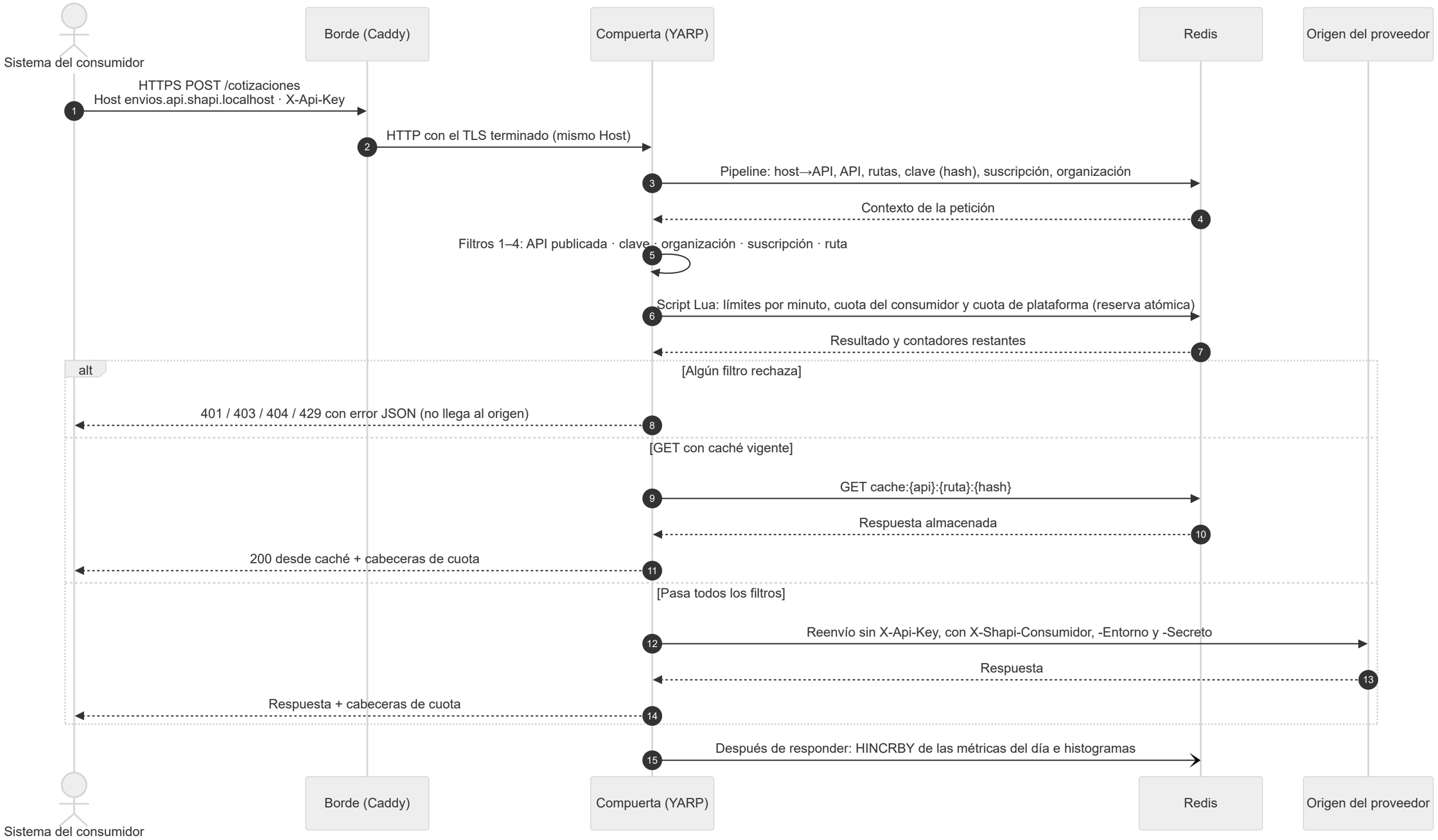
La salud de la API y de la compuerta se consulta en su propio `/salud` (§8), dentro del propio contenedor (host `localhost`) o en la máquina; Caddy no lo publica.

La **consola de pruebas** llama desde `https://{sub}.shapi.localhost` a `https://{sub}.api.shapi.localhost`, así que la compuerta responde el *preflight* CORS **sin pedir clave**, y solo permite como origen el host del portal de esa API (08 §6).

5. Diagramas de secuencia

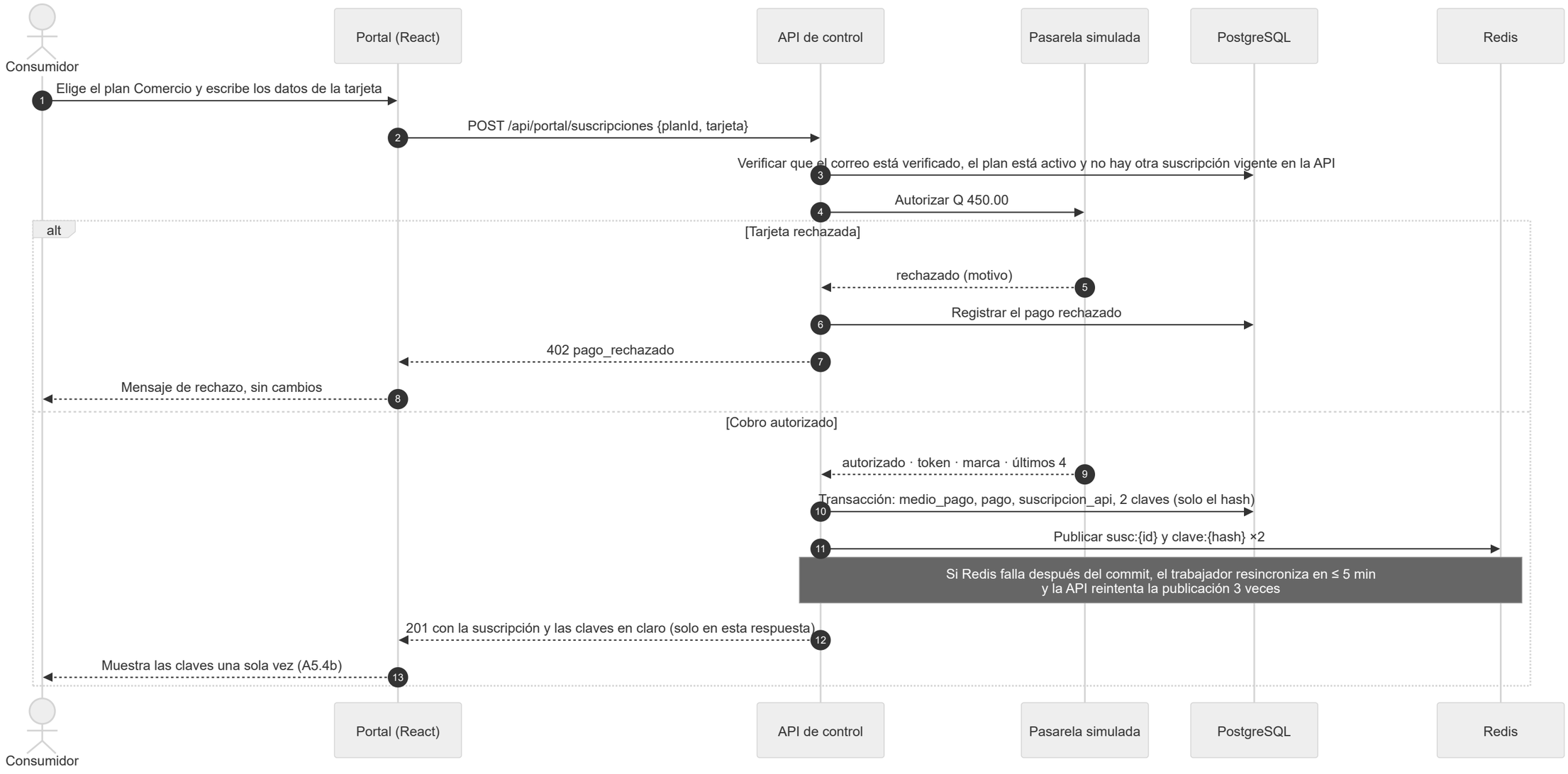
5.1 Procesamiento de una petición en la compuerta

Diagrama 3 · Vista general



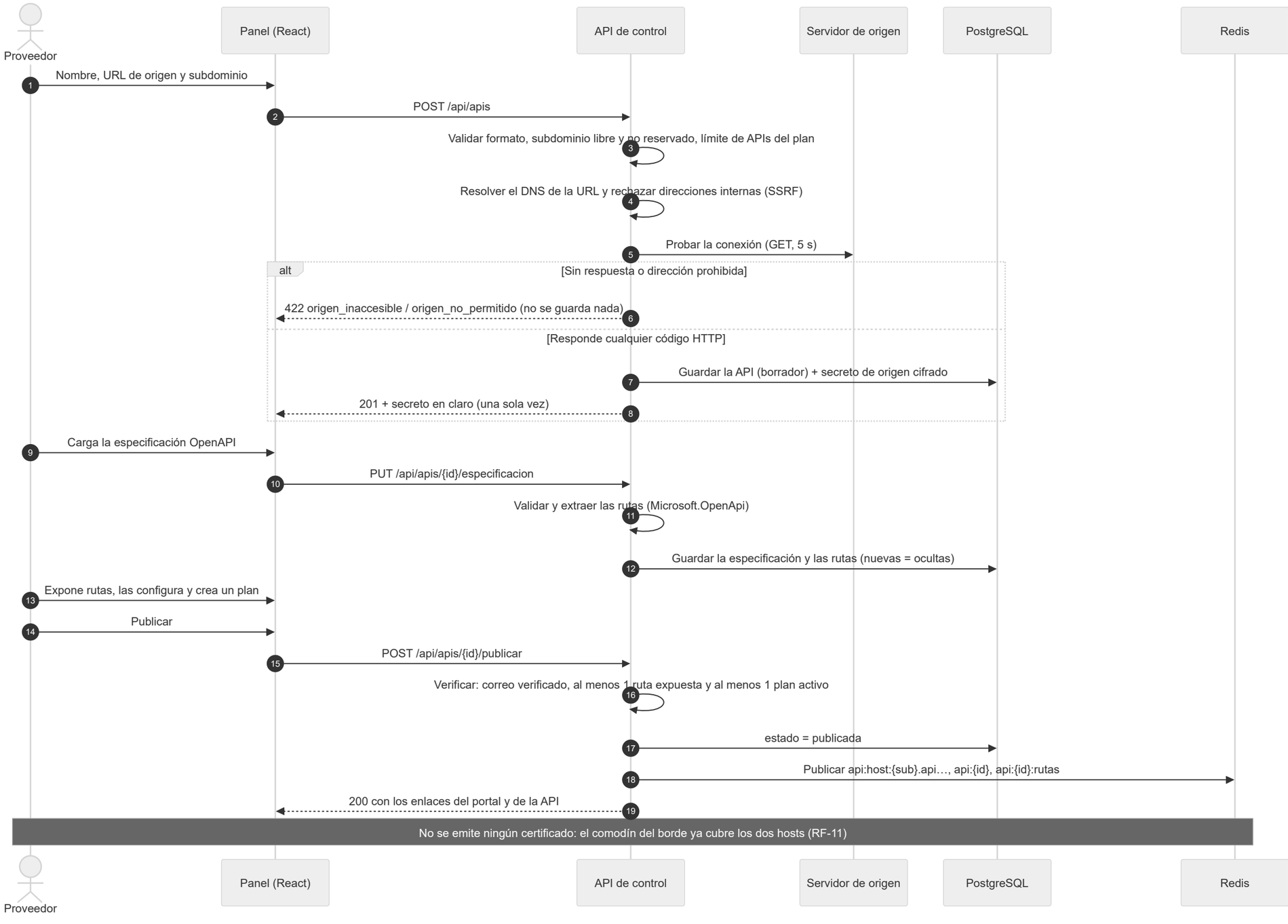
5.2 Contratación de un plan de API

Diagrama 4 · Vista general



5.3 Publicación de una API

Diagrama 5 · Vista general

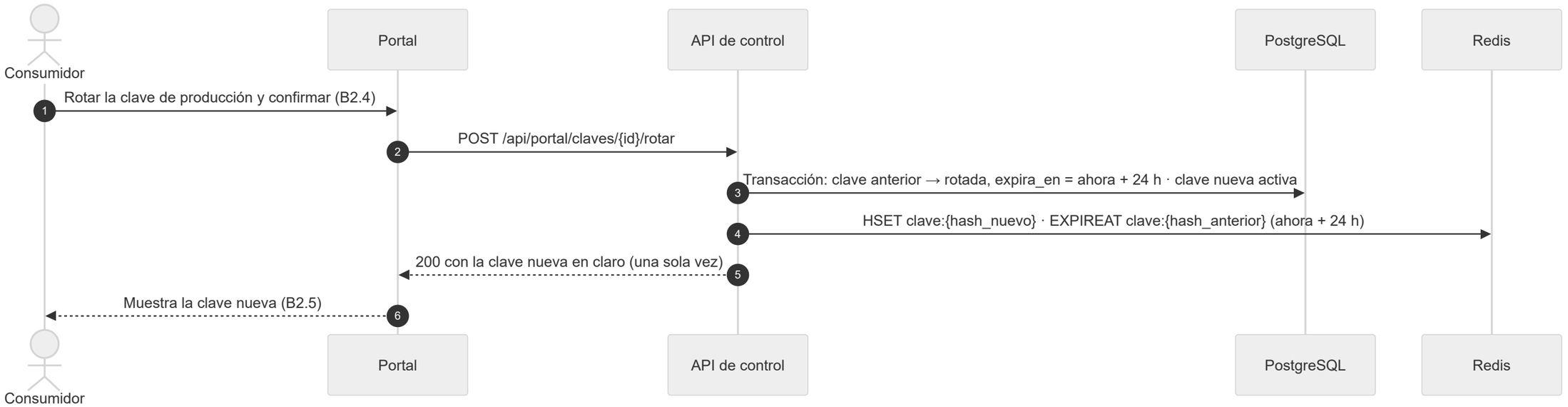


Respuestas del registro que el diagrama no detalla (DC-04):

- un subdominio reservado o con un formato inválido responde **400** `datos_invalidos`, con el error en `errores.subdominio`. Los campos se recortan antes de validarse;
- si el DNS de la URL de origen no resuelve, responde **422** `origen_inaccesible`, igual que si el origen no responde. Si la URL está mal formada o resuelve a una dirección interna, **422** `origen_no_permitido`. En los dos casos, `detalle.motivo` explica la causa y A3.2 la muestra.

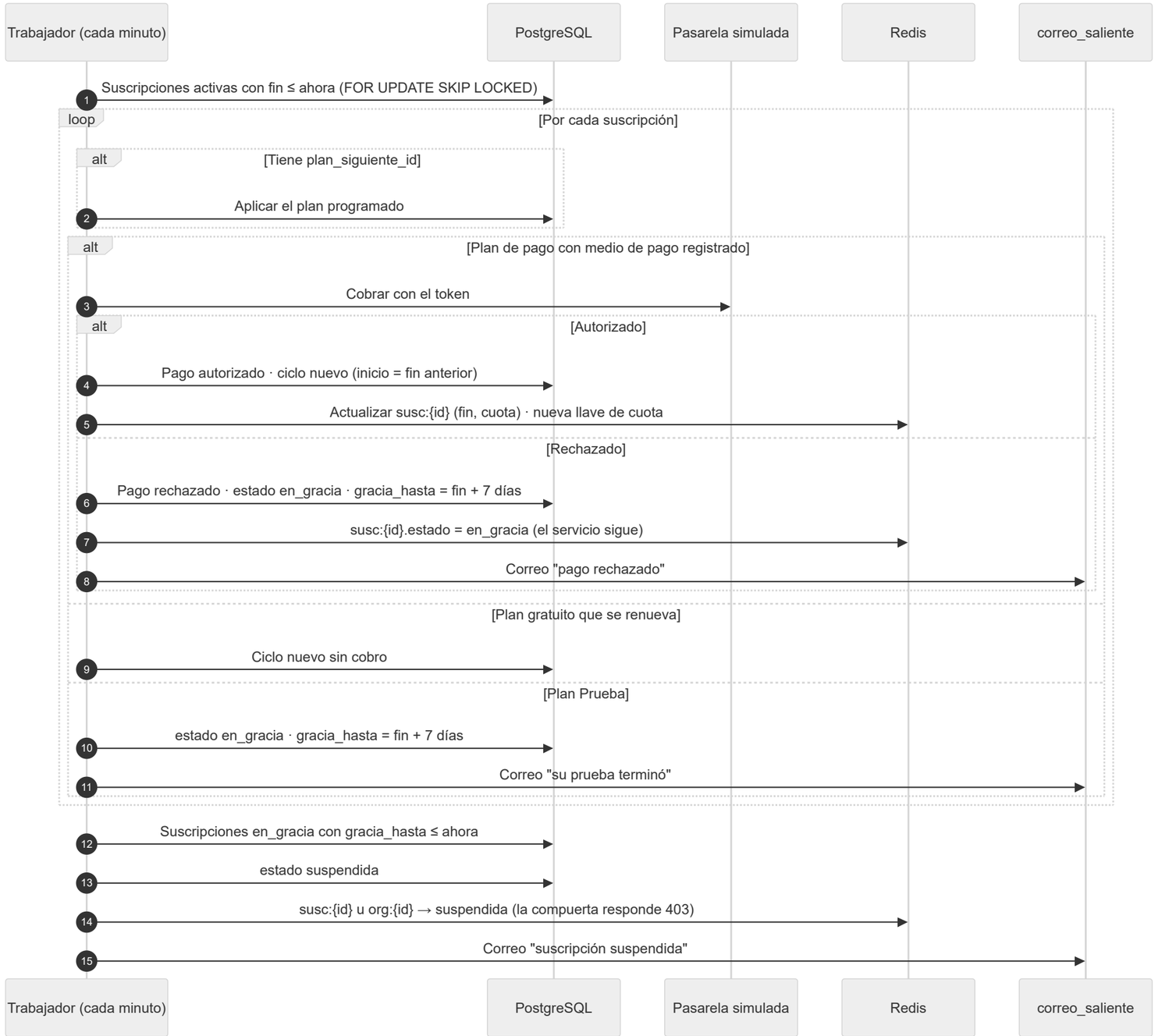
5.4 Rotación de una clave

Diagrama 6 · Vista general



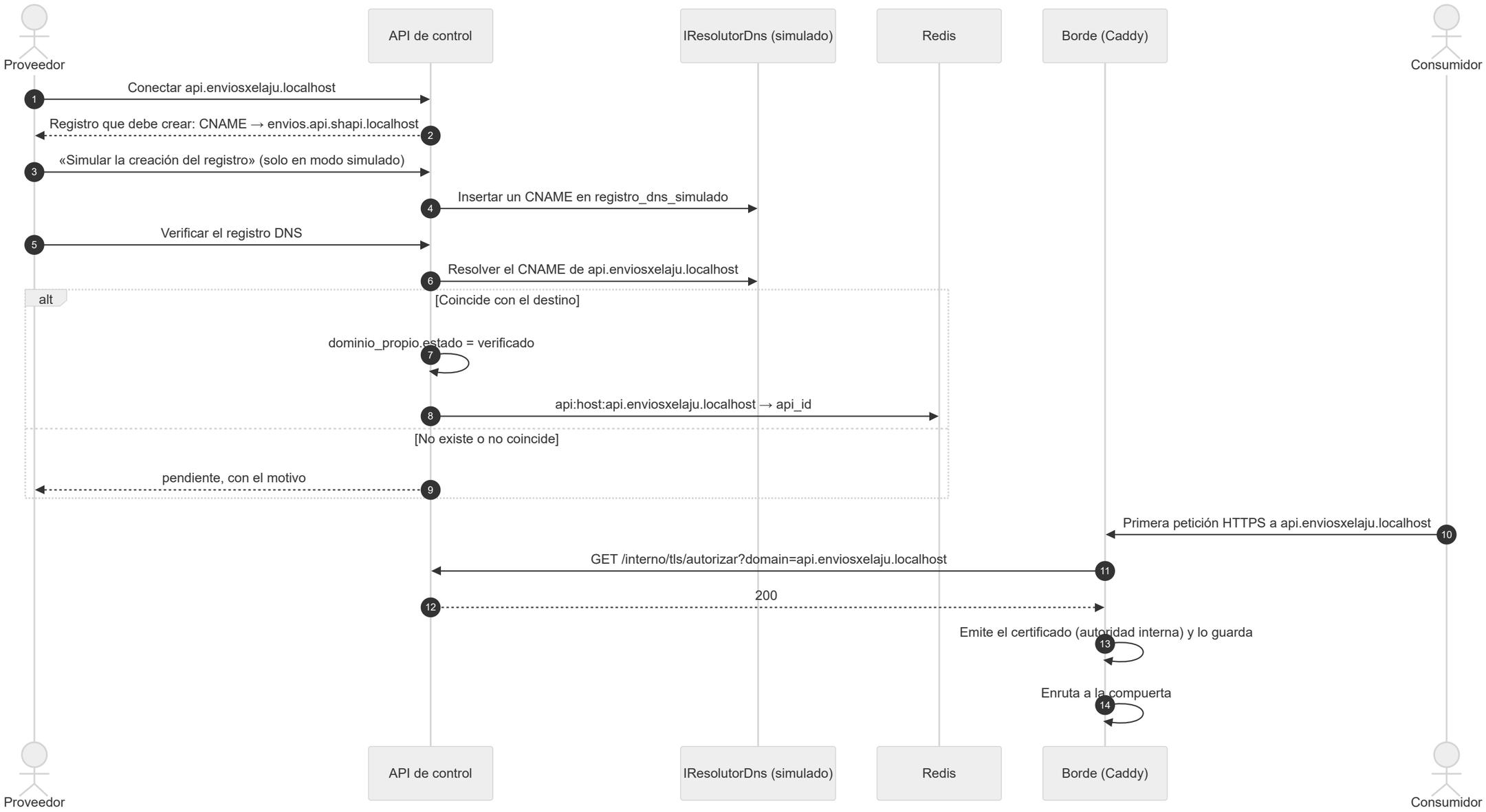
5.5 Cierre de ciclo, renovación, gracia y suspensión

Diagrama 7 · Vista general



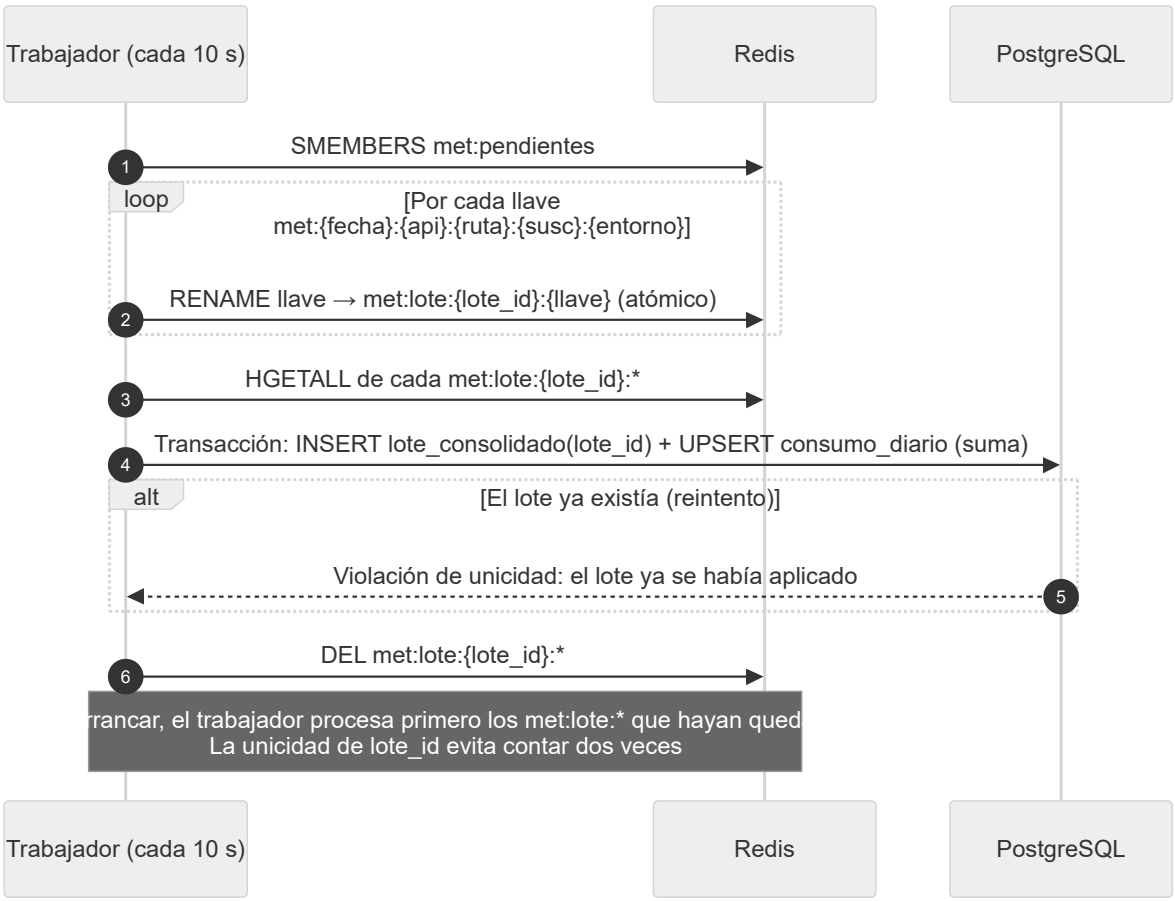
5.6 Verificación de un dominio propio y emisión del certificado

Diagrama 8 · Vista general



5.7 Consolidación del consumo

Diagrama 9 · Vista general



6. Dominios simulados

El proyecto tiene presupuesto Q 0.00 (ADR-09), así que:

1. **Resolución de nombres:** los navegadores actuales (Chrome, Edge y Firefox) resuelven cualquier `*.localhost` a `127.0.0.1` sin configurar nada (RFC 6761). Por eso todos los hosts de la plataforma cuelgan de `shapi.localhost`.
2. **Certificados:** Caddy usa su **autoridad certificadora interna** (`local_certs`). Emite al instante los comodines `*.shapi.localhost` y `*.api.shapi.localhost`, y los certificados *on-demand* de los dominios propios, y los renueva solo. Cada equipo instala una vez la raíz de Caddy como confiable, importando su `root.crt` en el almacén del equipo (`caddy trust` dentro del contenedor solo la instala en el contenedor, no en el navegador); el manual técnico (`docs/manual-tecnico.md`, se escribe durante el desarrollo) explica cómo.
3. **Dominios propios:** en el entorno simulado deben terminar en `.localhost` (por ejemplo `api.enviosxelaju.localhost`) para que resuelvan al equipo local. La verificación usa `IResolutorDns`:
    - `SHAPI_DNS_MODO=simulado`, el valor por defecto: consulta la tabla `registro_dns_simulado`. El botón "Simular la creación del registro" de A3.6 inserta el CNAME, haciendo el papel del proveedor de DNS del cliente.

- `SHAPI_DNS_MODO=real`: consulta el DNS público con DnsClient.NET. Queda disponible por si algún día hay un dominio real.

4. **Correo:** Mailpit hace de servidor SMTP y de bandeja web (`https://correo.shapi.localhost`). Si se quiere usar un servidor SMTP real, basta con configurar las variables `SHAPI_SMTP_*`.
5. **Clientes fuera del navegador:** las versiones actuales de `curl` también resuelven `*.localhost` a loopback. Para otros clientes se puede usar `curl --resolve` o una entrada en el archivo `hosts`.

7. Despliegue

7.1 Ambientes

Ambiente	Cómo se levanta	Diferencias
Desarrollo	`docker compose up` con los servicios de infraestructura, y los proyectos .NET y Vite corriendo en el equipo con recarga en caliente	`SHAPI_MODO_DEMO=true`, registros detallados y siembra de demostración
Productivo simulado (el que se presenta)	`docker compose -f compose.yml -f compose.prod.yml up -d`, con **imágenes publicadas en GHCR** por la integración continua	Compilación *Release*, frontends minificados, `ASPNETCORE_ENVIRONMENT=Production`, variables desde `.env`, volúmenes persistentes, reinicio automático, verificaciones de salud y sin puertos internos expuestos. `SHAPI_MODO_DEMO=true` solo para la exposición
Pruebas (CI)	GitHub Actions, con Testcontainers para PostgreSQL y Redis	Se ejecuta en cada *pull request*

El "ambiente productivo" que piden los lineamientos se cumple con el **mismo stack**, contenedores versionados, configuración externa, TLS, persistencia y verificaciones de salud. Como el presupuesto es cero, corre en un equipo del equipo de trabajo el día de la exposición, y no en un servidor público.

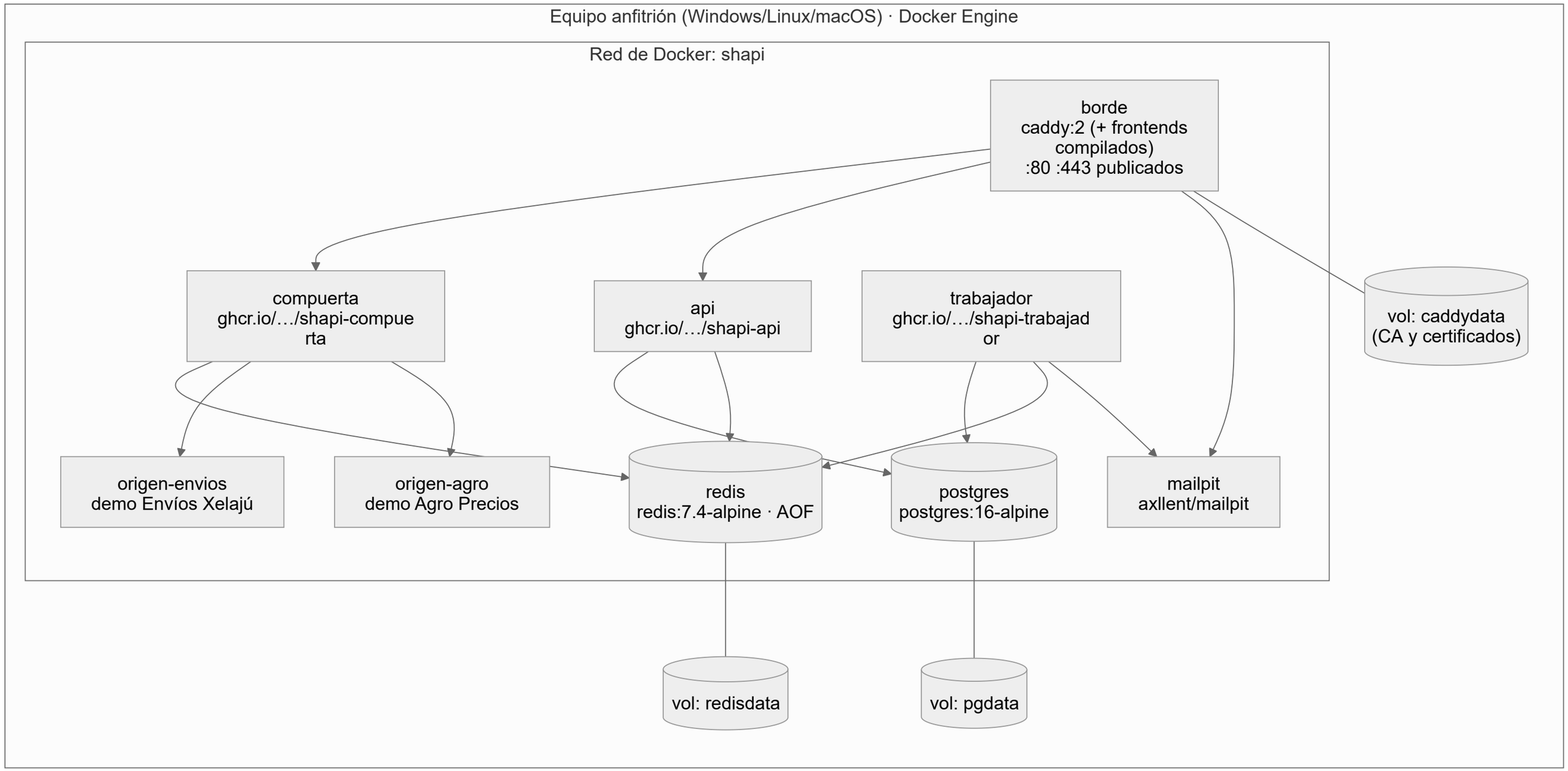
Para levantar el ambiente productivo simulado con los datos de los mockups en un solo comando (RNF-14), desde la raíz y con `.env` preparado:

```
docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml --profile demo up -d --build
```

El perfil `demo` agrega `siembra-demo`, de una sola ejecución: espera la salud de la API tras las migraciones y reutiliza la imagen, configuración y llaves del trabajador. Repetir el comando no duplica datos. Como el arranque es separado (`-d`), la demostración está lista cuando `siembra-demo` termina con código 0; `node infra/verificar.mjs --demo` verifica la clave fija y el portal de Envíos Xelajú. Sin el perfil no se siembra; un despliegue real usa `SHAPI_MODO_DEMO=false` y volúmenes sin datos de demostración. La siembra rechaza ese modo aunque se active el perfil por error.

7.2 Diagrama de despliegue

Diagrama 10 · Vista general



Los orígenes de demostración (`origen-envios` y `origen-agro`) son dos APIs mínimas con su especificación OpenAPI y los datos de los mockups. Sirven para demostrar el sistema de punta a punta (ADR-33). En la siembra, sus URL de origen son `http://origen-envios:8080` y `http://origen-agro:8080`, que están en la red de Docker. Por eso la **lista de redes permitidas** de la protección contra SSRF incluye esos dos nombres de servicio **solo** cuando `SHAPI_MODO_DEMO=true`.

7.3 Integración continua

- En cada *pull request*: compilar, ejecutar las pruebas del backend (xUnit con Testcontainers) y del frontend (Vitest), revisar formato y *lint*, y hacer un *build* de las imágenes. Los jobs `backend` y `frontend` siempre corren, pero omiten sus pasos si el PR no cambia nada que lean sus verificaciones: lo decide `scripts/cambios-ci.mjs` con una lista de rutas que seguro no les afectan, y cualquier otra ruta, un archivo movido (con su ruta de origen y la de destino) o un cambio en `ci.yml` o en el propio decisor ejecuta todo. En cada *merge* a `main` se verifica todo. El decisor es parte del PR, como `ci.yml`: un cambio en él lo revisan `revision-claude` y la CI completa de `main`.
- En cada *merge* a `main`: publicar las imágenes `shapi-api`, `shapi-compuerta`, `shapi-trabajador` y `shapi-borde` en GHCR, que es gratis para repositorios públicos.
- Pruebas de extremo a extremo con Playwright contra el ambiente productivo simulado: en cada *pull request*, en el ambiente que se levanta para verificarlo, y en cada *merge* a `main`.

8. Observabilidad y estado de componentes

- La API de control y la compuerta exponen `/salud`, con verificaciones de salud de ASP.NET Core. La compuerta solo lo responde con el host `localhost`, para no tapar una ruta `/salud` de las APIs de los proveedores. El trabajador no tiene HTTP: su salud es el latido de abajo. Los tres procesos escriben registros estructurados en JSON en la salida estándar.
- La compuerta y el trabajador escriben cada 10 segundos un **latido** en Redis (`salud:compuerta:{instancia}` y `salud:trabajador`, con un TTL de 30 s).
- La API de control calcula el estado de B3.1 cada 30 segundos:

Componente	En servicio	Degradado	Fuera de servicio
Compuerta de APIs	Latido vigente y p95 de la última hora ≤ 15 ms	Latido vigente y p95 > 15 ms	Sin latido
API de control	Siempre, si puede responder	Tarda más de 1 s en responder `/salud`	— (si está caída, B3 no carga)
Trabajador en segundo plano	Latido vigente y la consolidación va al día (atraso < 60 s)	Atraso de la consolidación ≥ 60 s	Sin latido
Caché (Redis)	`PING` < 50 ms	`PING` ≥ 50 ms	No responde
Base de datos (PostgreSQL)	Consulta de prueba < 200 ms	≥ 200 ms	No responde
Pasarela de pagos simulada	Siempre	—	Si `SHAPI_PASARELA_FALLA=true` (para demostraciones)

Componente	En servicio	Degradado	Fuera de servicio
Envío de correo	No hay correos pendientes con más de 2 minutos	Hay correos pendientes con más de 2 minutos o fallos recientes	El servidor SMTP no responde

Precisiones de B3.1 (RF-39): `GET /api/admin/estado` admite administrador y soporte. La comprobación se comparte durante 30 segundos por proceso de API, en memoria para seguir funcionando si Redis cae. Las sondas tienen plazos acotados y fallan independientemente. La API consulta su `/salud` por HTTP local, sin tomar la dirección de la cabecera Host. El correo comprueba la conexión SMTP y `NOOP`, sin enviar mensajes; «fallos recientes» son correos no enviados con error y `actualizado_en` en los últimos dos minutos. La fecha del aviso de correo es la creación del correo pendiente más antiguo o, si solo hay fallos recientes, la actualización más antigua de esos fallos.

Los latidos se comprueban por existencia con TTL positivo, sin comparar sus marcas de tiempo: el trabajador usa `IReloj`, que puede estar adelantado en demostración. Sin métricas pendientes de APIs identificadas ni lotes separados pendientes, la ausencia de un lote reciente no degrada al trabajador. Con trabajo pendiente, se compara `lote_consolidado.procesado_en` con `IReloj`; si todavía no existe un lote, la consolidación se considera degradada hasta poder comprobarla.

El p95 global de la última hora usa las muestras de todas las instancias y peticiones, incluidos rechazos, en una ventana móvil de 3600 segundos del reloj real. Se registra al completar la petición y se interpola en los mismos diez rangos de 08 §7; sin muestras es nulo, y el rango abierto se presenta como `> 2500 ms`. La fecha de comprobación y los datos de negocio usan `IReloj`. B3.1 actualiza automáticamente cada 30 segundos; solo afirma que la compuerta no está afectada cuando su estado comprobado es `en_servicio`.

Si PostgreSQL no responde, no se puede comprobar la cola: el correo se muestra degradado mientras SMTP responda, o fuera de servicio si SMTP tampoco responde. Los avisos de otros componentes muestran su nombre y «está degradado» o «está fuera de servicio», con el texto «El componente requiere atención. El estado se volverá a comprobar automáticamente.»; estos estados no tienen un mockup propio.

9. Estructura del repositorio

```
shapi/
├─ docs/
│  ├─ specs/            ← estas especificaciones (fuente de verdad)
│  ├─ lineamientos.md   ← reglas oficiales del curso
│  ├─ plan/             ← plan de desarrollo: guía, instalación, protocolo, convenciones, calendario, tareas
y bitácoras
│  ├─ pdf/              ← documentos entregables generados en PDF
│  ├─ manual-tecnico.md
│  └─ manual-usuario.md
├─ mockups/             ← diseños aprobados (ver 11 · Interfaz)
├─ contratos/openapi/   ← contratos HTTP de la API de control, uno por módulo
├─ scripts/             ← tareas.mjs, tablero.mjs, veredicto-revision.mjs, cambios-ci.mjs, verificar-
entorno.mjs, generar-pdf.mjs
├─ AGENTS.md            ← instrucciones para cualquier agente de IA (CLAUDE.md lo importa)
├─ src/
│  ├─ Shapi.Dominio/          ← capa de lógica: entidades, reglas y máquinas de estado (sin dependencias)
│  ├─ Shapi.Aplicacion/       ← capa de lógica: casos de uso e interfaces (IPasarelaPagos, IResolutorDns,
IReloj, IColaCorreo, IPublicadorCache)
│  ├─ Shapi.Infraestructura/  ← capa de datos: EF Core (Npgsql), migraciones, Redis, pasarela simulada,
SMTP, DNS
│  ├─ Shapi.Api/              ← capa de presentación: API REST (panel, portal, administración, interno)
```

```
│   ├─ Shapi.Contratos/        ← compartido: formato de las llaves de Redis, códigos de error, DTO de la
caché
│   ├─ Shapi.Compuerta/        ← plano de datos: YARP + filtros (depende solo de Contratos y de
StackExchange.Redis)
│   └─ Shapi.Trabajador/       ← procesos en segundo plano
├─ tests/
│   ├─ Shapi.Dominio.Tests/
│   ├─ Shapi.Api.Tests/        ← integración con Testcontainers, incluido el aislamiento entre organizaciones
│   ├─ Shapi.Compuerta.Tests/  ← un conjunto de pruebas por filtro, más la tubería completa
│   └─ e2e/                    ← Playwright
├─ frontend/                   ← pnpm workspace
│   ├─ apps/panel/             ← sitio público, panel y administración
│   ├─ apps/portal/            ← portal de marca blanca
│   └─ packages/ui/            ← componentes y tokens de diseño (variante 4)
├─ origenes-demo/
│   ├─ envios-xelaju/
│   └─ agro-precios/
├─ infra/
│   ├─ caddy/Caddyfile.dev       ← desarrollo (JZ-01)
│   ├─ caddy/Caddyfile.prod      ← ambiente productivo simulado (JZ-06)
│   ├─ compose.yml
│   └─ compose.prod.yml
└─ .github/workflows/
```

Stack definitivo

Capa	Tecnología
Backend	.NET 10 (LTS), ASP.NET Core Minimal APIs, YARP 2, EF Core 10 + Npgsql, StackExchange.Redis, Microsoft.OpenApi, DnsClient.NET, MailKit, FluentValidation
Frontend	React 19, TypeScript, Vite, React Router, TanStack Query, Tailwind CSS 4 con los tokens de la variante 4, Recharts
Datos	PostgreSQL 16, Redis 7.4
Borde	Caddy 2
Pruebas	xUnit, Testcontainers, FluentAssertions, Vitest, Testing Library, Playwright, k6 (carga, para RNF-01 y RNF-03)
Infraestructura	Docker Compose, GitHub Actions, GHCR
Paquetes de apoyo (autorizados por JG-01 y DC-01)	NuGet: `EFCore.NamingConventions`, `Microsoft.Extensions.Identity.Core` (`PasswordHasher`), `Microsoft.Extensions.Hosting`, `Microsoft.AspNetCore.OpenApi`, `Microsoft.OpenApi.YamlReader`, `FluentValidation.DependencyInjectionExtensions`, `Microsoft.EntityFrameworkCore.Design` (migraciones), `NSubstitute`, `Microsoft.AspNetCore.Mvc.Testing`. npm: `openapi-fetch`, `openapi-typescript`, `msw`, `react-markdown`, `rehype-sanitize`, `@fontsource/sora`, `@fontsource/ibm-plex-sans`, `eslint` y sus complementos, `jsdom`, `@testing-library/user-event`

Justificación de cada tecnología

Tecnología	Por qué se eligió	Dónde se usa
.NET 10 con ASP.NET Core	Todo el backend queda en un solo lenguaje (C#). Trae de fábrica inyección de dependencias, autenticación, limitación de peticiones y servicios en segundo plano	API de control, compuerta y trabajador

Tecnología	Por qué se eligió	Dónde se usa
YARP	Biblioteca de Microsoft para construir proxies inversos sobre ASP.NET Core. Resuelve el reenvío al origen, la transformación de cabeceras y el cambio de rutas sin reiniciar. Sobre ella se monta la tubería de filtros	Compuerta
Entity Framework Core + Npgsql	Traduce las clases del dominio a tablas, maneja las migraciones y aplica el filtro global por organización (RNF-08)	Capa de acceso a datos
PostgreSQL 16	Relacional, transaccional y gratuito. Tiene `NULLS NOT DISTINCT`, arreglos para los histogramas y `jsonb`	Fuente de verdad de los datos de negocio
Redis 7.4	Guarda los datos en memoria y ofrece operaciones atómicas y scripts Lua para contar sin condiciones de carrera. Persiste con AOF	Vista de lectura de la compuerta, contadores, métricas y caché
React 19 + Vite + Tailwind	Componentes reutilizables para el panel y el portal. La marca de cada portal se aplica con variables CSS	Frontends `panel` y `portal`
Caddy 2	HTTPS automático con autoridad interna y certificados *on-demand* para los dominios propios, sin clientes ACME ni scripts (ADR-09)	Borde
OpenAPI (Microsoft.OpenApi)	Estándar para describir APIs. De él se extraen las rutas, la documentación del portal y los campos de la consola	API de control
Mailpit	Servidor SMTP con bandeja web, gratuito. Simula el correo sin depender de terceros	Correo simulado
Docker Compose	Levanta todo el sistema con un solo comando e idéntico en cualquier equipo (RNF-14)	Desarrollo y ambiente productivo simulado
Git, GitHub, Actions y GHCR	Control de versiones, revisión por *pull request*, integración continua y registro de imágenes, gratis en repositorios públicos	Trabajo del equipo y despliegue

07 · Modelo de datos

El modelo se apoya en cinco decisiones:

1. Hay **dos ámbitos de identidad**: `usuario` para el personal y `consumidor` para los clientes de cada proveedor, este último aislado por organización (ADR-03).
2. Hay **dos jerarquías de suscripción** independientes: la del proveedor con la plataforma y la del consumidor con una API. Las dos usan la misma máquina de estados.
3. El **subdominio, el portal y la marca son de cada API** (ADR-07).
4. El consumo se guarda **ya consolidado por día**, con histogramas de latencia; nunca hay una fila por petición (ADR-23).
5. Las claves y los tokens se guardan **con hash**. El secreto de origen se guarda **cifrado**. De las tarjetas solo se guarda el **token** que entrega la pasarela.

1. Diagrama de clases del dominio

Diagrama 11 · Vista general

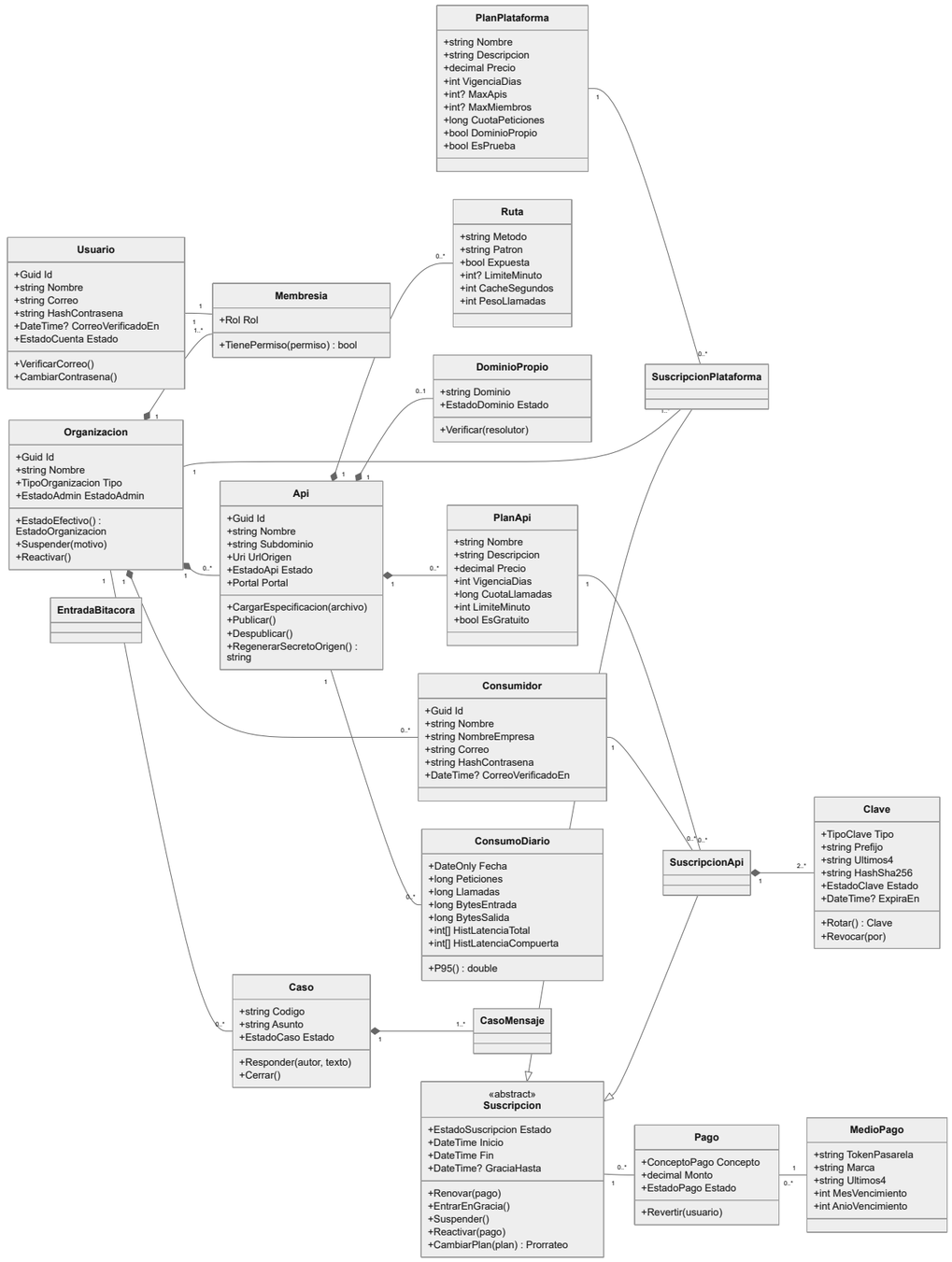


Diagrama 11 · Detalle 1 de 3 · Fila 1, columna 1

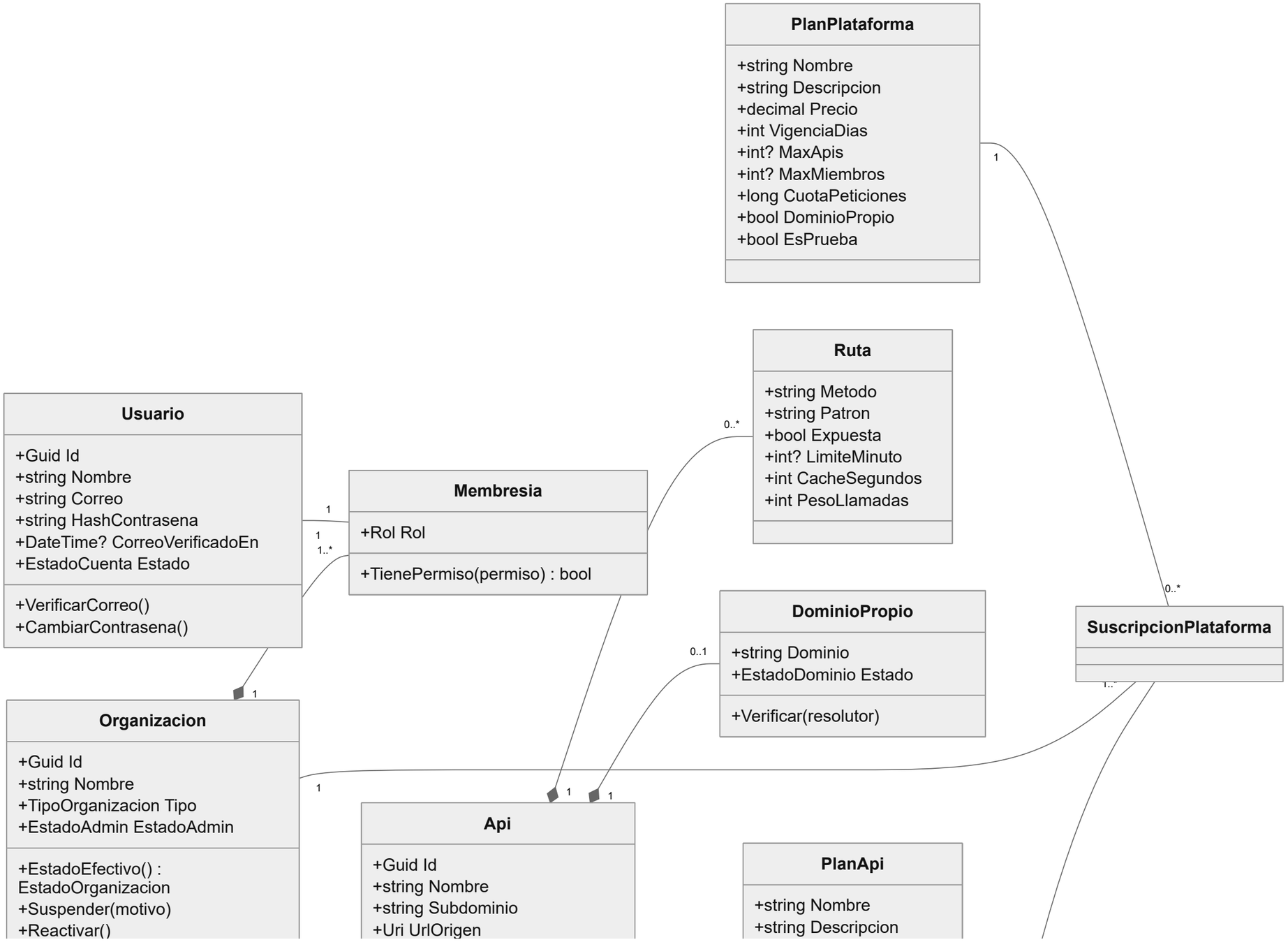


Diagrama 11 · Detalle 2 de 3 · Fila 2, columna 1

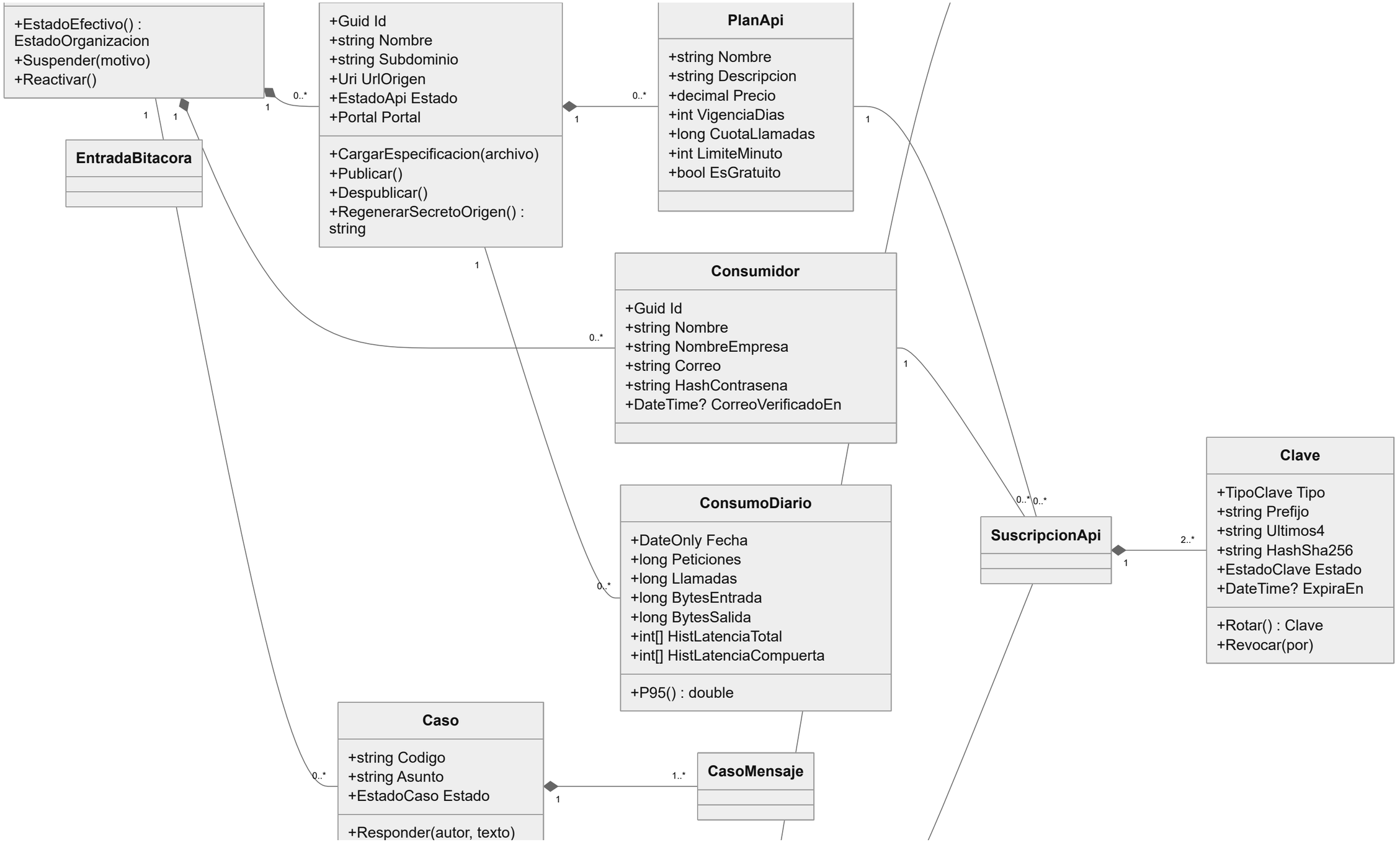
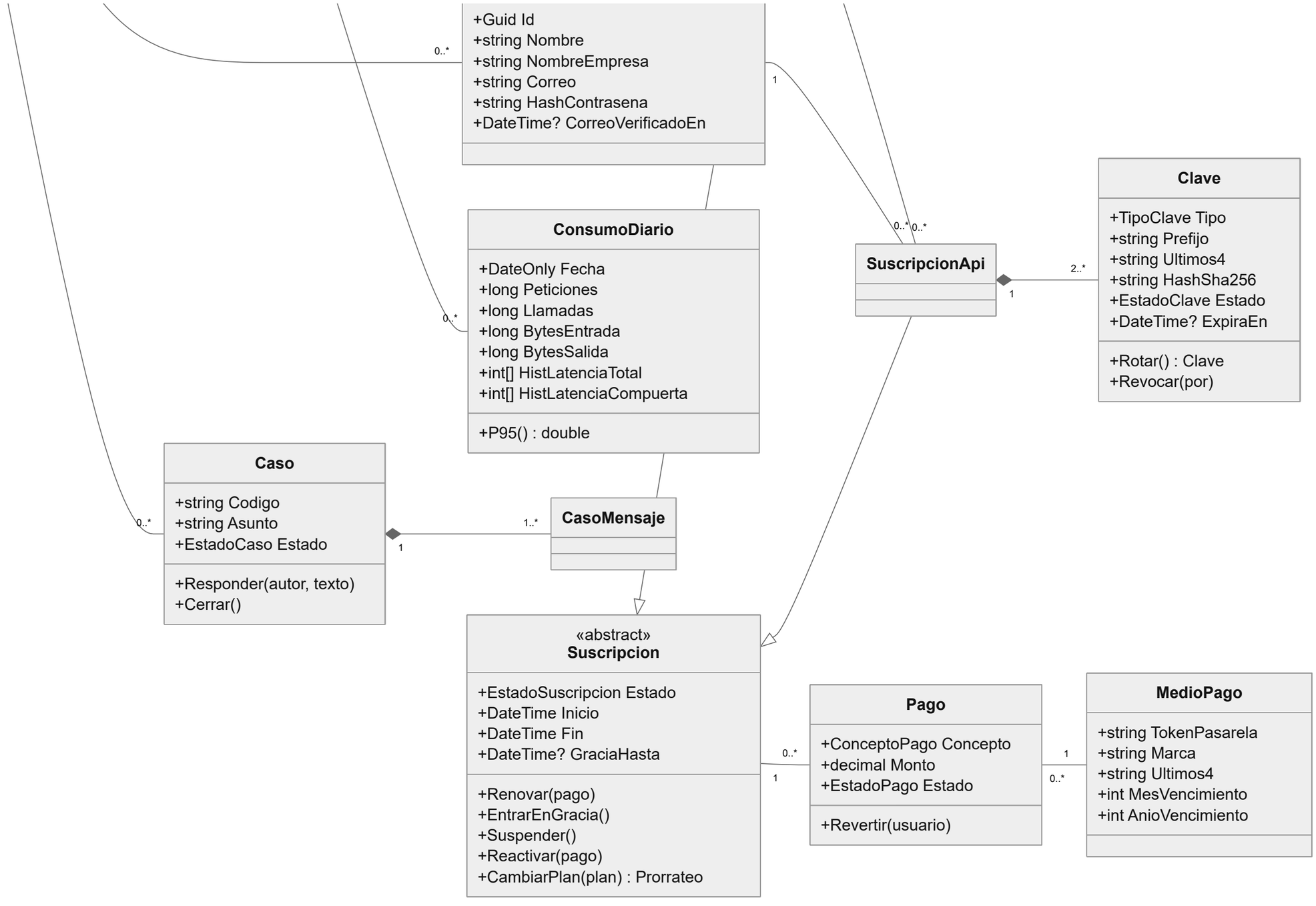


Diagrama 11 · Detalle 3 de 3 · Fila 3, columna 1



2. Modelo entidad-relación

Diagrama 12 · Vista general

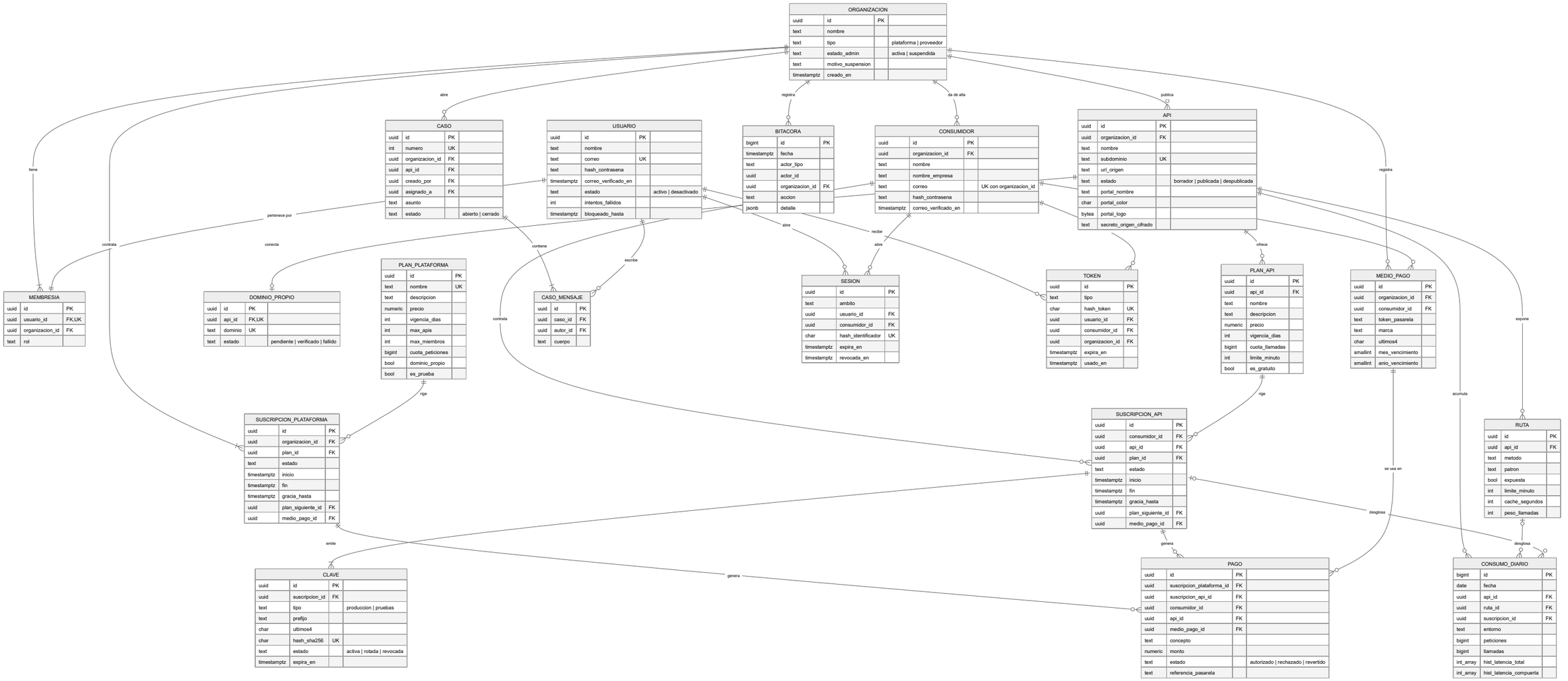


Diagrama 12 · Detalle 1 de 12 · Fila 1, columna 1

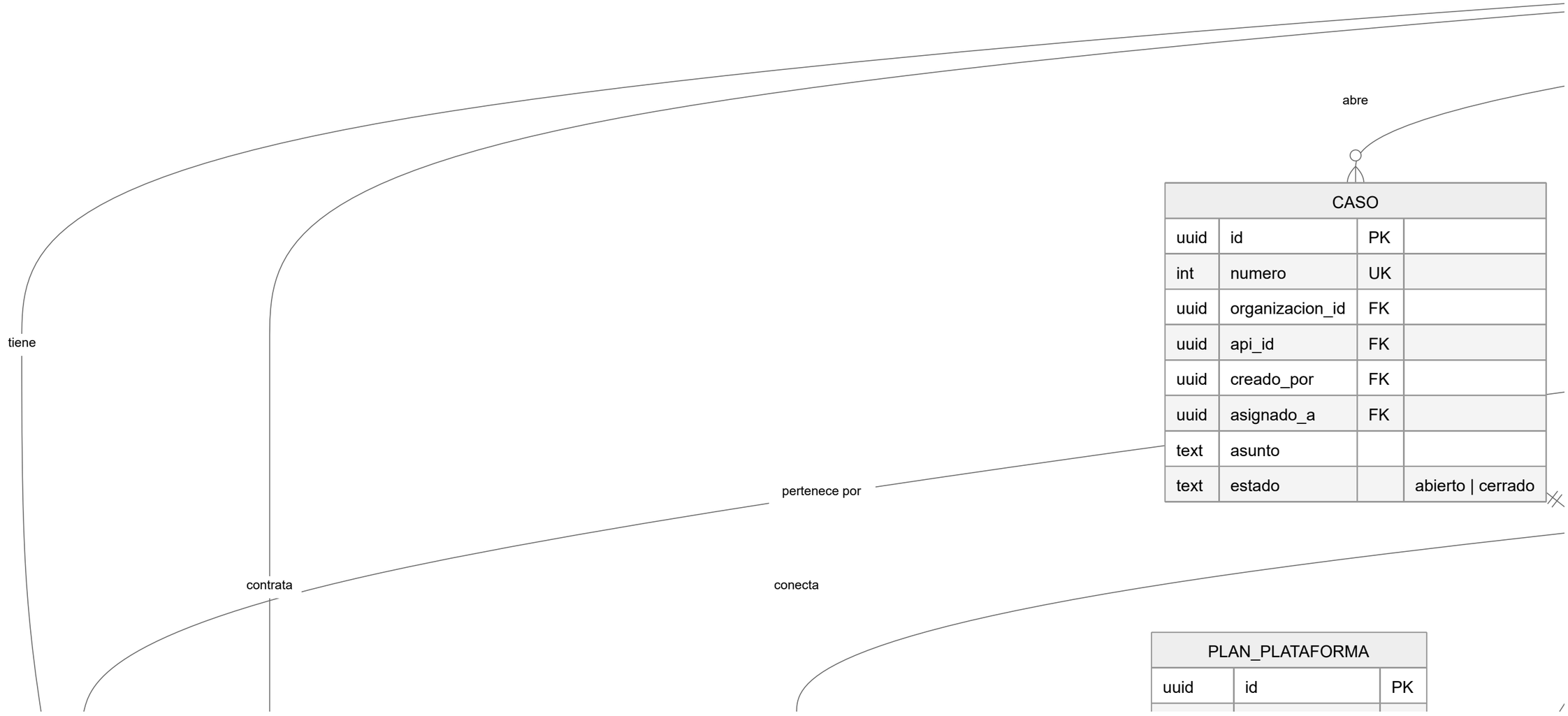


Diagrama 12 · Detalle 2 de 12 · Fila 1, columna 2

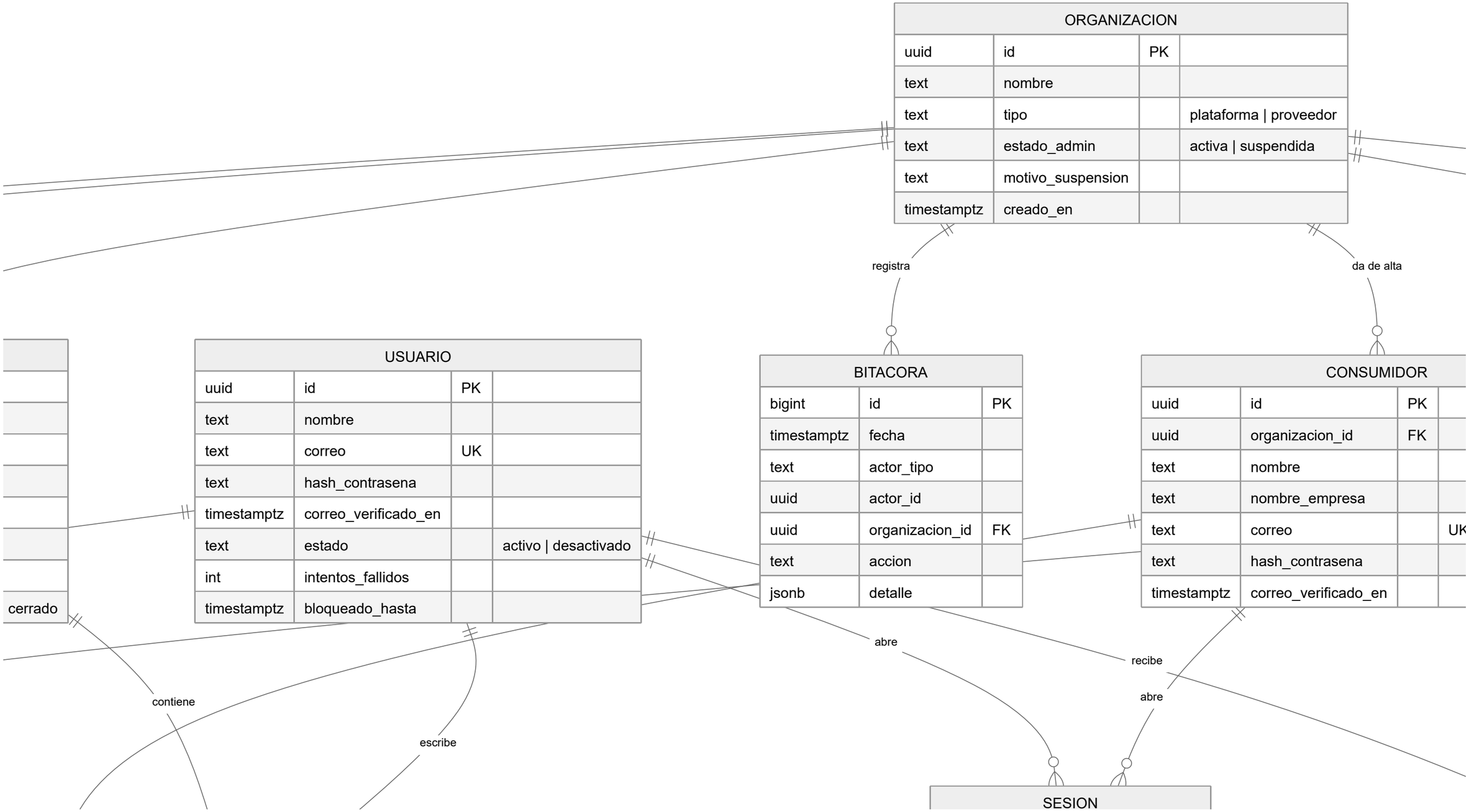


Diagrama 12 · Detalle 3 de 12 · Fila 1, columna 3

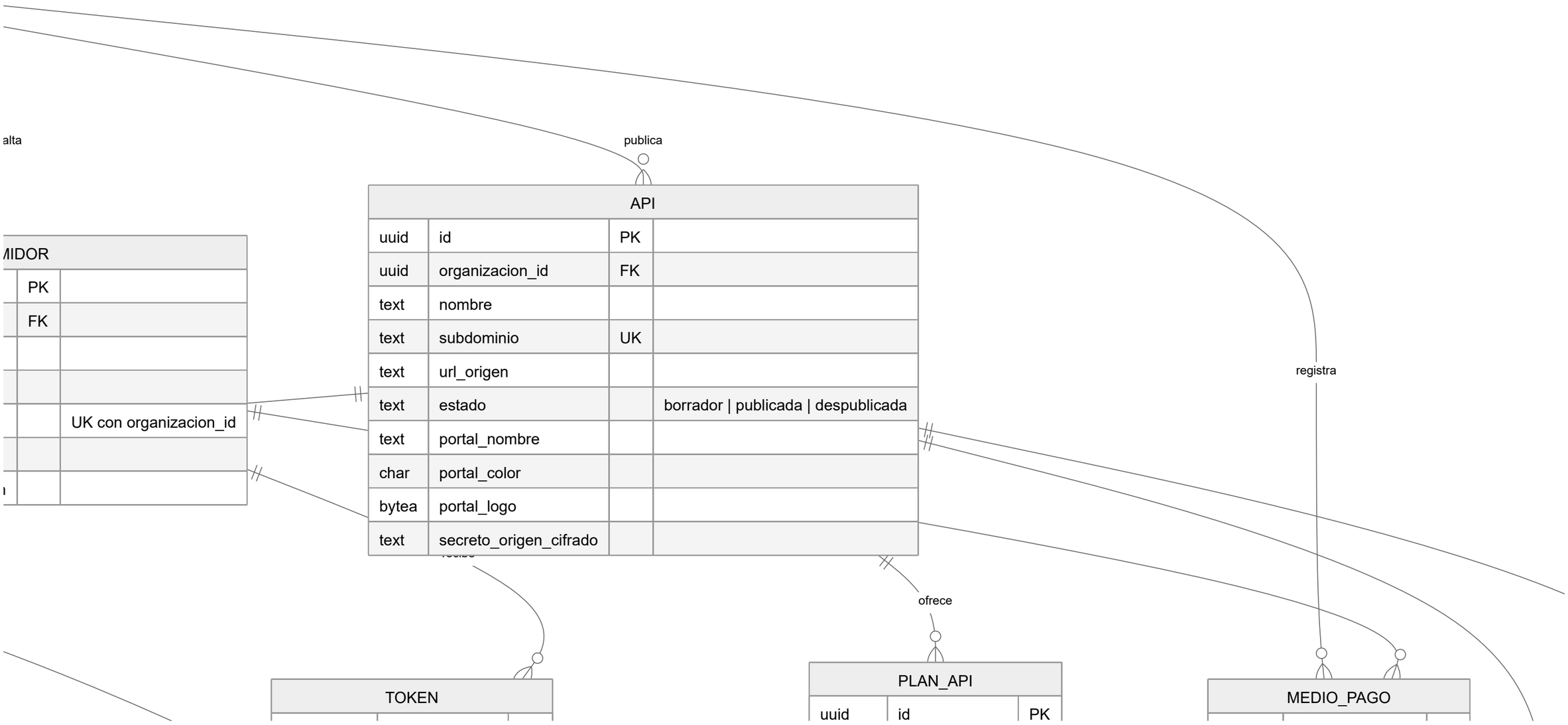


Diagrama 12 · Detalle 4 de 12 · Fila 1, columna 4

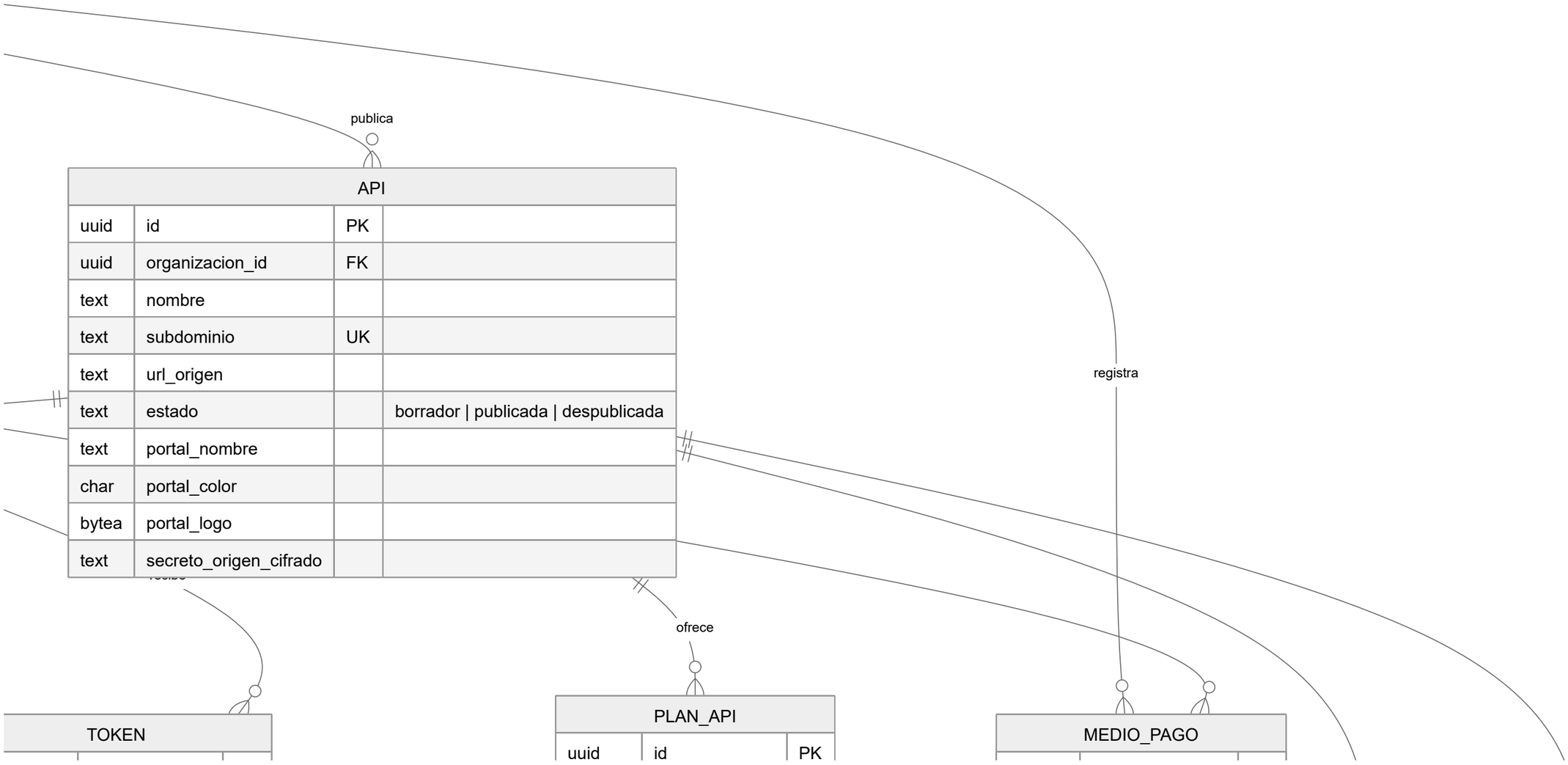


Diagrama 12 · Detalle 5 de 12 · Fila 2, columna 1

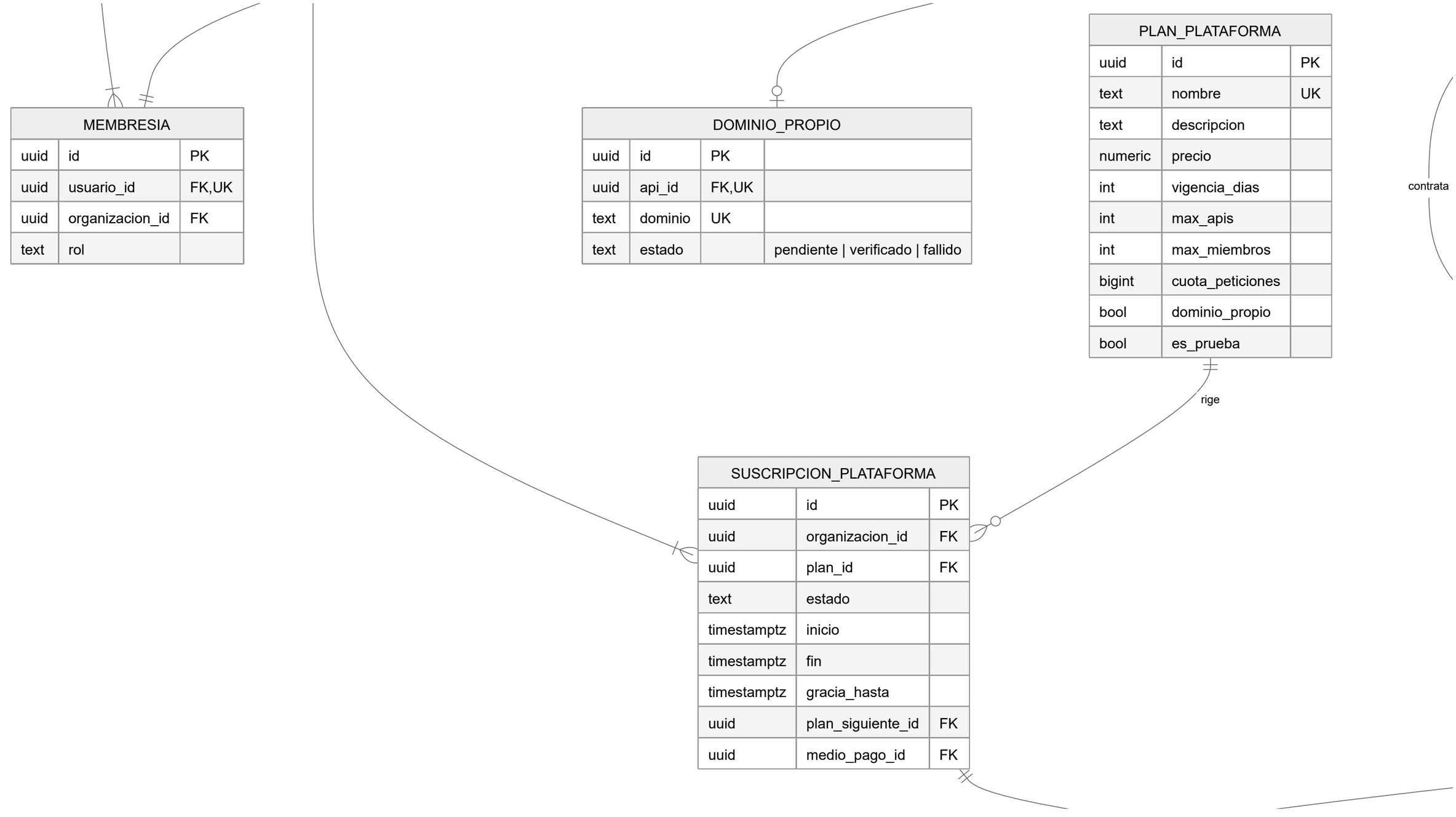


Diagrama 12 · Detalle 6 de 12 · Fila 2, columna 2

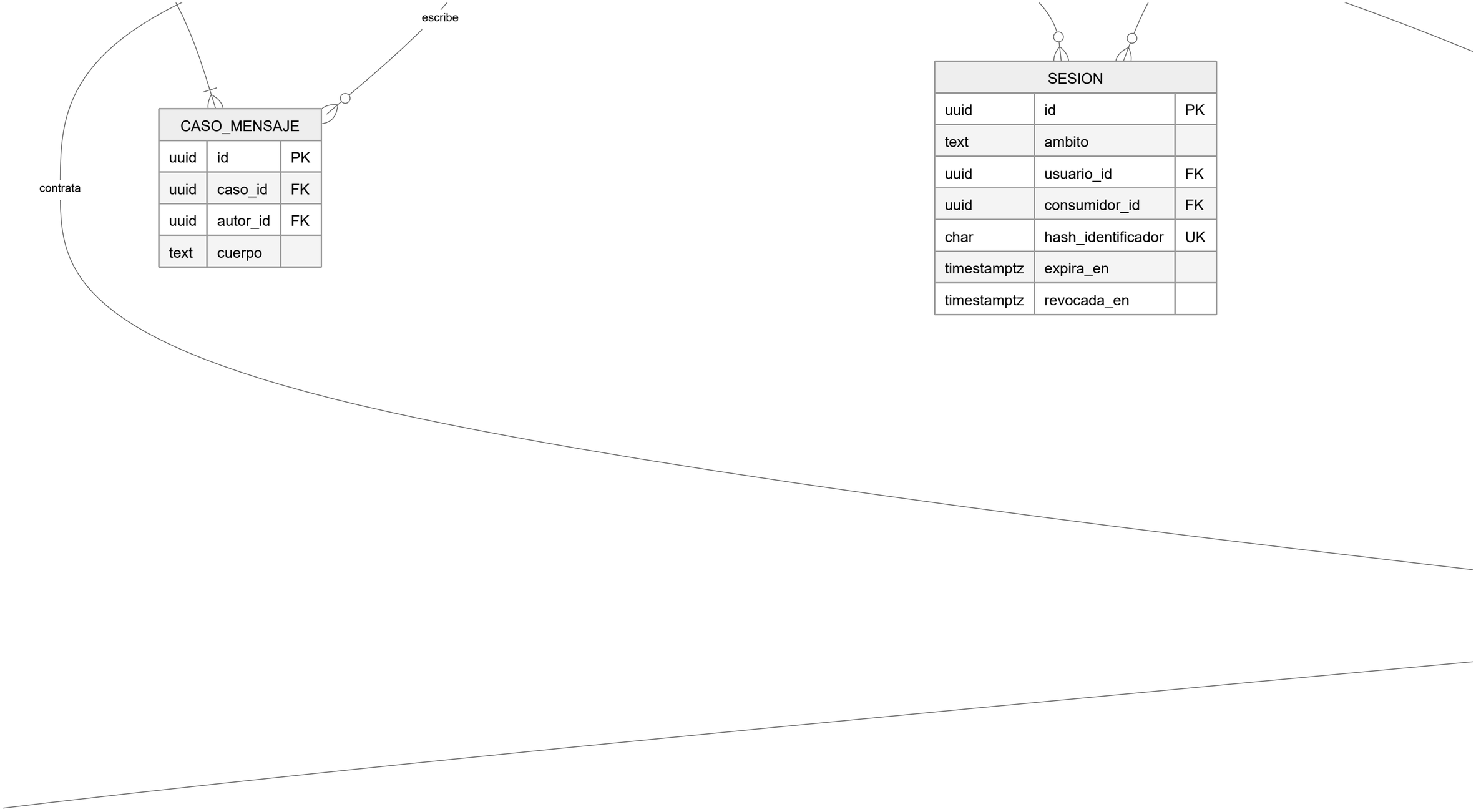


Diagrama 12 · Detalle 7 de 12 · Fila 2, columna 3

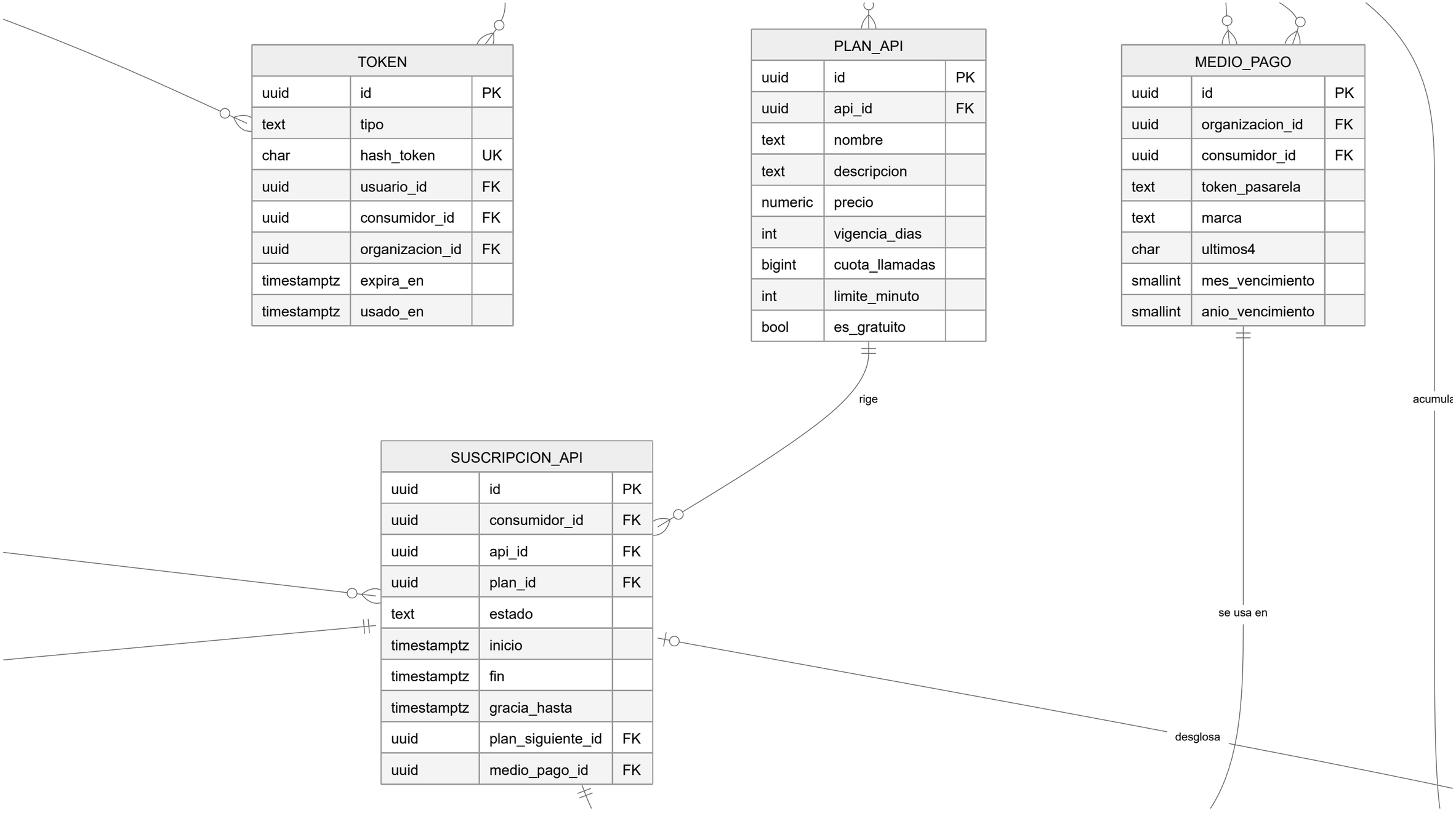


Diagrama 12 · Detalle 8 de 12 · Fila 2, columna 4

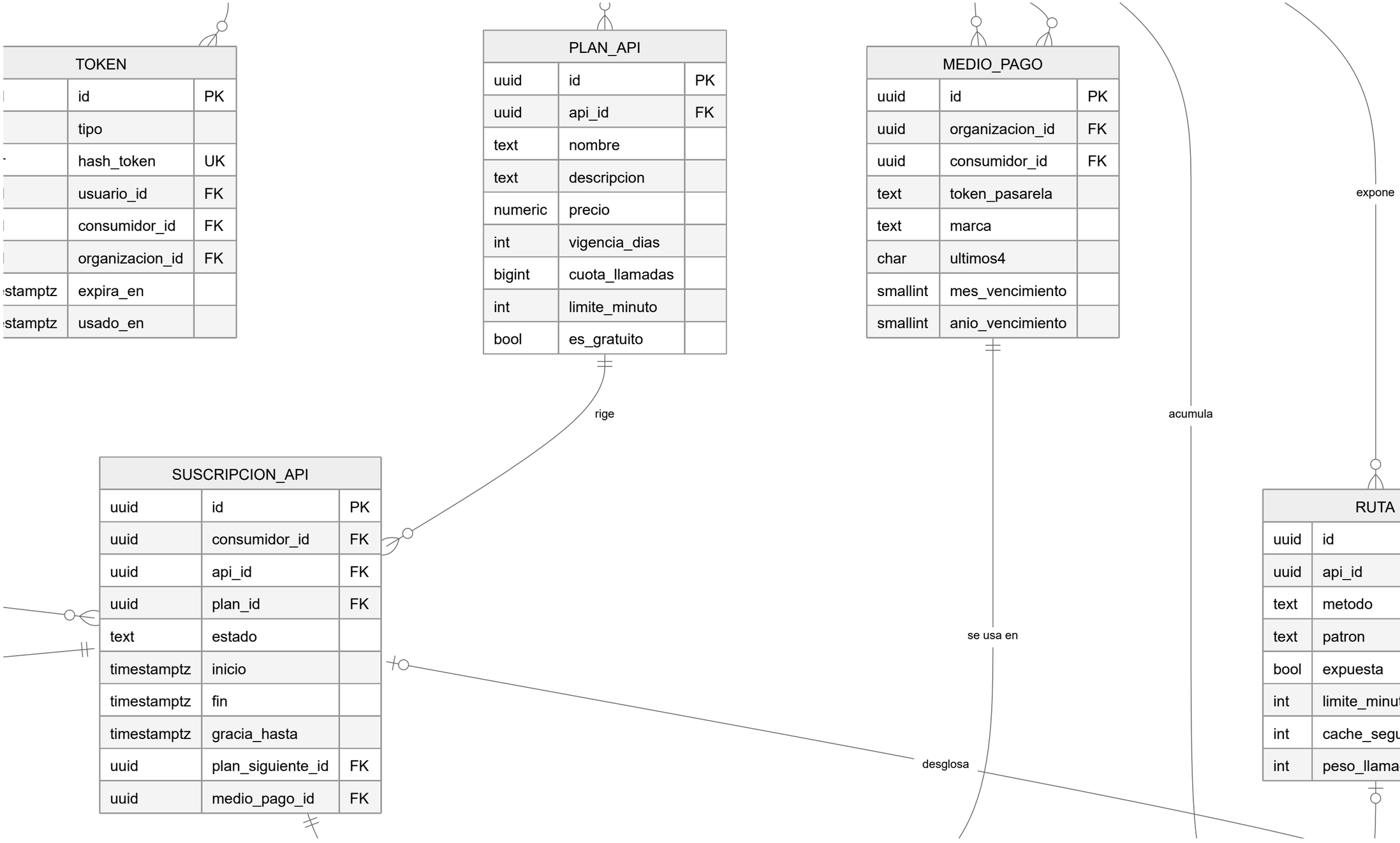


Diagrama 12 · Detalle 9 de 12 · Fila 3, columna 1

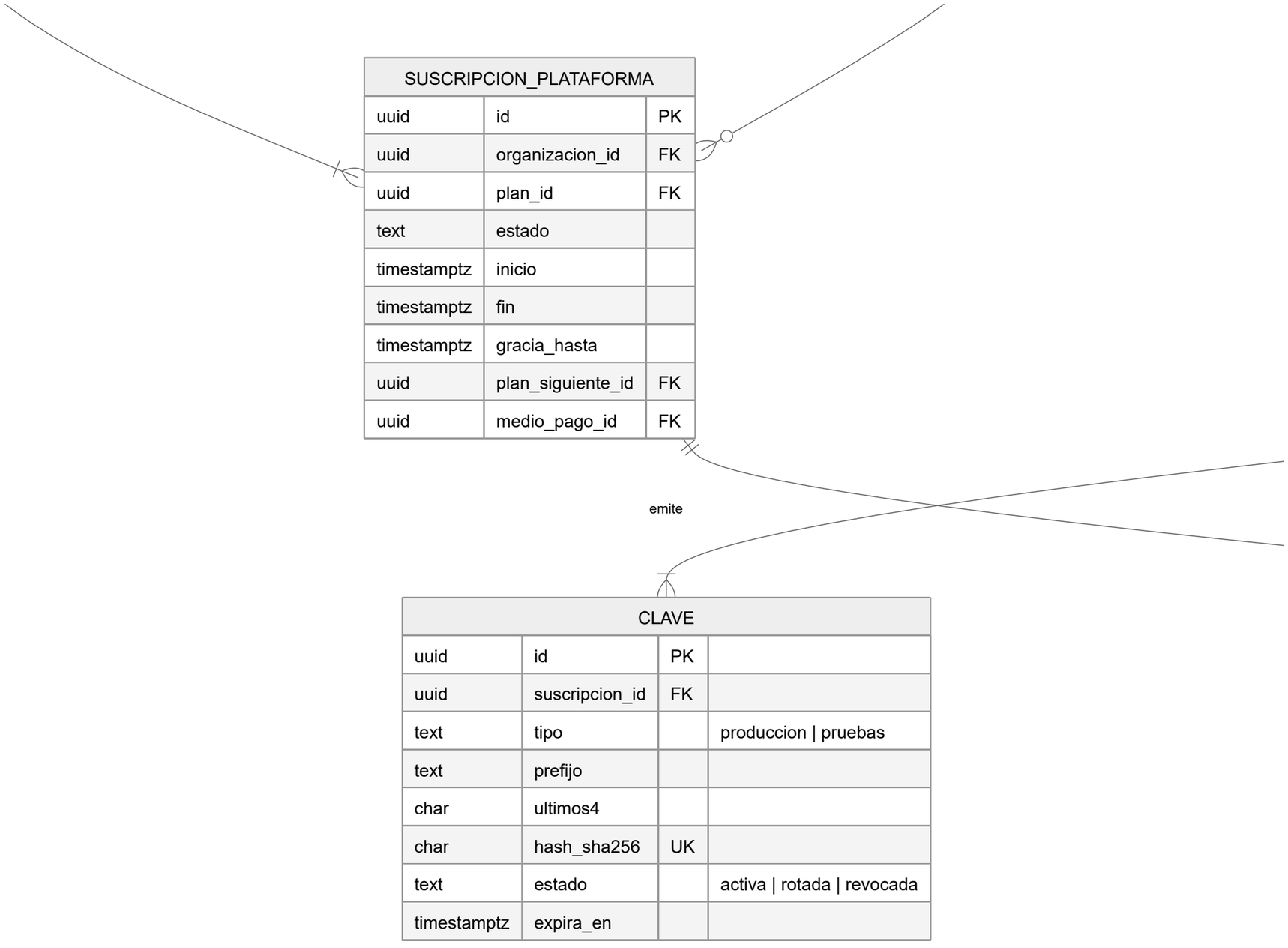


Diagrama 12 · Detalle 10 de 12 · Fila 3, columna 2

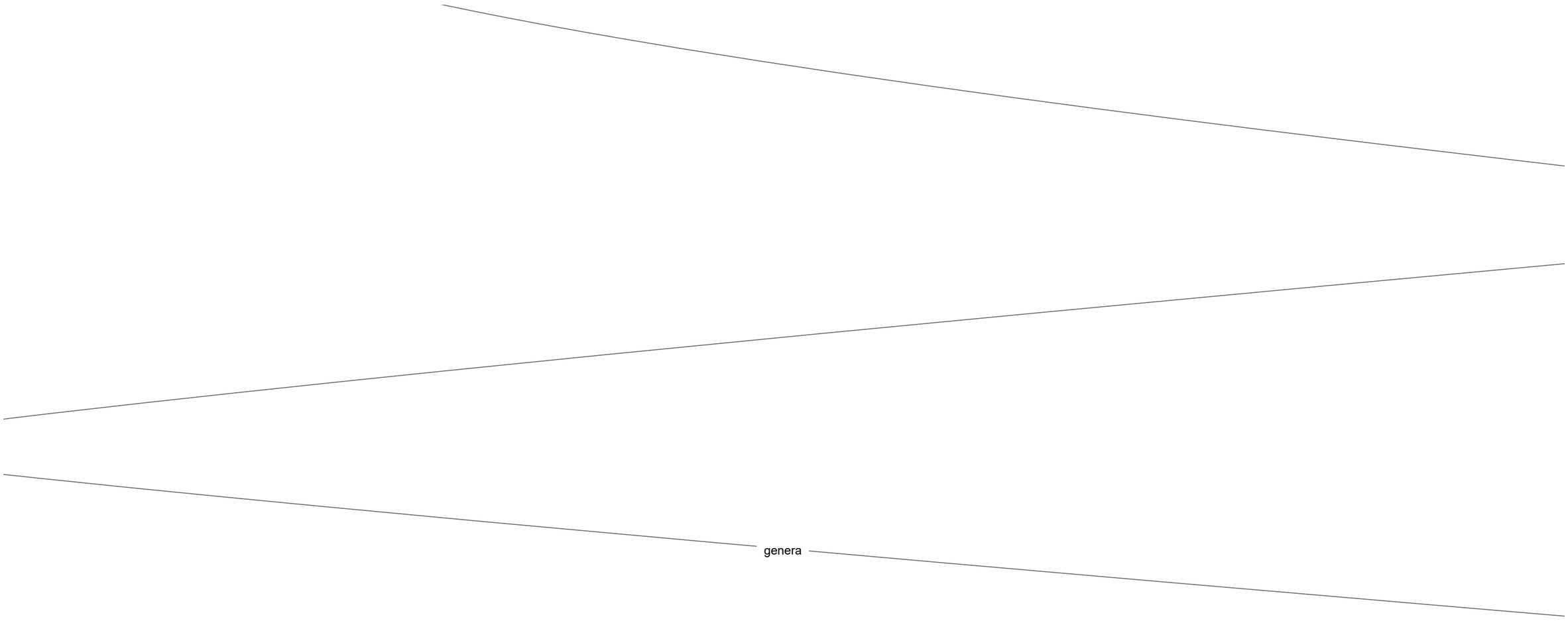


Diagrama 12 · Detalle 11 de 12 · Fila 3, columna 3

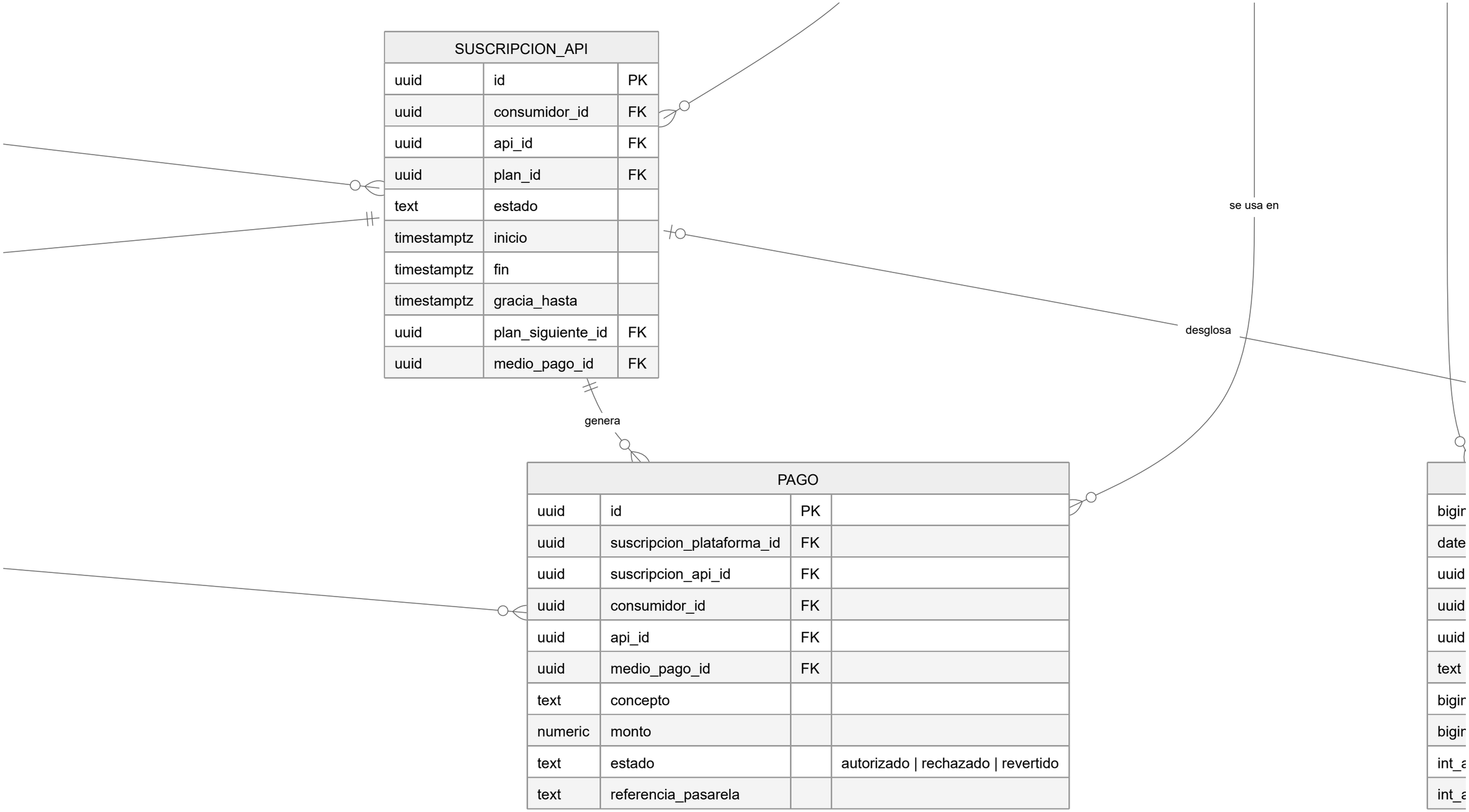
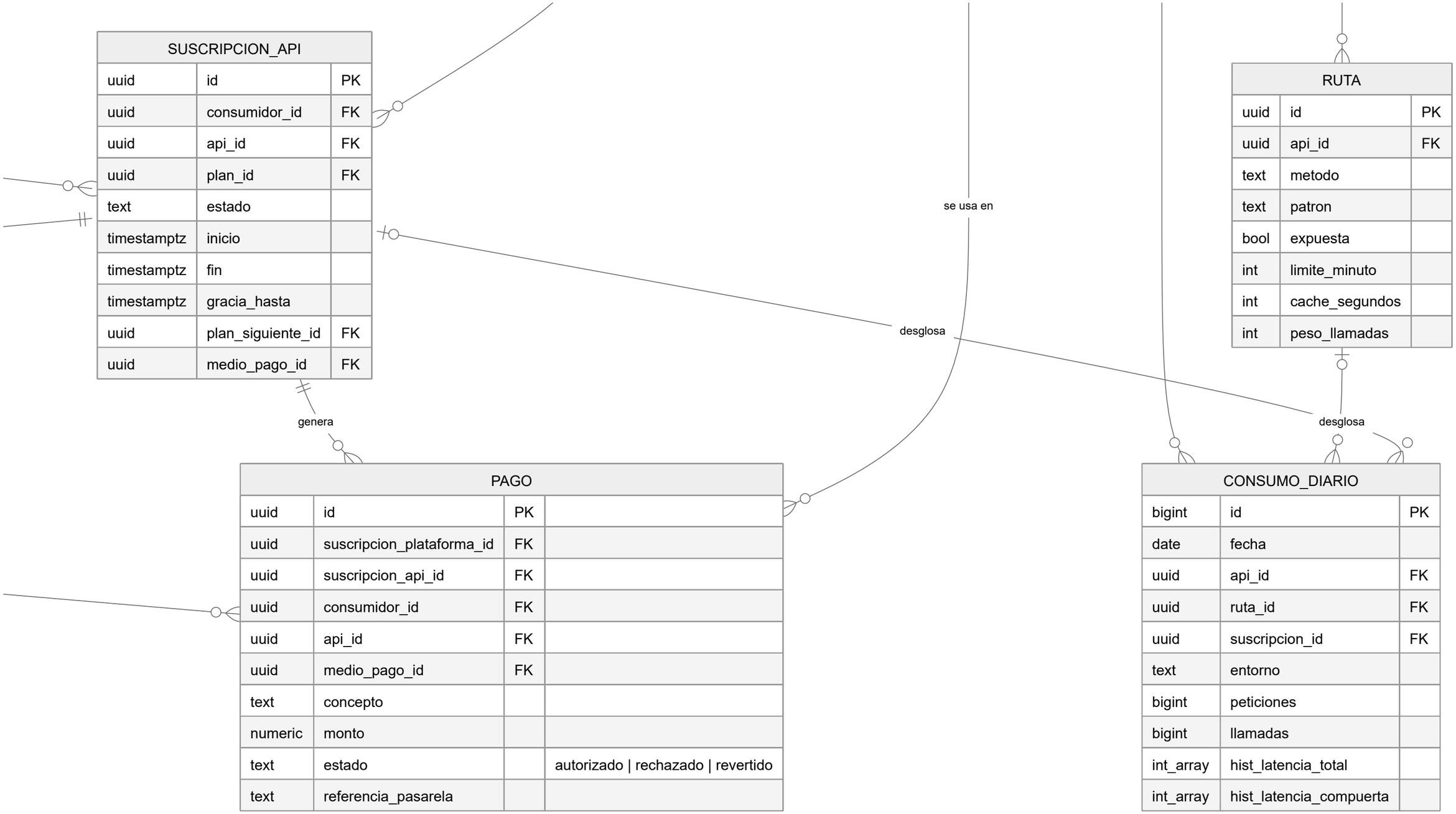


Diagrama 12 · Detalle 12 de 12 · Fila 3, columna 4



También hay tablas auxiliares que no aparecen en el diagrama: `correo_saliente`, `registro_dns_simulado` y `lote_consolidado`.

3. Diseño físico (PostgreSQL 16)

Convenciones:

- Los nombres van en `snake_case` y en español.
- Las llaves primarias son `uuid` (UUID v7, generado en la aplicación), salvo en las tablas de alto volumen (`bigint identity`).
- Todas las fechas son `timestamptz` en UTC.
- El dinero es `numeric(12,2)`.
- Los enumerados son `text` con una restricción `CHECK`. Las restricciones también van en `snake_case`: `ck_<tabla>_<regla>`.
- Toda tabla tiene `creado_en timestamptz not null default now()`. Las tablas que se modifican tienen además `actualizado_en`. En `sesion`, `bitacora` y `lote_consolidado`, las columnas `creada_en`, `fecha` y `procesado_en` hacen de `creado_en`. `medio_pago` y `caso_mensaje` no se modifican y solo tienen `creado_en`. La aplicación llena las dos columnas con la hora de `IReloj`, para que el modo demostración (09 §9) también las adelante. Las actualizaciones que no pasan por el seguimiento de cambios de EF (`ExecuteUpdate` o SQL directo, como el UPSERT de `consumo_diario`) ponen `actualizado_en` a mano.
- Los correos se guardan en minúsculas, normalizados en la aplicación, y son únicos con un índice sobre `lower(correo)`.
- Las migraciones se hacen con EF Core, un proyecto por migración, en `Shapi.Infraestructura`.

3.1 Identidad y acceso

`organizacion`

Columna	Tipo	Restricciones
id	uuid	PK
nombre	text	not null, largo entre 2 y 120
tipo	text	not null, `CHECK (tipo IN ('plataforma','proveedor'))`. Hay un índice único parcial `WHERE tipo='plataforma'` para que exista una sola
estado_admin	text	not null, default `'activa'`, `CHECK IN ('activa','suspendida')`
motivo_suspension	text	null
creado_en, actualizado_en	timestamptz	not null

> El **estado efectivo** de una organización es `suspendida` si `estado_admin = 'suspendida'` **o** si su suscripción de plataforma no finalizada está `suspendida`. En cualquier otro caso es `activa`. Ese valor se calcula en la aplicación y se publica en Redis (`org:{id}`).

`usuario`

Columna	Tipo	Restricciones
id	uuid	PK
nombre	text	not null
correo	text	not null, UNIQUE (`lower(correo)`)
hash_contrasena	text	null hasta que la persona defina su contraseña (cuentas de plataforma creadas por el administrador)
correo_verificado_en	timestamptz	null
estado	text	not null, default `'activo'`, `CHECK IN ('activo','desactivado')`
intentos_fallidos	int	not null, default 0
bloqueado_hasta	timestamptz	null

`membresia`

Columna	Tipo	Restricciones
id	uuid	PK
usuario_id	uuid	FK → usuario, **UNIQUE** (una sola organización por usuario)
organizacion_id	uuid	FK → organizacion, not null
rol	text	`CHECK IN ('administrador','soporte','propietario','editor','lector')`

Hay un índice único parcial `(organizacion_id) WHERE rol='propietario'` para que cada organización tenga un solo propietario. Que `administrador` y `soporte` solo existan en la organización de plataforma lo valida la aplicación.

`consumidor`

Columna	Tipo	Restricciones
id	uuid	PK
organizacion_id	uuid	FK → organizacion, not null
nombre	text	not null
nombre_empresa	text	not null
correo	text	not null. Es UNIQUE junto con `organizacion_id` (`organizacion_id, lower(correo)`)
hash_contrasena	text	not null

Columna	Tipo	Restricciones
correo_verificado_en	timestamptz	null
estado	text	default `'activo'`, `CHECK IN ('activo','desactivado')`
intentos_fallidos, bloqueado_hasta	int, timestamptz	igual que en `usuario`

`token`: enlaces de un solo uso.

Columna	Tipo	Restricciones
id	uuid	PK
tipo	text	`CHECK IN ('verificacion_correo','recuperacion','invitacion_miembro','invitacion_consumidor','definir_contrasena')`
hash_token	char(64)	UNIQUE. Es el SHA-256 del valor aleatorio de 32 bytes que va en el enlace
usuario_id	uuid	FK null
consumidor_id	uuid	FK null
organizacion_id	uuid	FK null (en las invitaciones)
correo	text	not null (en las invitaciones es el correo invitado)
rol	text	null (en `invitacion_miembro`)
expira_en	timestamptz	not null. Vence a las 24 h la verificación, a los 60 min la recuperación y a los 7 días las invitaciones y `definir_contrasena`
usado_en	timestamptz	null

`sesion`

Columna	Tipo	Restricciones
id	uuid	PK
ambito	text	`CHECK IN ('personal','consumidor')`
usuario_id / consumidor_id	uuid	FK. Exactamente uno de los dos tiene valor (`CHECK num_nonnulls(usuario_id, consumidor_id) = 1`)
hash_identificador	char(64)	UNIQUE. Es el SHA-256 del valor de la cookie
host	text	not null (el host del portal, en el ámbito consumidor)
creada_en, ultimo_uso_en, expira_en	timestamptz	not null
revocada_en	timestamptz	null

Columna	Tipo	Restricciones
ip	inet, agente_usuario text	null

3.2 APIs

`api`

Columna	Tipo	Restricciones
id	uuid	PK
organizacion_id	uuid	FK, not null
nombre	text	not null
subdominio	text	not null, UNIQUE, `CHECK (subdominio ~ '^[a-z0-9][a-z0-9-]{1,28}[a-z0-9]$')`. Que no esté reservado lo valida la aplicación
url_origen	text	not null
estado	text	default `'borrador'`, `CHECK IN ('borrador','publicada','despublicada')`
especificacion	text	null (el archivo original, de hasta 2 MB)
especificacion_formato	text	`CHECK IN ('json','yaml')`, null
especificacion_titulo, especificacion_descripcion, especificacion_version	text	null (salen de `info`)
especificacion_cargada_en	timestamptz	null
portal_nombre	text	null (si está vacío se usa el nombre de la API)
portal_color	char(7)	default `'#3B6FF0'`, `CHECK (portal_color ~ '^#[0-9A-Fa-f]{6}$')`
portal_logo	bytea	null, de hasta 512 KB
portal_logo_tipo	text	`CHECK IN ('image/png','image/svg+xml')`, null
portal_bienvenida	text	null, de hasta 280 caracteres
secreto_origen_cifrado	text	not null (cifrado con Data Protection, propósito `Shapi.SecretoOrigen`)
publicada_en	timestamptz	null

`ruta`

Columna	Tipo	Restricciones
id	uuid	PK

Columna	Tipo	Restricciones
api_id	uuid	FK, not null, `ON DELETE CASCADE`
metodo	text	`CHECK IN ('GET','POST','PUT','PATCH','DELETE','HEAD','OPTIONS')`
patron	text	not null, en sintaxis OpenAPI (`/rastreo` o, con un parámetro, `/guias/{numero}`)
resumen, descripcion	text	null (salen de la especificación)
definicion	jsonb	not null (los parámetros, el cuerpo y los ejemplos de la operación, para la documentación y la consola). Forma: `{"orden": n, "parametros": [...], "cuerpo": {...} o null, "respuestas": {...}}` . `orden` es la posición de la operación en la especificación; los otros tres son los objetos `parameters` (incluidos los del camino), `requestBody` y `responses` de OpenAPI, con las referencias locales resueltas
expuesta	bool	not null, default **false**
limite_minuto	int	null, `CHECK (limite_minuto > 0)`
cache_segundos	int	not null, default 0, `CHECK (cache_segundos BETWEEN 0 AND 86400)` . Solo es mayor que 0 si el método es GET
peso_llamadas	int	not null, default 1, `CHECK (peso_llamadas BETWEEN 1 AND 1000)`

UNIQUE `(api_id, metodo, patron)` .

`dominio_propio`

Columna	Tipo	Restricciones
id	uuid	PK
api_id	uuid	FK, UNIQUE
dominio	text	UNIQUE, en minúsculas
destino_cname	text	not null (`{sub}.api.{dominio_base}`)
estado	text	`CHECK IN ('pendiente','verificado','fallido')`
motivo	text	null
verificado_en, ultimo_intento_en	timestamptz	null

`registro_dns_simulado` : solo se usa con `SHAPI_DNS_MODO=simulado` .

Columna	Tipo	Restricciones
id	uuid	PK
nombre	text	UNIQUE
tipo	text	`CHECK IN ('CNAME')`

Columna	Tipo	Restricciones
valor	text	not null

3.3 Planes, suscripciones y claves

`plan_plataforma`

Columna	Tipo	Restricciones
id	uuid	PK
nombre	text	UNIQUE
descripcion	text	not null
precio	numeric(12,2)	`CHECK (precio >= 0)`
vigencia_dias	int	`CHECK (vigencia_dias BETWEEN 1 AND 366)`
max_apis	int	null significa sin límite
max_miembros	int	null significa sin límite
cuota_peticiones	bigint	not null, `CHECK > 0`
dominio_propio	bool	not null
es_prueba	bool	not null, default false. Hay un índice único parcial `WHERE es_prueba` para que exista uno solo
activo	bool	default true
orden	int	para ordenarlos en la página de precios

`plan_api`

Columna	Tipo	Restricciones
id	uuid	PK
api_id	uuid	FK
nombre	text	not null, UNIQUE junto con `api_id`
descripcion	text	not null
precio	numeric(12,2)	`CHECK (precio >= 0)`, con 2 decimales como máximo
es_gratuito	bool	`CHECK (NOT es_gratuito OR precio = 0)` y `CHECK (es_gratuito OR precio > 0)`: un plan de pago con precio 0 no se podría contratar, porque `pago.monto` debe ser mayor que 0
vigencia_dias	int	`CHECK BETWEEN 1 AND 366`
cuota_llamadas	bigint	`CHECK > 0`

Columna	Tipo	Restricciones
limite_minuto	int	`CHECK > 0`
activo	bool	default true

`suscripcion_plataforma` y `suscripcion_api`: tienen columnas comunes.

Columna	Tipo	Restricciones
id	uuid	PK
organizacion_id (plataforma) · consumidor_id + api_id (API)	uuid	FK, not null
plan_id	uuid	FK → el plan de su nivel
estado	text	`CHECK IN ('activa','en_gracia','suspendida','finalizada')`
inicio, fin	timestamptz	not null, `CHECK (fin > inicio)`
gracia_hasta	timestamptz	null
plan_siguiente_id	uuid	FK null (una bajada de plan programada)
medio_pago_id	uuid	FK null

Índices únicos parciales `WHERE estado <> 'finalizada'`: en `(organizacion_id)` para la de plataforma y en `(consumidor_id, api_id)` para la de API. Así hay **una sola suscripción vigente** por organización y una por consumidor en cada API. Hay también un índice `(estado, fin)` y otro `(estado, gracia_hasta)` para el cierre de ciclo.

`clave`

Columna	Tipo	Restricciones
id	uuid	PK
suscripcion_id	uuid	FK → suscripcion_api
tipo	text	`CHECK IN ('produccion','pruebas')`
prefijo	text	`shp_prod_` o `shp_prueba_`
ultimos4	char(4)	not null
hash_sha256	char(64)	UNIQUE (hex en minúsculas)
estado	text	`CHECK IN ('activa','rotada','revocada')`
expira_en	timestamptz	null (en una clave `rotada` es la hora a la que deja de funcionar)
revocada_en	timestamptz	null

Columna	Tipo	Restricciones
revocada_por	text	`CHECK IN ('consumidor','proveedor')`, null

Hay un índice único parcial `(suscripcion_id, tipo) WHERE estado='activa'` para que haya **una sola clave activa por tipo**.

3.4 Pagos

`medio_pago`

Columna	Tipo	Restricciones
id	uuid	PK
organizacion_id / consumidor_id	uuid	exactamente uno de los dos tiene valor (`CHECK num_nonnulls(...) = 1`)
token_pasarela	text	not null
marca	text	`CHECK IN ('Visa','Mastercard','American Express')`
ultimos4	char(4)	not null
titular	text	not null
mes_vencimiento	smallint	1–12
anio_vencimiento	smallint	≥ el año actual al registrarse

`pago`

Columna	Tipo	Restricciones
id	uuid	PK
suscripcion_plataforma_id / suscripcion_api_id	uuid	exactamente uno de los dos tiene valor para pagos asociados a suscripciones; ambos son null en un intento de contratación de API rechazado
consumidor_id · api_id	uuid	FK → consumidor y FK → api, null. Ambos tienen valor únicamente en un intento de contratación de API rechazado sin suscripción; en los demás pagos ambos son null
medio_pago_id	uuid	FK null (los planes gratuitos no generan pago)
concepto	text	`CHECK IN ('contratacion','renovacion','cambio_plan','reactivacion')`
descripcion	text	not null (por ejemplo "Lanzamiento → Producto · diferencia prorrateada")
monto	numeric(12,2)	`CHECK (monto > 0)`
estado	text	`CHECK IN ('autorizado','rechazado','revertido')`

Columna	Tipo	Restricciones
referencia_pasarela	text	null
motivo_rechazo	text	null
periodo_inicio, periodo_fin	timestamptz	null
revertido_en	timestamptz	null
revertido_por	uuid	FK → usuario, null

Índices: `(suscripcion_plataforma_id, creado_en desc)`, `(suscripcion_api_id, creado_en desc)`, `(creado_en desc)`, este último para A6.3, y uno por cada FK nueva: `(consumidor_id)` y `(api_id)`.

La restricción `ck_pago_suscripcion` admite exactamente dos formas: un pago asociado a una suscripción de plataforma o API (sin `consumidor_id` ni `api_id`), o un intento de contratación de API rechazado (sin suscripción y con ambos `consumidor_id` y `api_id`). Esto permite conservar el rechazo exigido por EM-08 sin crear una suscripción ni perder a quién y a qué API corresponde el intento. El intento rechazado no guarda medio de pago ni datos de tarjeta.

3.5 Consumo

`consumo_diario`

Columna	Tipo	Restricciones
id	bigint identity	PK
fecha	date	not null (día en la zona America/Guatemala)
api_id	uuid	FK, not null
ruta_id	uuid	FK null (si la ruta no se pudo identificar, o si se retiró al recargar la especificación: su consumo se suma a la fila con `ruta_id` nulo del mismo día, suscripción y entorno, y la fila de la ruta se borra)
suscripcion_id	uuid	FK → suscripcion_api, null (si la clave no era válida)
entorno	text	`CHECK IN ('produccion','pruebas')`
peticiones, llamadas	bigint	default 0
bytes_entrada, bytes_salida	bigint	default 0
rechazos_401, rechazos_403, rechazos_404, rechazos_429	bigint	default 0
origen_2xx, origen_3xx, origen_4xx, origen_5xx, origen_fallo	bigint	default 0 (`origen_fallo` cuenta los 502 y 504)
hist_latencia_total	int[]	10 rangos, en ms: ≤5, ≤10, ≤25, ≤50, ≤100, ≤250, ≤500, ≤1000, ≤2500 y >2500
hist_latencia_compuerta	int[]	los mismos 10 rangos

Columna	Tipo	Restricciones
latencia_total_suma_ms, latencia_compuerta_suma_ms	bigint	para calcular promedios

UNIQUE `NULLS NOT DISTINCT (fecha, api_id, ruta_id, suscripcion_id, entorno)`. Hay un índice `(api_id, fecha)` y otro `(suscripcion_id, fecha)`.

`lote_consolidado`: `lote_id uuid PK`, `procesado_en timestamptz`. Garantiza que un lote se consolide una sola vez (08 §7).

3.6 Soporte, auditoría y correo

`caso`

Columna	Tipo	Restricciones
id	uuid	PK
numero	int	UNIQUE, secuencia que empieza en 100. En pantalla se muestra `CAS-{numero}`
organizacion_id	uuid	FK (siempre una organización proveedora)
api_id	uuid	FK null
creado_por	uuid	FK → usuario (un proveedor o alguien del soporte)
asignado_a	uuid	FK → usuario, null
asunto	text	not null, de hasta 120 caracteres
estado	text	`CHECK IN ('abierto','cerrado')`
cerrado_en	timestamptz	null

`caso_mensaje`: `id uuid PK`, `caso_id FK`, `autor_id FK → usuario`, `cuerpo text not null`, `creado_en`. El primer mensaje del caso es su descripción.

`bitacora`

Columna	Tipo	Restricciones
id	bigint identity	PK
fecha	timestamptz	default now()
actor_tipo	text	`CHECK IN ('usuario','consumidor','sistema')`
actor_id	uuid	null (si el actor es el sistema)
actor_nombre	text	not null (una copia, para que se pueda leer aunque la cuenta cambie)
organizacion_id	uuid	FK null

Columna	Tipo	Restricciones
accion	text	not null (del catálogo de 10 §7)
objetivo_tipo, objetivo_id	text, uuid	null
descripcion	text	not null (el texto que se ve en B3.2)
detalle	jsonb	null
ip	inet	null

La bitácora solo admite inserciones: dos disparadores rechazan cualquier UPDATE, DELETE o TRUNCATE sobre `bitacora`, con el mensaje "La bitácora solo admite inserciones: no se permite <operación>.". La aplicación y las migraciones usan el mismo rol de base de datos (decisión de Jordin del 26 sep, auditoría H-40). Por eso los disparadores protegen contra errores de la aplicación, pero no contra alguien con acceso de dueño a la base.

`correo_saliente`: `id uuid PK`, `destinatario text`, `asunto text`, `plantilla text`, `datos jsonb`, `estado CHECK IN ('pendiente','enviado','fallido')`, `intentos int`, `proximo_intento_en timestamptz`, `ultimo_error text`, `enviado_en timestamptz`, `creado_en timestamptz`, `actualizado_en timestamptz`. Índice: `(estado, proximo_intento_en)`, para la búsqueda de pendientes que el trabajador hace cada 5 segundos. Mientras el correo está `pendiente`, `datos` puede llevar el `token` en claro, porque hace falta para armar el enlace. Al pasar a `enviado` o `fallido`, el `token` se borra de `datos` (10 §3).

4. Estructura de las llaves en Redis

El formato de cada llave está en `Shapi.Contratos.LlavesRedis`, y es la **única** definición que usan la API, la compuerta y el trabajador. Todos los valores se escriben con `HSET` (hash de Redis), salvo que se indique otra cosa.

Llave	Tipo	Contenido	Escribe	Lee
`api:host:{host}`	string	`api_id`. Es el host de la API (`envios.api.shapi.localhost`) o el dominio propio verificado	API de control	Compuerta
`api:{api_id}`	hash	`organizacion_id`, `estado`, `url_origen`, `secreto` (en claro, porque Redis solo es accesible desde la red interna), `portal_host` (`{sub}.{dominio_base}`), `version` (sube en 1 con cada publicación, también en la resincronización)	API de control	Compuerta
`api:{api_id}:rutas`	string (JSON)	Arreglo de `{ruta_id, metodo, patron, expuesta, limite_minuto, cache_segundos, peso}` con todas las rutas, también las ocultas. Se escribe en la misma transacción que `api:{api_id}`	API de control	Compuerta

Llave	Tipo	Contenido	Escribe	Lee
`clave:{sha256}`	hash	`clave_id`, `suscripcion_id`, `api_id`, `organizacion_id`, `consumidor_id`, `tipo`. Cuando la clave está rotada, la llave tiene `EXPIREAT`. Como `expira_en` está en la hora de `IReloj`, que el modo demostración adelanta, y Redis usa la hora real, el instante se traslada: hora real + (`expira_en` − `IReloj.Ahora`). Una clave revocada no tiene llave	API de control	Compuerta
`susc:{suscripcion_id}`	hash	`plan_id`, `plan_nombre`, `estado`, `inicio` (epoch), `fin` (epoch), `cuota_llamadas`, `limite_minuto`. Solo las suscripciones de API sin finalizar: al finalizar, la llave se borra. La suscripción de plataforma no tiene llave propia: su estado va en `org:{id}`	API de control y trabajador	Compuerta
`org:{organizacion_id}`	hash	`estado_efectivo`, `cuota_peticiones`, `ciclo_inicio` (epoch), `ciclo_fin`. Los tres últimos salen de la suscripción de plataforma sin finalizar; si no hay ninguna (la organización de la plataforma), no se escriben	API de control y trabajador	Compuerta
`cuota:susc:{suscripcion_id}:{inicio}`	string (int)	Llamadas consumidas en el ciclo. TTL = `fin` + `8 días` (si el `fin` ya pasó, 8 días desde ahora; 08 §3)	Compuerta	Compuerta y API (B2.1)
`cuota:org:{organizacion_id}:{inicio}`	string (int)	Peticiones del ciclo de plataforma. TTL = `fin` + `8 días` (igual que la anterior)	Compuerta	Compuerta y API (avisos de RF-43)
`rl:s:{suscripcion_id}:{minuto_epoch}`	string (int)	Peticiones en ese minuto. TTL 120 s	Compuerta	Compuerta
`rl:r:{suscripcion_id}:{ruta_id}:{minuto_epoch}`	string (int)	Peticiones a esa ruta en ese minuto. TTL 120 s	Compuerta	Compuerta
`rl:p:{clave_id}:{minuto_epoch}` y `dia:p:{clave_id}:{aaaammdd}`	string (int)	Límites de la clave de pruebas: peticiones en ese minuto (TTL 120 s) y en ese día de Guatemala (vence un día después de la medianoche en que termina)	Compuerta	Compuerta
`cache:{api_id}:{ruta_id}:{sha256(metodo+ruta+query)}`	hash	`status`, `headers` (JSON), `body` (hasta 1 MB), `version` (la de `api:{id}` al obtener la respuesta). TTL = `cache_segundos`	Compuerta	Compuerta
`met:{aaaammdd}:{api}:{ruta|-}:{susc|-}:{entorno}`	hash	Contadores y rangos del histograma (`h_t_0` … `h_t_9`, `h_c_0` … `h_c_9`)	Compuerta	Trabajador
`met:pendientes`	set	Llaves `met:` que tienen datos	Compuerta	Trabajador
`met:lote:{lote_id}:{llave}`	hash	Una instantánea que se está consolidando	Trabajador	Trabajador

Llave	Tipo	Contenido	Escribe	Lee
`salud:compuerta:{instancia}` y `salud:trabajador`	string	Marca de tiempo. TTL 30 s	Compuerta y trabajador	API
`salud:latencia-compuerta:{minuto_unix}`	hash	Contadores por `segundo_unix:rango` del histograma de latencia añadida; minuto y segundo del reloj real. TTL 65 min	Compuerta	API (ventana móvil de la última hora, RF-39)
`demo:reloj:desplazamiento`	string	Desplazamiento total en segundos del reloj del modo demostración; el canal del mismo nombre publica ese total (09 §9)	API de control	API y trabajador

Normalización de las llaves: los UUID van en minúsculas con guiones, el `{host}` en minúsculas y el `{sha256}` de la clave en hex minúsculas. En `cache:`, el hash se calcula sobre el método en mayúsculas seguido del camino y de la query tal como llega (con su `?` inicial), en UTF-8. Las fechas `{aaaammdd}` son el día en la zona America/Guatemala, el mismo que se guarda en `consumo_diario.fecha`.

Escritura: cada hash se reemplaza completo, con `DEL` y luego `HSET` en una misma transacción, para que no quede un campo viejo (por ejemplo, un secreto borrado). `PublicarApi` solo escribe los hosts actuales, así que, cuando un dominio propio se desconecta o se reemplaza por otro, la API de control llama además a `EliminarHost(host, apiId)`, que borra `api:host:{host}` en el momento si todavía apunta a esa API. La resincronización solo es el respaldo, si Redis falló. Si Redis falla después del *commit* en PostgreSQL, el publicador reintenta 3 veces con espera (200 ms, 500 ms y 1 s), registra el error y no revierte ni hace fallar la operación de negocio: la resincronización la corrige (06 §5.2).

Resincronización: cada 5 minutos, y al arrancar, el trabajador recalcula desde PostgreSQL todas las llaves de configuración (`api:*`, `clave:*`, `susc:*` y `org:*`) y las reescribe: las APIs publicadas, las claves activas y las rotadas vigentes, las suscripciones de API sin finalizar y todas las organizaciones. Los contadores no se tocan. Si Redis se vació, la compuerta vuelve a funcionar en cuanto termina la resincronización. También borra las llaves de configuración que ya no corresponden (una clave revocada o un dominio propio quitado mientras Redis fallaba, una API despublicada, una suscripción finalizada). Para no borrar una llave que otro proceso publique durante la resincronización, las llaves existentes se leen con `SCAN` **antes** de leer PostgreSQL. Al final vuelve a leer las claves que escribió, y corrige las que se revocaron o rotaron mientras tanto. Lo demás que cambie entre la lectura de PostgreSQL y la escritura en Redis (por ejemplo, una API que se despublica o se vuelve a publicar en ese instante) se corrige en la siguiente resincronización, 5 minutos después como máximo. Si el trabajador no puede descifrar el secreto de una API publicada, no la reescribe ni borra sus llaves, y registra el error.

5. Máquinas de estado de las entidades

La máquina de estados de las suscripciones está en 09 §3.

Diagrama 13 · Vista general

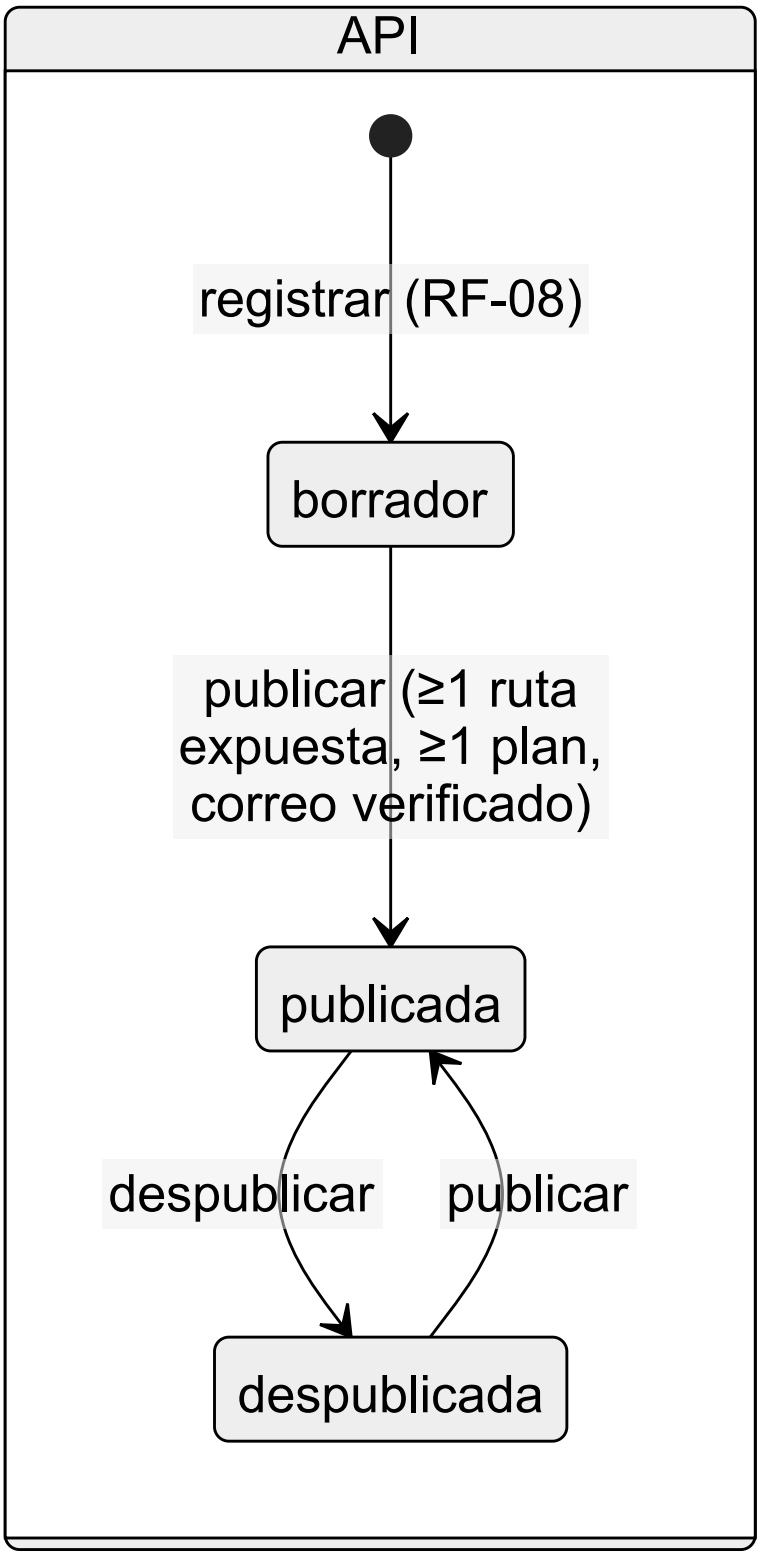


Diagrama 14 · Vista general

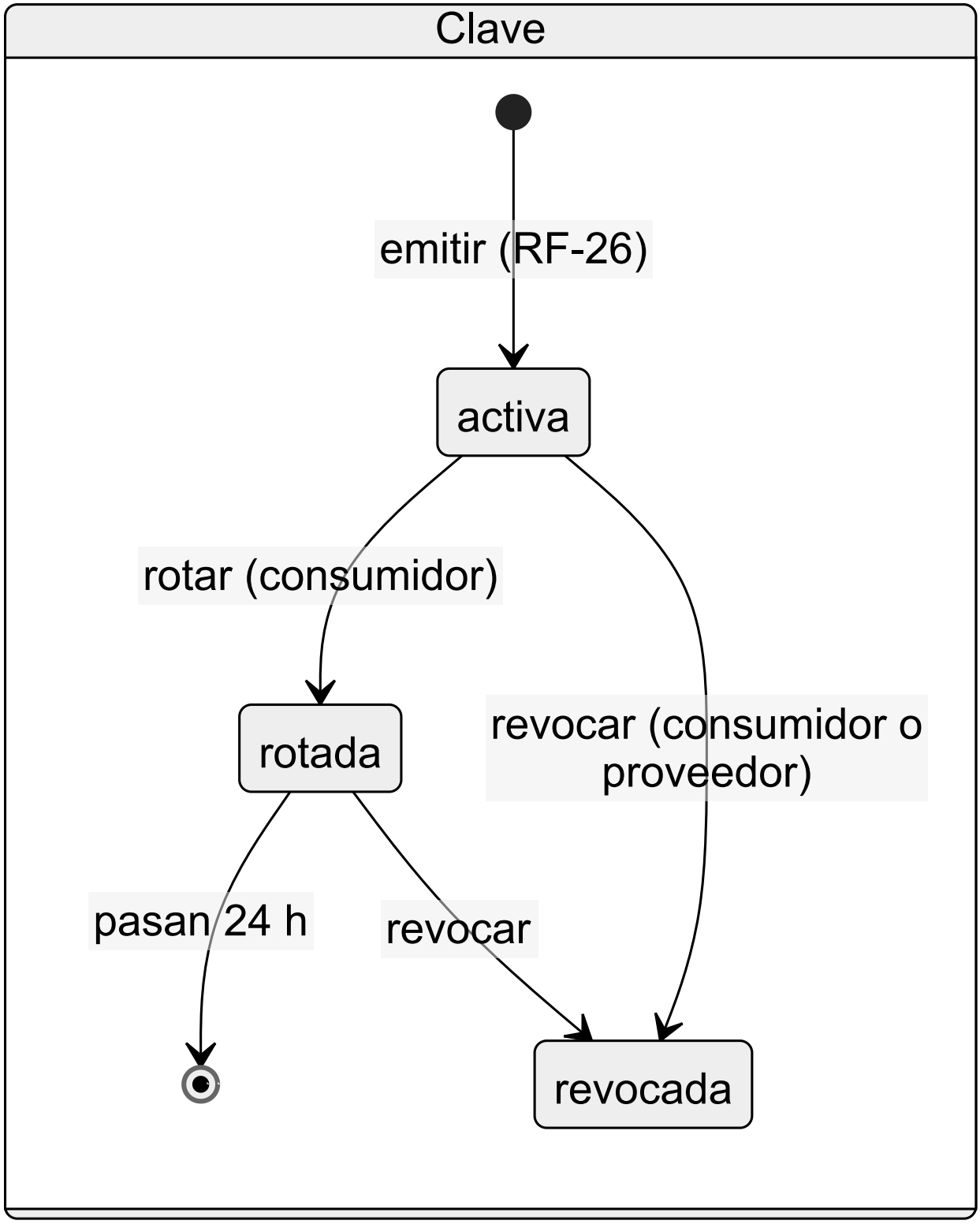


Diagrama 15 · Vista general

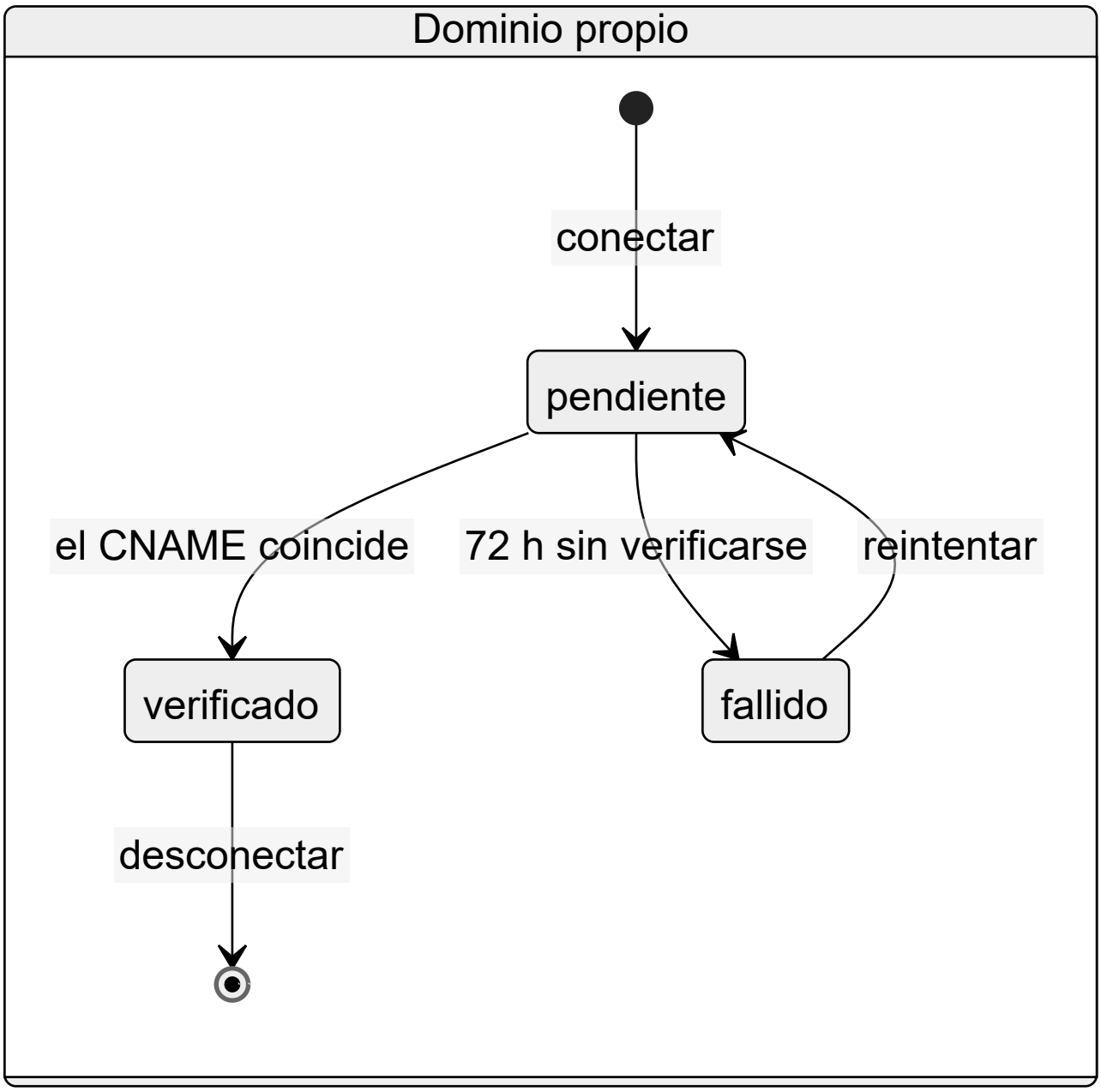
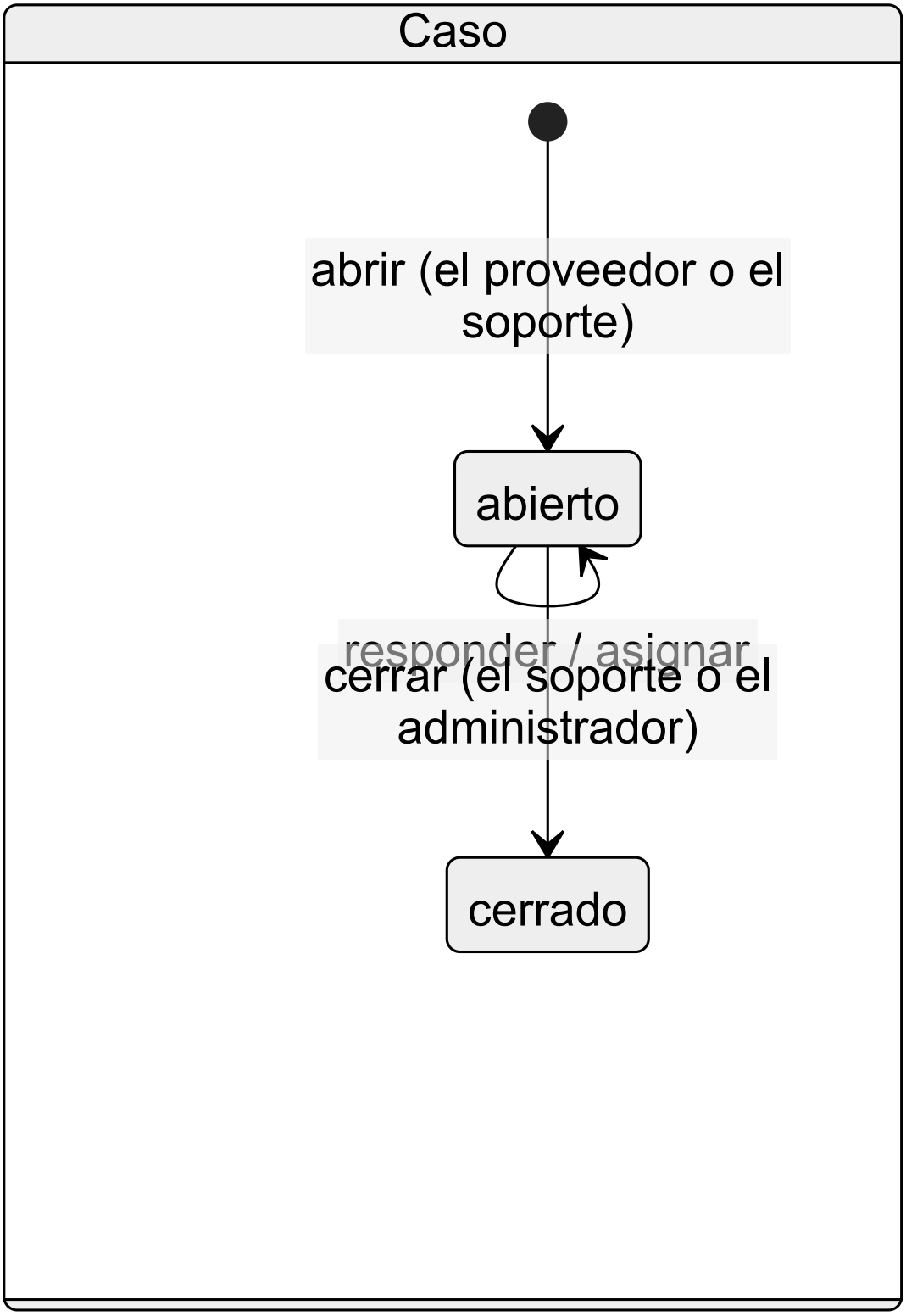


Diagrama 16 · Vista general



6. Datos de siembra

La siembra de **demostración** (`SHAPI_MODO_DEMO=true`) reproduce los datos de los mockups para que el sistema y los diseños coincidan:

- **Plataforma:** Rodrigo Alvarado y Lucía Ramírez Pineda (administración); Sofía Menchú Cojtí y Julio Estrada Ixcot, este último desactivado (soporte).
- **Planes de plataforma:** los de 01 §6.
- **Envíos Xelajú, S.A.**, en el plan Producto: Ana Lucía Morales (propietario), Diego Us Pérez (editor) y Karla Batres (lector). Tiene dos APIs:
  - **API de Cotización de Envíos**, con subdominio `envios`, publicada. Rutas `POST /cotizaciones`, `POST /guias` (peso 5), `GET /tarifas` (oculta), `GET /rastreo` y `GET /cobertura`. Planes Básico, Comercio y Volumen. Consumidores Mercadito Antigua, Boutique Cayalá, Ferretería Zona 11 y Tienda Sololá.
  - **API de Recolecciones**, con subdominio `recolecciones`, despublicada.
- **Agro Precios, S.A.**, en el plan Lanzamiento, con la API de Precios de Mercado (subdominio `agro`), sus planes Consulta, Mayorista e Integración, y la consumidora Distribuidora San Lucas.
- **Otras organizaciones:** Cafetalera del Altiplano (Prueba; lucia.cotom@cafetaleraaltiplano.com), Transportes Petén (en gracia; mario.pop@transportespeten.com) y Datos Chapines (suspendida; gabriela.sac@datoschapines.com). El propietario de Agro Precios es carlos.tzul@agroprecios.com. Para que A6.2 cuente 1, 2 y 1 APIs, tienen APIs en borrador: «API de Lotes de Café» (`lotes-cafe`), «API de Rutas de Carga» (`rutas-carga`), «API de Seguimiento de Flota» (`seguimiento-flota`) y «API de Datos Demográficos» (`datos-demograficos`).
- **Casos:** CAS-100 a CAS-104, tal como aparecen en A6.4 y A7.1. Si otra organización ya tiene alguno de esos números (un caso creado antes de sembrar), los de la demostración toman los siguientes libres. Al terminar, la secuencia queda lista para el siguiente caso.
- **Fechas:** son relativas al día en que se siembra, y cada grupo de pantallas tiene su «hoy» en los mockups. A6, B1.1, B1.3, B1.4 y los casos usan el 13 de septiembre: los ciclos de plataforma empiezan 20 (Envíos Xelajú), 12 (Agro Precios), 16 (Cafetalera), 32 (Transportes Petén) y 39 (Datos Chapines) días antes, y cada pago cubre el ciclo que empieza ese día; una renovación rechazada, el siguiente. B1.2 y B2 usan el 22 de septiembre para las suscripciones de los consumidores.
- **Reinicio (`--reiniciar`):** borra y vuelve a crear los datos de las cinco organizaciones de la demostración, conserva sus identificadores y no borra las cuentas del personal ni de los proveedores, porque pueden haber dejado casos, mensajes o reversiones en otras organizaciones: las restablece. La bitácora es solo de inserción, así que no repite las entradas que ya existen.
- **Consumo:** los datos de consumo de los últimos 30 días se generan con la misma forma que los de B1.1 y B2.1.
- **Contraseñas:** todas las cuentas de la siembra usan la contraseña `Shapi2026!demo`, que se documenta solo en el manual técnico.

08 · Compuerta de tráfico (plano de datos)

La compuerta es el proceso `Shapi.Compuerta`: ASP.NET Core con YARP y una tubería de filtros. **Solo depende de Redis** (RNF-02, RNF-04).

1. Contrato de entrada

- **Host:** `{sub}.api.shapi.localhost` o un dominio propio verificado.
- **Clave:** en la cabecera `X-Api-Key: shp_prod_…` o `shp_prueba_…`. **No se aceptan claves en la query string**, para que no queden en los registros de acceso. Si el nombre o el valor de un parámetro de la query contiene una clave con el formato de §2, sola o dentro de un texto más largo, la compuerta responde 401 `clave_en_url` y no reenvía la petición, aunque también venga `X-Api-Key`. Los demás parámetros (por ejemplo, un `key` del proveedor) se reenvían. La compuerta tampoco registra la URL de destino con su query.
- **Ruta y método:** los de la especificación del proveedor. El patrón se compara con la sintaxis de OpenAPI (por ejemplo, `/guias/{numero}` coincide con `/guias/GT123`). Si dos patrones coinciden, gana el más específico: primero el que tiene más segmentos literales y, si empatan, el que tiene menos parámetros. Los segmentos literales no distinguen mayúsculas, porque muchos orígenes tampoco las distinguen: así `/guias/RECIENTES` no se salta una ruta oculta `/guias/recientes`.
- **Cuerpo:** máximo 10 MB (10 × 1024 × 1024 bytes). Si es más grande se responde 413 `cuerpo_demasiado_grande`. Si el `Content-Length` lo anuncia, se rechaza antes de leer Redis; si no lo anuncia (cuerpo por partes), se corta al pasar el límite mientras se reenvía.
- **Tiempo de espera del origen:** 30 segundos **en total**, desde que se reenvía la petición hasta que termina la respuesta; no es un tiempo de inactividad. Si vence antes de que el origen responda, 504 `origen_sin_respuesta`. Si la respuesta ya empezó, se corta. Dentro de esos 30 segundos, la conexión con el origen tiene 10 segundos, incluida la resolución del nombre; si no se conecta, 502 `origen_inaccesible` (no 504, aunque el cliente lo reporte como tiempo vencido).
- **Salud:** `GET /salud` solo responde cuando el `Host` es `localhost`, para no tapar una ruta `/salud` de las APIs. Con cualquier otro host, la petición pasa por la tubería.

2. Formato de la clave

`{prefijo}{26 caracteres base62}`, con prefijo `shp_prod_` o `shp_prueba_`. Por ejemplo, `shp_prod_4fN8qT2xLm6Rv0Zk9Wd3Hs7c2e` (unos 154 bits de entropía). Se genera con `RandomNumberGenerator`. En la base de datos se guarda `SHA-256(clave completa)` en hex, el prefijo y los últimos 4 caracteres (ADR-01).

3. Tubería de filtros

Cada filtro implementa `IFiltroCompuerta` y devuelve `Continuar` o `Rechazar(código, error)`. El orden se define en un solo lugar (`TuberiaCompuerta`), y **agregar una regla es agregar una clase y una línea** (RNF-13).

Diagrama 17 · Vista general

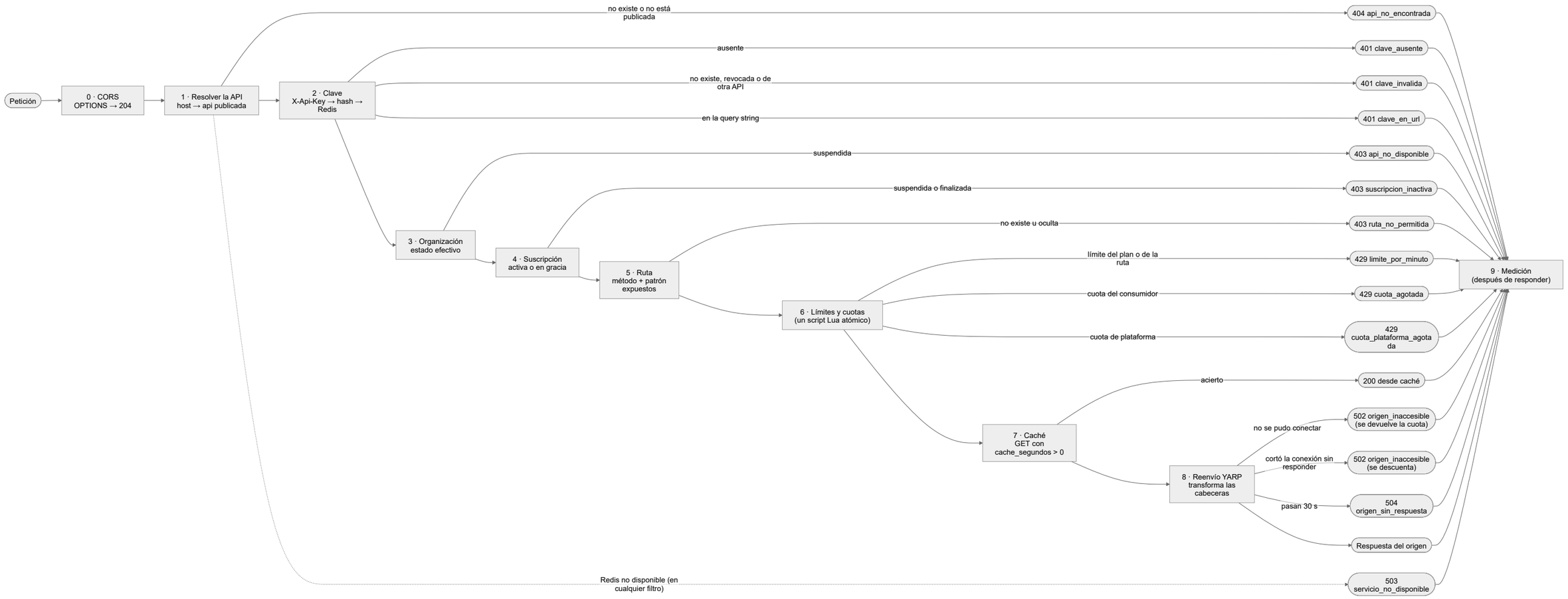


Diagrama 17 · Detalle 1 de 6 · Fila 1, columna 1

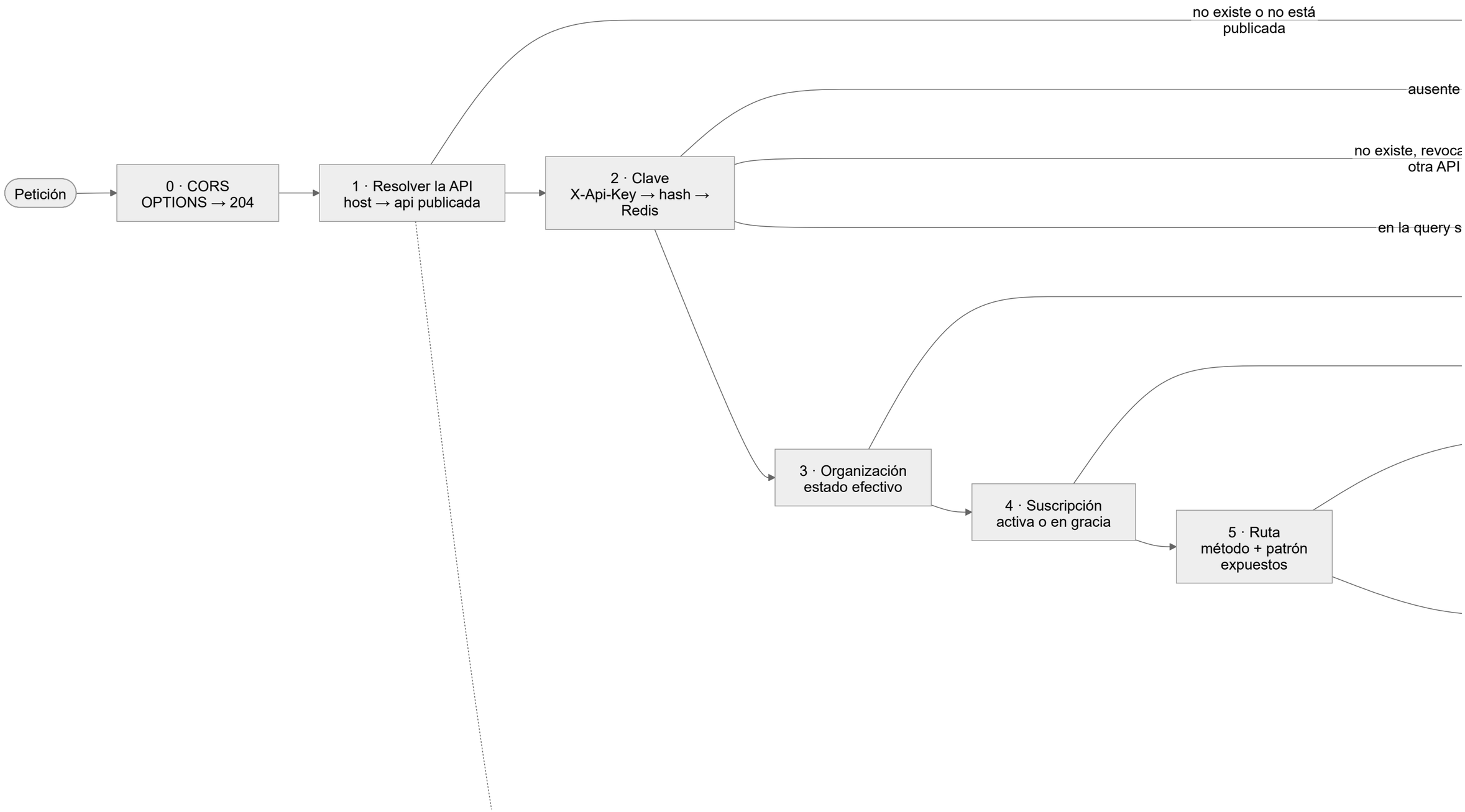


Diagrama 17 · Detalle 2 de 6 · Fila 1, columna 2

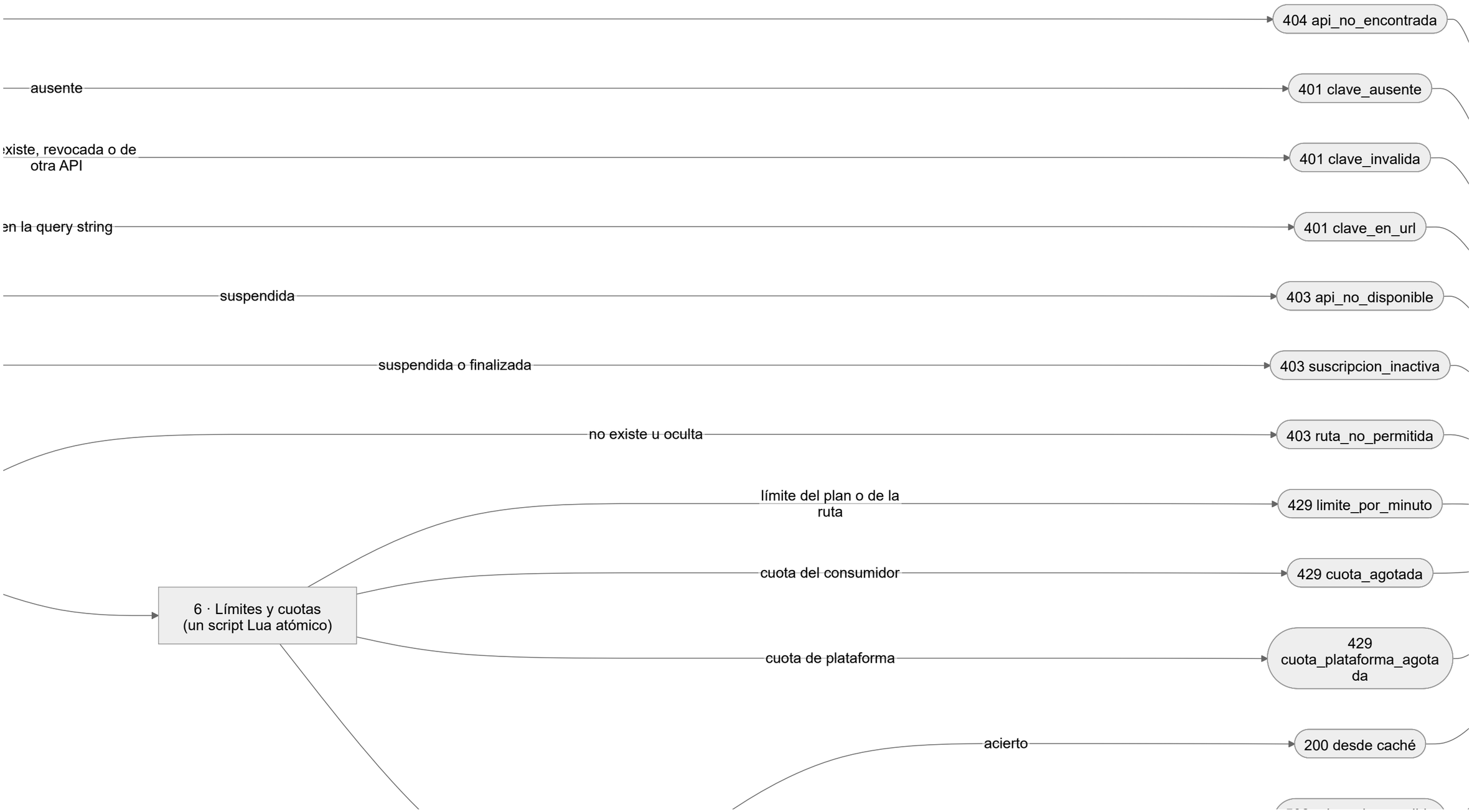


Diagrama 17 · Detalle 3 de 6 · Fila 1, columna 3

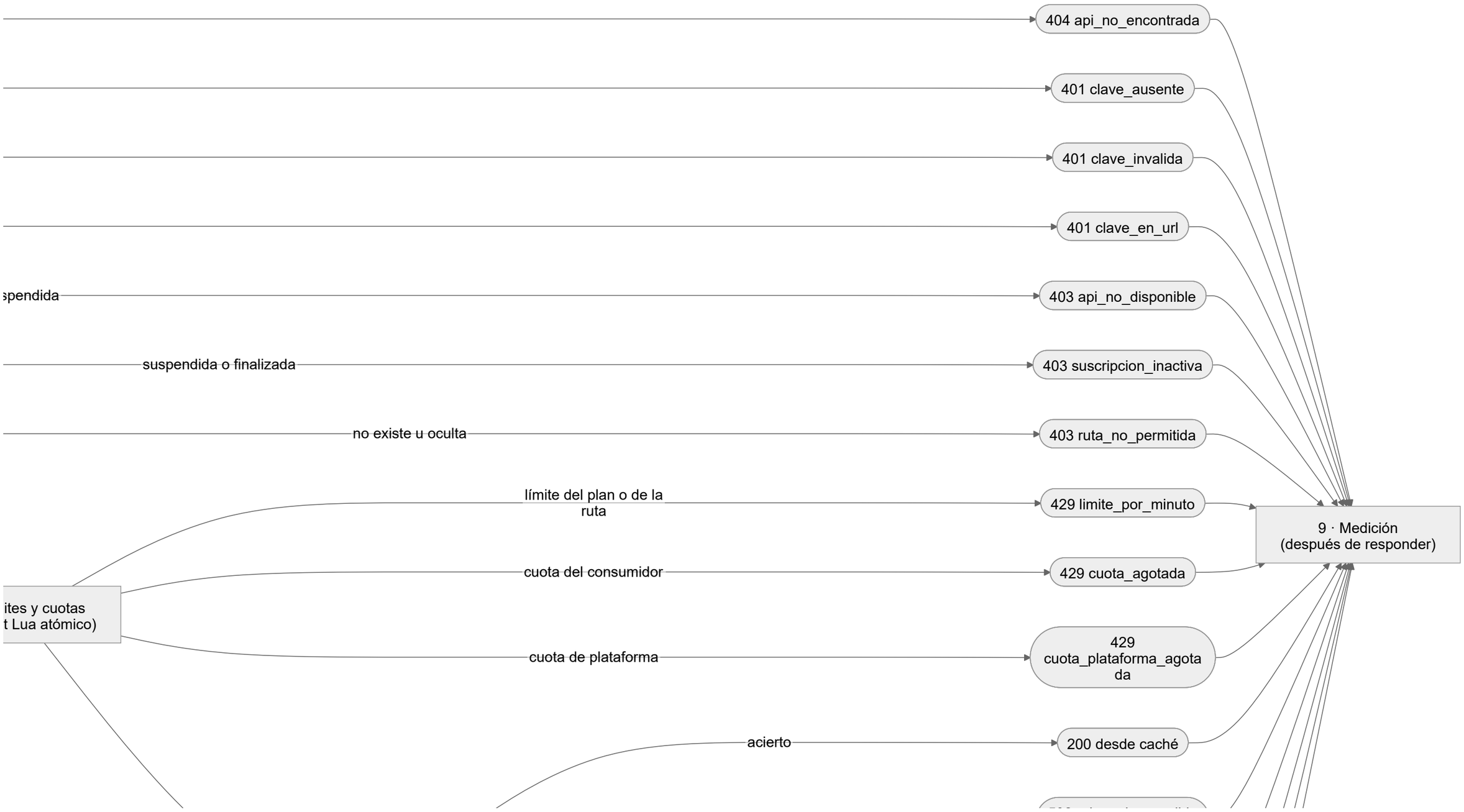


Diagrama 17 · Detalle 4 de 6 · Fila 2, columna 1

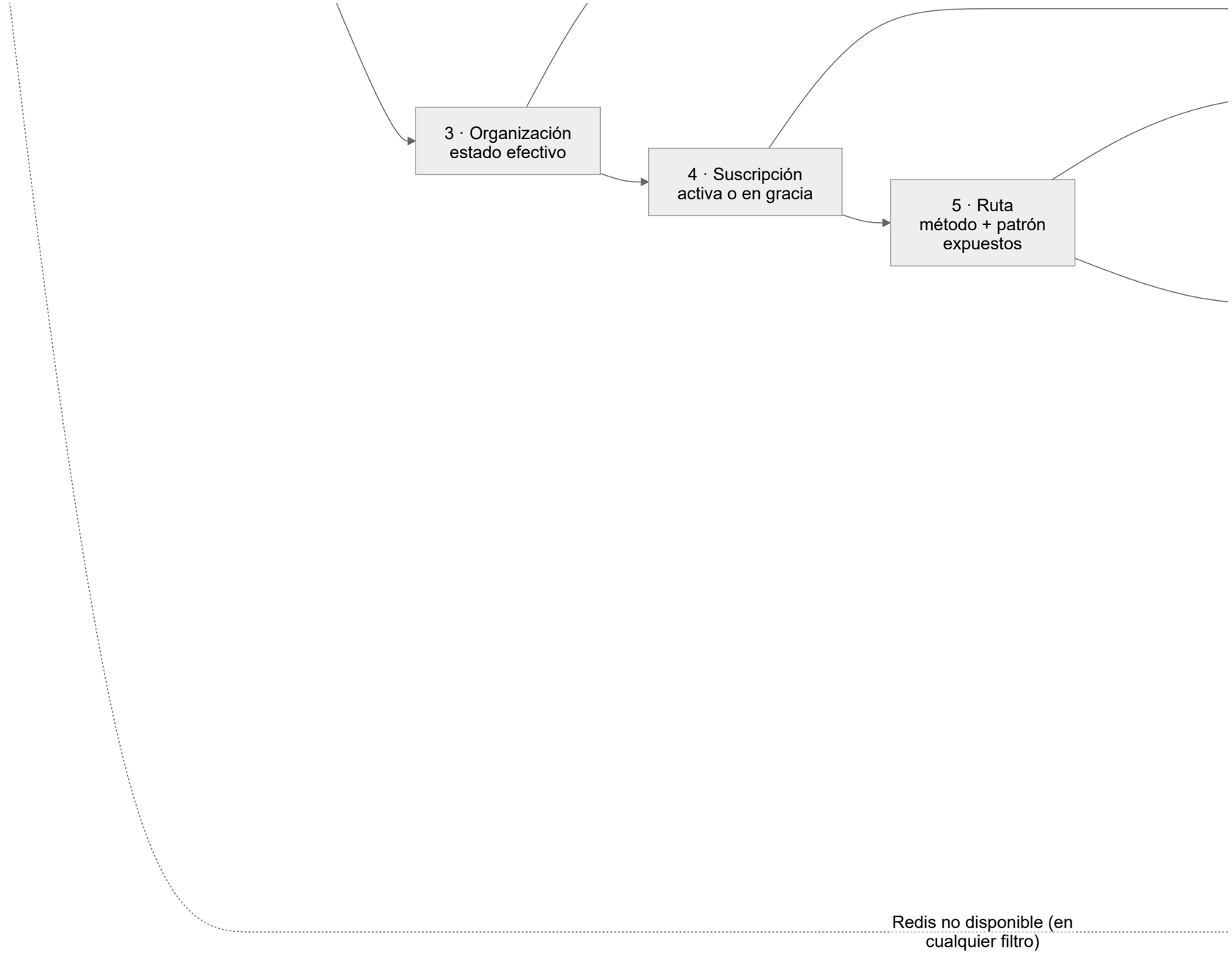


Diagrama 17 · Detalle 5 de 6 · Fila 2, columna 2

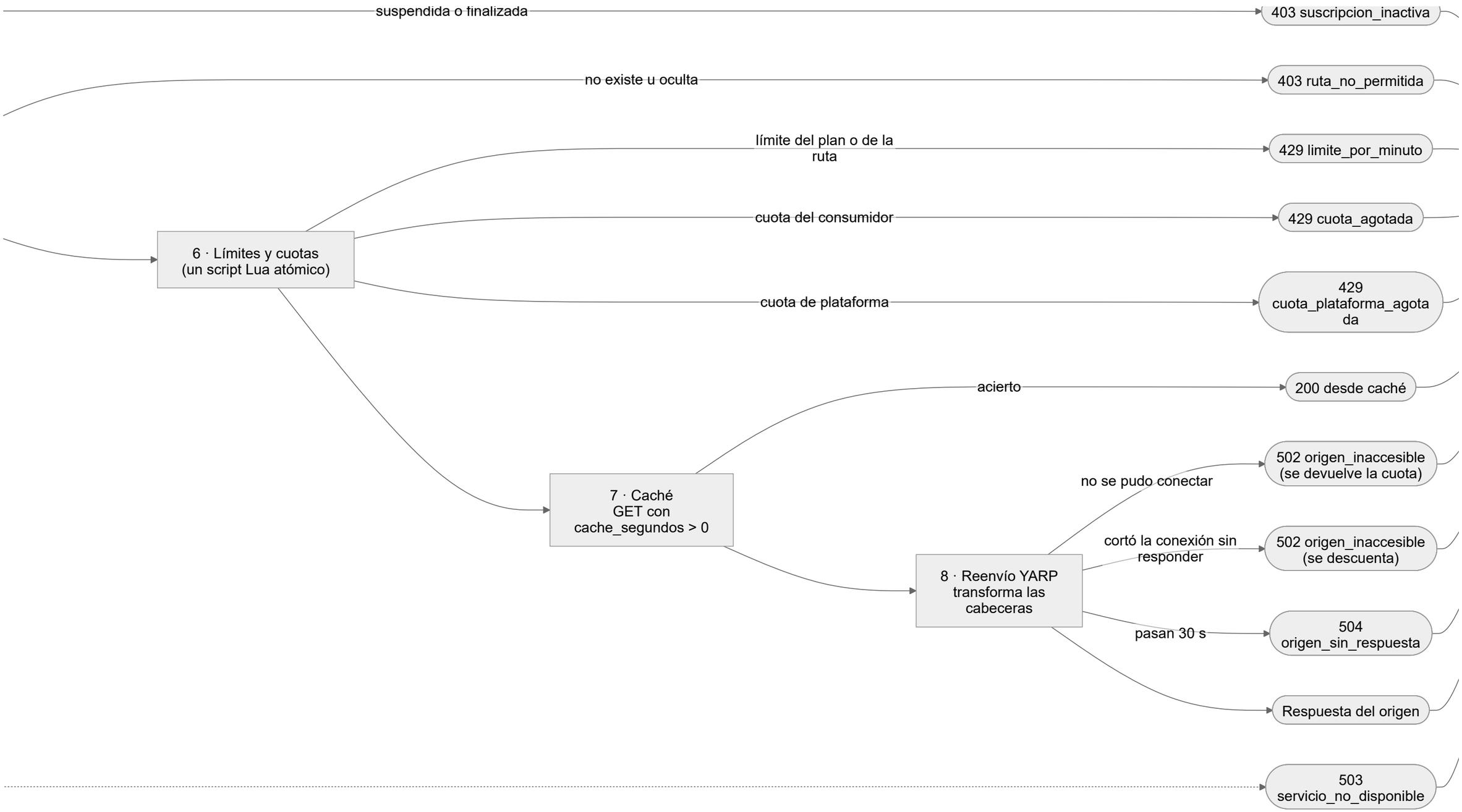
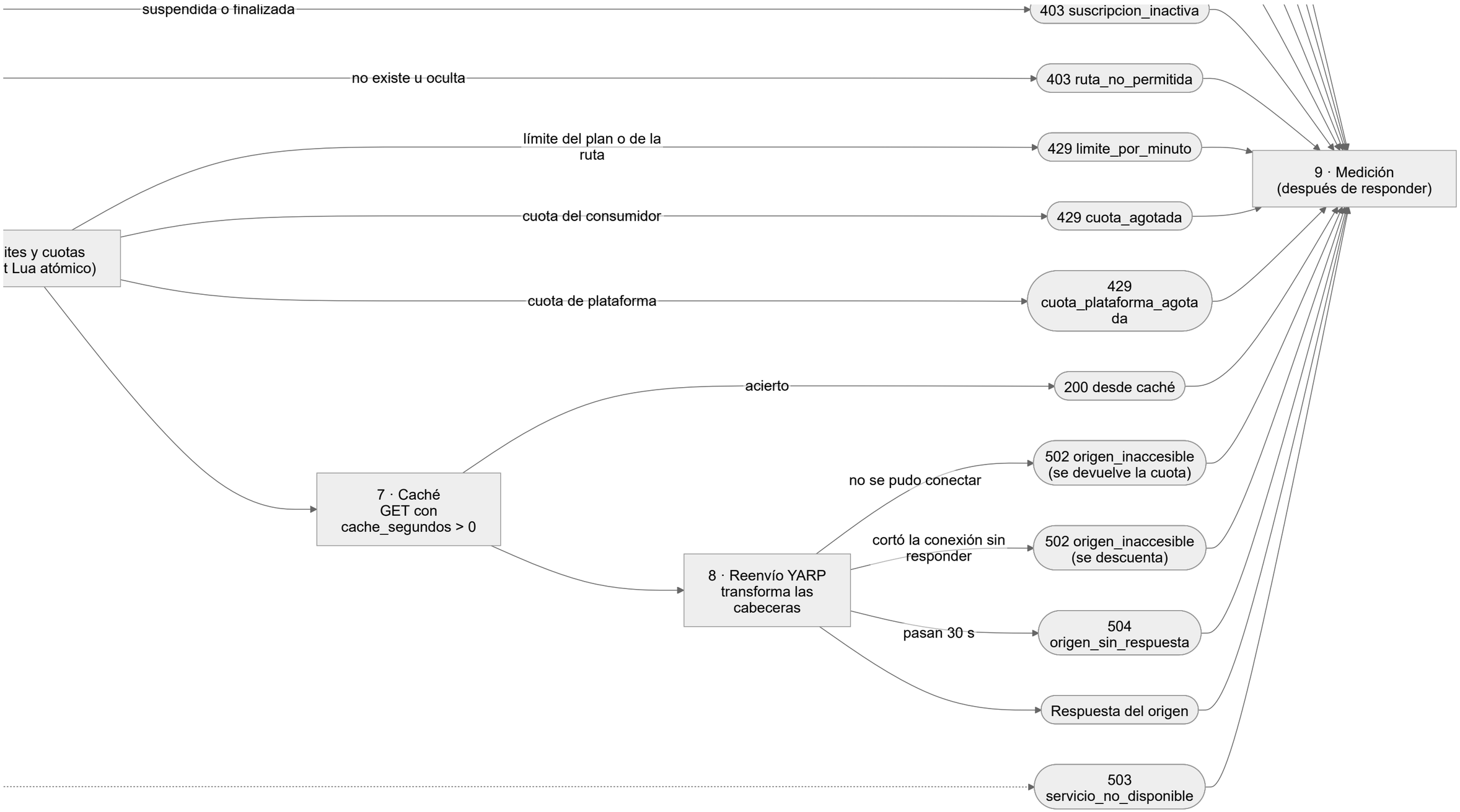


Diagrama 17 · Detalle 6 de 6 · Fila 2, columna 3



Detalle de cada filtro

	Filtro	Qué hace	Datos que usa (Redis)
0	`FiltroCors`	Responde el *preflight* (`OPTIONS` con `Origin` y `Access-Control-Request-Method`) sin clave, con 204 y las cabeceras de §6. A las demás respuestas les agrega `Access-Control-*` . Si el host no tiene una API publicada, el *preflight* sigue y el filtro 1 responde 404	`api:{id}.portal_host`
1	`FiltroApi`	`host → api_id → api:{id}` . Exige `estado = publicada`	`api:host:{host}` , `api:{id}` , `api:{id}:rutas`
2	`FiltroClave`	Rechaza una clave en la query string (§1). Calcula el SHA-256 de `X-Api-Key` y busca `clave:{hash}` . Exige que `api_id` coincida con la API resuelta; si no coincide, responde `clave_invalida` , porque no se aceptan claves de otra API	`clave:{hash}`
3	`FiltroOrganizacion`	Exige `estado_efectivo = activa` . Si `org:{id}` no está en Redis, el estado no se puede comprobar y también se rechaza	`org:{id}`
4	`FiltroSuscripcion`	Exige que `estado ∈ {activa, en_gracia}` . Una suscripción finalizada no tiene `susc:{id}` (07 §4), así que una llave ausente se rechaza igual. **No compara fechas**: los cambios de estado los hace el trabajador cada minuto (CU-16). Así, si el trabajador está caído, el servicio sigue funcionando en vez de cortarse (RNF-04), y el reloj del modo demostración no afecta a la compuerta	`susc:{id}`
5	`FiltroRuta`	Busca, entre todas las rutas del método, la que mejor coincide con el camino según la regla de especificidad de §1, y exige que esté `expuesta = true` . Si la que gana está oculta, se rechaza aunque un patrón más general esté expuesto: ocultar `/guias/recientes` no sirve si `/guias/{numero}` la deja pasar (RF-10). En un empate total gana la oculta	`api:{id}:rutas` (ya cargado)
6	`FiltroLimitesYCuotas`	Ejecuta el script Lua `evaluar_limites.lua` (ver abajo). Con la clave de pruebas usa los límites fijos de RF-45 y no toca las cuotas	`rl:*` , `cuota:*`
7	`FiltroCache`	Solo aplica a GET con `cache_segundos > 0` . Si encuentra `cache:…` , responde desde ahí; si no, marca la respuesta para guardarla si el origen devuelve 200	`cache:*`
8	Reenvío (YARP)	Destino = `url_origen` + el camino y la query. Quita `X-Api-Key` y las cabeceras `X-Shapi-*` que haya puesto el cliente. Agrega `X-Shapi-Consumidor: {consumidor_id}` , `X-Shapi-Entorno: produccion|pruebas` , `X-Shapi-Secreto: {secreto}` (si la API lo tiene) y `X-Forwarded-For/Proto/Host` (§5). Usa un `SocketsHttpHandler.ConnectCallback` que **rechaza direcciones internas** y se conecta a una de las direcciones ya validadas (RNF-10, 10 §4)	`api:{id}.url_origen` , `.secreto`

	Filtro	Qué hace	Datos que usa (Redis)
9	`MedicionMiddleware`	Siempre se ejecuta y va **al final**. Incrementa `met:{…}` en un *pipeline* sin esperar la respuesta de Redis (*fire-and-forget*) y agrega la llave a `met:pendientes`	`met:*`

Caché de respuestas (RF-13)

Solo se consultan y guardan respuestas de GET en rutas con `cache_segundos > 0`. La llave es la de 07 §4: método en mayúsculas, camino escapado y query tal como llegan. La caché es compartida entre consumidores y entornos; la validación y la reserva de cuota siempre ocurren antes del acierto. `X-Shapi-Cache` vale `HIT` o `MISS` y reemplaza cualquier valor del origen.

Se guardan únicamente respuestas 200 completas de hasta 1 MB (1,048,576 bytes), con su cuerpo binario y las cabeceras de la respuesta. La copia se limita a ese tamaño sin detener la transmisión de respuestas mayores. Las respuestas que establecen cookies (`Set-Cookie`), declaran `Cache-Control: private`, `no-store` o `no-cache`, o varían según cabeceras distintas de `Origin` (`Vary`, incluido `*`) se transmiten sin guardarse. Así no se comparten cookies, datos privados ni, por ejemplo, cuerpos gzip con clientes que no los aceptan; la llave contractual no incluye variantes por cabecera y la compuerta no implementa revalidación. No se almacenan cabeceras de conexión, ni las de cuota, plan, caché o CORS: estas últimas se calculan para cada petición. `Vary: Origin` también se agrega para la petición actual, sin acumularlo desde la copia.

El hash almacena la `version` de configuración que obtuvo la respuesta. Una entrada con otra versión se considera un fallo y se reemplaza al guardar la respuesta nueva; las entradas que no se vuelven a consultar desaparecen por su TTL. Un cambio de versión invalida de forma conservadora todas las respuestas de esa API, también durante una petición en vuelo: una respuesta de la versión anterior nunca produce un acierto en la nueva. Cuerpo, cabeceras, versión y TTL se escriben en una sola transacción. Un fallo de Redis al consultar sigue el 503 habitual; al guardar, cuando la respuesta ya se transmitió, solo se registra el fallo sin cambiarla.

Script `evaluar_limites.lua`

Es una sola llamada a Redis, que es atómica:

```
ENTRADAS: rl_s, rl_r (opcional), cuota_s, cuota_o (opcional), limite_plan, limite_ruta, peso,
cuota_llamadas, cuota_peticiones, ttl_s, ttl_o
1. n_s = INCR rl_s (EXPIRE 120 si es la primera vez); si n_s > limite_plan         → DECR rl_s; devolver {1,
contadores}  (limite_por_minuto, del plan)
2. si rl_r: n_r = INCR rl_r (EXPIRE 120); si n_r > limite_ruta                     → revertir 1 y 2;
devolver {2, contadores}  (limite_por_minuto, de la ruta)
3. c_s = INCRBY cuota_s peso (EXPIREAT ttl_s); si c_s > cuota_llamadas             → revertir 1, 2 y 3;
devolver {3, contadores}  (cuota_agotada)
4. si cuota_o: c_o = INCR cuota_o (EXPIREAT ttl_o); si c_o > cuota_peticiones                  → revertir 1
a 4; devolver {4, contadores}  (cuota_plataforma_agotada)
5. devolver {0, contadores}

contadores = n_s, n_r, c_s, c_o, leídos después de reservar o de revertir (0 si la llave no aplica)
```

- Como la reserva es atómica, **la cuota nunca se excede**, ni siquiera con peticiones concurrentes (ADR-21).
- Si el origen **no se pudo conectar** (502), la compuerta devuelve la reserva: `DECRBY cuota_s peso` y `DECR cuota_o`. Los contadores por minuto no se devuelven. Solo se devuelve si la petición no llegó al origen: la dirección se rechazó o no resolvió, el origen no aceptó la conexión, falló la conexión segura (TLS) o

venció el tiempo de conexión. Si el origen aceptó la conexión, recibió la petición y la cortó antes de responder, también es 502 `origen_inaccesible`, pero la cuota **se descuenta**.

- Los errores 4xx y 5xx del origen y los 504 **sí descuentan**, porque la petición llegó al origen.
- Las respuestas desde caché **descuentan**, porque el consumidor recibió los datos.
- El límite por minuto usa una ventana fija de 60 segundos alineada al minuto (`minuto_epoch = floor(unix/60)`).
- `rl_r` solo se usa si la ruta tiene `limite_minuto`, y `cuota_o` solo si `org:{id}` tiene `cuota_peticiones` y ciclo: la organización de la plataforma no tiene cuota de plataforma (07 §4).
- `ttl_s` y `ttl_o` son `fin + 8 días`. Si el `fin` ya pasó (un ciclo que el trabajador todavía no cierra, RNF-04), se cuentan 8 días desde ahora: un `EXPIREAT` en el pasado borraría el contador y reiniciaría la cuota en cada petición.
- **Modo demostración** (09 §9): el `inicio` y el `fin` de `susc:{id}` y de `org:{id}` vienen en la hora de `IReloj`, que puede ir adelantada, mientras la compuerta usa la hora real. La cuota se cuenta bien, porque la llave depende del `inicio`, pero el `Retry-After` de `cuota_agotada` y de `cuota_plataforma_agotada` se mide contra la hora real y puede salir más largo que en el reloj de la demostración. `X-Cuota-Reinicio` y la fecha del mensaje de `cuota_agotada` muestran el `fin` tal cual, en la hora de la demostración. Se acepta así.
- Al rechazar, el script devuelve también los contadores ya revertidos, para las cabeceras de §5. Así no hace falta otra llamada.
- La compuerta lo llama siempre con `EVALSHA`. Si Redis no tiene el script, porque se reinició o vació su caché de scripts, responde `NOSCRIPT` sin ejecutar nada. Entonces la compuerta lo ejecuta una vez con `EVAL`, que lo deja guardado.
- **Clave de pruebas** (RF-45): el mismo script, con `rl_s = rl:p:{clave_id}:{minuto_epoch}` y `limite_plan = 10`, sin `rl_r` ni `cuota_o`, y con el contador diario `dia:p:{clave_id}:{aaaammdd}` en el lugar de `cuota_s` (`peso = 1` y límite 1,000). No se usa el límite de la ruta. Al pasar las 1,000 peticiones del día se responde 429 `cuota_agotada`, con `Retry-After` hasta la medianoche de Guatemala. Como no toca las cuotas, un 502 no devuelve nada.

4. Contrato de errores

Todos los rechazos de la compuerta responden con `Content-Type: application/json; charset=utf-8`:

```
{
  "error": {
    "codigo": "cuota_agotada",
    "mensaje": "Agotó las 50,000 llamadas de su plan Comercio en este ciclo. La cuota se renueva el 1 de
octubre de 2026.",
    "estado": 429
  }
}
```

Estado	Código	Cuándo	Cabeceras adicionales
401	`clave_ausente`	Falta `X-Api-Key`	`WWW-Authenticate: ApiKey header="X-Api-Key"`

Estado	Código	Cuándo	Cabeceras adicionales
401	`clave_invalida`	La clave no existe, está revocada, era una clave rotada que ya venció o es de otra API. También si llegan varias `X-Api-Key`	ídem
401	`clave_en_url`	Un parámetro de la query string contiene una clave (§1)	ídem
403	`api_no_disponible`	La organización proveedora está suspendida (por falta de pago o por el administrador)	—
403	`suscripcion_inactiva`	La suscripción del consumidor está suspendida o finalizada	—
403	`ruta_no_permitida`	El método y la ruta no existen o están ocultos	—
404	`api_no_encontrada`	El host no corresponde a ninguna API publicada	—
413	`cuerpo_demasiado_grande`	El cuerpo pesa más de 10 MB	—
429	`limite_por_minuto`	Se pasó el límite de peticiones por minuto del plan o de la ruta	`Retry-After` (segundos para el siguiente minuto)
429	`cuota_agotada`	Se agotó la cuota de llamadas del ciclo. Con la clave de pruebas, se pasaron las 1,000 peticiones del día	`Retry-After` (segundos para que termine el ciclo o, con la clave de pruebas, para la medianoche de Guatemala; 1 si el ciclo ya terminó)
429	`cuota_plataforma_agotada`	El **proveedor** agotó la cuota de peticiones de su plan de plataforma	`Retry-After` (segundos para que termine el ciclo de plataforma)
502	`origen_inaccesible`	No se pudo conectar con el origen en 10 segundos, el origen apunta a una dirección prohibida, o cortó la conexión sin responder	—
503	`servicio_no_disponible`	Redis no está disponible: la compuerta no puede validar la petición (RNF-04)	`Retry-After: 5`
504	`origen_sin_respuesta`	El origen no respondió en 30 segundos (si la respuesta ya había empezado, se corta)	—

Las respuestas del **origen** se devuelven tal cual, incluidos sus errores. Las métricas las cuentan como `origen_4xx` y `origen_5xx`.

5. Cabeceras

En las respuestas que traen una clave válida (incluidos los 429). Son las que llegan al filtro 6: un rechazo de los filtros 1 a 5 no las lleva. También las llevan los 502 y los 504 del reenvío, y reemplazan las que mande el origen con el mismo nombre:

Cabecera	Valor
`X-Shapi-Plan`	Nombre del plan, por ejemplo `Comercio`. En las claves de pruebas es `Pruebas`. Va codificado como componente de URI, en UTF-8 con porcentajes, porque una cabecera solo admite ASCII: `Básico` viaja como `B%C3%A1sico` y se lee con `decodeURIComponent`
`X-RateLimit-Limit`	El límite por minuto que aplica (el menor entre el del plan y el de la ruta)
`X-RateLimit-Remaining`	Peticiones que quedan en el minuto actual: la menor entre lo que queda del límite del plan y del de la ruta
`X-RateLimit-Reset`	Segundos hasta el siguiente minuto
`X-Cuota-Limite`	Llamadas del ciclo según el plan
`X-Cuota-Restante`	Llamadas que quedan en el ciclo
`X-Cuota-Reinicio`	Fecha ISO 8601 en UTC en que termina el ciclo, por ejemplo `2026-10-31T06:00:00Z`
`X-Shapi-Cache`	`HIT` o `MISS` (solo en las rutas con caché)

Con la clave de pruebas, `X-RateLimit-Limit` es 10, y las `X-Cuota-*` informan el límite diario: `X-Cuota-Limite: 1000`, las peticiones que quedan en el día y la medianoche de Guatemala en que se renueva.

Hacia el origen: `X-Shapi-Consumidor`, `X-Shapi-Entorno`, `X-Shapi-Secreto` y `X-Forwarded-*`. **Nunca** se envían `X-Api-Key` ni las cookies de sesión de Shapi: de la cabecera `Cookie` se quitan `shapi_sesion` y `portal_sesion`, y las demás cookies pasan. El `Host` que recibe el origen es el de `url_origen`, no el de la API en Shapi. El host original viaja en `X-Forwarded-Host`. Cualquier `X-Shapi-*` que mande el cliente se quita, así que no puede cambiar el consumidor, el entorno ni el secreto.

- `X-Forwarded-For`: la cadena que haya llegado, más la IP de quien se conectó a la compuerta.
- `X-Forwarded-Proto`: el que mande el borde (Caddy), que le habla a la compuerta por `http`; si no llega `http` o `https`, el esquema de la conexión.
- `X-Forwarded-Host`: el `Host` con que llegó la petición.

6. CORS

- `Access-Control-Allow-Origin`: solo el host del portal de esa API (`https://{sub}.shapi.localhost`). Se acepta un `Origin` cuyo host es el `portal_host` de la API, con `http` o `https` y cualquier puerto, y se devuelve tal como llegó. Las peticiones sin `Origin`, que son las de servidor a servidor, pasan sin restricción. Las de otro origen pasan, pero sin las cabeceras de CORS.
- `Access-Control-Allow-Methods`: los de las rutas expuestas (solo en el *preflight*).
- `Access-Control-Allow-Headers`: `X-Api-Key, Content-Type, Accept` (solo en el *preflight*).
- `Access-Control-Expose-Headers`: todas las de §5 (en las demás respuestas).
- `Access-Control-Max-Age: 600` (solo en el *preflight*).
- El *preflight* de otro origen también recibe 204, sin esas cabeceras.
- Las cabeceras se agregan a todas las respuestas, también a los rechazos de §4, para que el portal pueda leer el error. Hay dos excepciones, porque salen antes de conocer la API: el 413 de un `Content-Length` de más de 10 MB y el 503 de Redis no disponible. Las `Access-Control-*` que mande el origen se

quitan: el origen no puede abrir el CORS a otros sitios. Toda respuesta a una petición con `Origin` lleva `Vary: Origin`.

7. Medición y consolidación

Durante cada petición, el filtro 9 hace `HINCRBY` sobre la llave `met:{aaaammdd}:{api}:{ruta|-}:{susc|-}:{entorno}`:

- `peticiones +1`.
- `llamadas +peso`, solo si la petición descontó cuota.
- `bytes_entrada` y `bytes_salida`.
- El contador del código correspondiente: `r401`, `r403`, `r404`, `r429`, `o2xx`, `o3xx`, `o4xx`, `o5xx` u `ofallo`.
- `h_t_{i}` y `h_c_{i}`, que son el rango del histograma para la latencia total y para la de la compuerta.
- `lt_suma` y `lc_suma`.

La **latencia de la compuerta** es la latencia total menos el tiempo de espera del origen, medido desde que se envía la petición hasta que llega el primer byte de la respuesta.

Los incrementos de una petición y su `SADD met:pendientes` se envían juntos en un pipeline `MULTI/EXEC` con *fire-and-forget*. Así la consolidación nunca separa una petición entre dos lotes. Los bytes son los cuerpos efectivamente leídos y escritos, sin almacenar su contenido. El día se toma al comenzar la petición, en Guatemala. Si el origen falla sin responder, la espera termina al producirse el fallo; sus 502 y 504 incrementan `ofallo`, mientras que un 502 devuelto por el propio origen incrementa `o5xx`. Las claves rechazadas no atribuyen consumo a una suscripción, aunque el lector haya encontrado una clave de otra API.

Si no se pudo resolver una API (por ejemplo, un host desconocido o un cuerpo rechazado antes de leer Redis), `{api}` es el UUID nulo `00000000-0000-0000-0000-000000000000`, `{ruta}` y `{susc}` son `-` y el entorno es `produccion`. Esos contadores globales se conservan en Redis; no se insertan en `consumo_diario`, que exige una API real. No se atribuyen a un proveedor.

Consolidación (el trabajador, cada 10 segundos, es idempotente; ver la secuencia en 06 §5.7):

1. `lote_id = nuevo UUID`. Por cada llave de `met:pendientes`: `SREM` y luego `RENAME llave → met:lote:{lote_id}:{llave}`. Si la llave ya no existe, se ignora. El `SREM` y los `RENAME` del lote se ejecutan en un único script Lua, que primero valida todas las llaves y cierra la instantánea antes de que pueda leerla otro trabajador. Cada ciclo lee una sola vez las llaves pendientes de APIs identificadas y las divide en grupos de hasta 256, con un nuevo UUID por grupo. Las llaves que llegan después de esa lectura esperan al siguiente intervalo: el tráfico continuo no prolonga el ciclo. Si falla la aplicación de un lote nuevo, no se separan más grupos en ese ciclo; sus llaves permanecen en `met:pendientes`. Una interrupción no deja un hash sin pertenecer a `met:pendientes` o a un lote recuperable.
2. Lee todos los `met:lote:{lote_id}:*` y abre una transacción en PostgreSQL:
   - `INSERT INTO lote_consolidado(lote_id)`;
   - `INSERT … ON CONFLICT (fecha, api_id, ruta_id, suscripcion_id, entorno) DO UPDATE SET` sumando cada contador y cada posición de los arreglos del histograma;
   - `COMMIT`.
3. `DEL met:lote:{lote_id}:*`.
4. **Recuperación:** al arrancar, el trabajador busca `met:lote:*`. Si el `lote_id` ya está en `lote_consolidado`, solo borra las llaves; si no, repite los pasos 2 y 3.

La recuperación se repite también al principio de cada intervalo de 10 segundos, para recuperarse de fallos temporales sin reiniciar. El marcador usa `ON CONFLICT DO NOTHING` dentro de la misma transacción; un lote ya aplicado no suma contadores otra vez, incluso si otro trabajador lo recupera simultáneamente. Cada lote deduplica las llaves devueltas por `SCAN` antes de leer sus hashes: Redis puede devolver una misma llave varias veces. Al consolidar, una ruta retirada se trata como `ruta_id` nulo, conforme a 07 §3.5. El UPSERT conserva `creado_en` y actualiza `actualizado_en` con `IReloj`.

Si una API fue eliminada (por ejemplo al reiniciar la siembra de demostración), se descartan sus filas pendientes y el marcador registra que el lote se procesó; no se atribuyen a otra API. Si solo desapareció la suscripción, sus contadores se conservan bajo la API con `suscripcion_id` nulo. Las referencias vigentes se protegen hasta el commit. Un fallo de un lote recuperado no impide procesar los otros lotes recuperados ni intentar el primer lote nuevo: se conserva la instantánea fallida y, al terminar el ciclo acotado, se informa el error para reintentar en el próximo intervalo. Si el primer lote nuevo falla, se pausa la separación conforme al paso 1.

Consulta del consumidor (B2.1, RF-37): `GET /api/portal/consumo` exige sesión de consumidor y resuelve la API publicada por el host. Solo devuelve su suscripción no finalizada a esa API, aislada por organización y consumidor. Sin suscripción responde 404 `suscripcion_no_encontrada`; un portal inexistente, despublicado o de otra organización responde 404 `api_no_encontrada`.

El resumen lee las llamadas usadas de `cuota:susc:{suscripcion_id}:{inicio_epoch}` en Redis, con el inicio persistido del ciclo. Una llave ausente significa cero; un error de Redis responde 503 `servicio_no_disponible`, nunca consumo cero. La cuota es la del plan vigente; el restante es `max(0, cuota − usadas)` y el porcentaje es `usadas × 100 / cuota`. La respuesta lleva `Cache-Control: no-store`.

El detalle suma **solo producción** de `consumo_diario`, desde el día de inicio hasta el último día del ciclo en Guatemala. Si el ciclo todavía no se ha renovado y su fin ya pasó, abarca también hasta el día actual de `IReloj`, porque durante la gracia la compuerta sigue usando el mismo contador. La pantalla conserva el periodo contractual (inicio incluido y fin menos un día). Las llamadas descontadas se suman de las filas persistidas: no se reconstruyen multiplicando peticiones por el peso actual, porque hay rechazos y el peso puede haber cambiado. El peso mostrado es el actual de la ruta; para rutas retiradas o no identificadas es nulo y se muestra «—». Esas filas conservan sus peticiones y llamadas bajo «Sin ruta identificada».

Las rutas se ordenan por llamadas descendentes y luego por método y patrón. Los porcentajes por ruta usan el total de llamadas **consolidadas**, con cero si no hay llamadas. «Total del ciclo» suma las peticiones y llamadas del detalle, y muestra 100 % si se descontaron llamadas o 0 % en caso contrario. El detalle puede ir hasta diez segundos detrás del contador en tiempo real, conforme al intervalo de consolidación; el resumen no sustituye su contador por esa suma. La pantalla consulta de nuevo cada diez segundos mientras está abierta. Sin suscripción reproduce `B2/ConsumoVacia`; con suscripción sin uso mantiene el ciclo, la cuota completa y los totales en cero.

Cálculo del p95 (RF-35): se suman los histogramas de las filas del periodo y se busca el rango `i` donde la suma acumulada alcanza el 95 % del total. Luego se interpola linealmente entre el límite inferior y el superior de ese rango. El último rango (más de 2500 ms) se reporta como `> 2500 ms`.

Consulta del proveedor (B1.1, RF-35): `GET /api/apis/{apiId}/consumo` consulta solo `consumo_diario`, incluyendo producción y pruebas. `desde` y `hasta` son fechas inclusivas `AAAA-MM-DD`; `hasta` vale hoy en Guatemala por defecto, y `desde`, nueve días antes de `hasta`. El periodo admite hasta 366 días para acotar la respuesta. Un formato inválido, un inicio posterior al fin o un periodo mayor responde 400 `datos_invalidos`, con `errores` por campo. La serie incluye los días sin peticiones. Los porcentajes por ruta y por error usan todas las peticiones del periodo como denominador (cero cuando no hay peticiones). Las filas sin ruta o de rutas

retiradas se agrupan como «Sin ruta identificada» y conservan su consumo. Las rutas se ordenan por peticiones descendentes y luego por método y patrón. Sin muestras, el p95 es nulo; el rango abierto conserva `supera2500 = true`.

La tabla de errores distingue los rechazos 401/403/429 de la compuerta y los 4xx, 5xx y fallas de conexión o tiempo de espera del origen. Por incluir las fallas que no llegaron al servidor, el encabezado del segundo grupo dice «Errores de su servidor de origen», sin afirmar que todas las peticiones llegaron. Los bytes se presentan en GB decimales (1 GB = 1,000,000,000 bytes). El personal propietario, editor o lector de la organización dueña puede consultar; una API inexistente o de otra organización responde 404 `api_no_encontrada`.

B1.1 conserva las cuatro tarjetas del mockup y muestra también las llamadas descontadas como detalle de «Peticiones a la API», conforme a RF-35; este dato se omitía en el ejemplo anterior. La respuesta conserva la precisión decimal del p95; la pantalla lo presenta redondeado al milisegundo entero más cercano.

El ejemplo de B1.1 y su paquete se sincronizaron con la siembra al implementar JG-11: 93,147 llamadas descontadas, p95 agregado de la compuerta de 10 ms (suma de histogramas, en lugar de los 11 ms del ejemplo anterior) y errores del origen separados en 250 respuestas 4xx, 66 respuestas 5xx y cero fallas. Prevalecen RF-35 y el cálculo de este apartado, conforme al protocolo §C; las fechas del ejemplo siguen siendo relativas en la siembra (07 §6).

8. Rendimiento

Para RF-39, el mismo pipeline de medición incrementa también `salud:latencia-compuerta:{minuto_unix}` (07 §4), con campos `segundo_unix:rango` y TTL de 65 minutos. Esta medición global usa el reloj real y no participa en `met:pendientes` ni en la consolidación diaria; permite consultar la última hora aunque el trabajador esté caído, sin añadir viajes síncronos a Redis.

- Por cada petición se hacen tres viajes a Redis antes de reenviar, más uno en segundo plano para la medición:

  1. un *pipeline* con `api:host:{host}` y `clave:{hash}`;
  2. un *pipeline* con lo que depende de esos valores: `api:{id}`, sus rutas, `org:{id}` y `susc:{id}`;
  3. el script Lua de límites y cuotas.

  Si la ruta usa caché (filtro 7), se suma la consulta de `cache:*`. `org:{id}` y `susc:{id}` solo se piden si la clave existe y es de esa API; `org:{id}` es el `organizacion_id` de la clave, que es el de la API.

- La compuerta **no guarda en memoria** los datos de claves, suscripciones ni organizaciones. Así una revocación o una suspensión se aplica al instante (RF-28), a costa de consultar Redis en cada petición (ADR-22). Lo único que se guarda en memoria, durante 5 segundos como máximo, es `api:{id}:rutas`, junto con su `version`. Mientras las rutas están en memoria, el segundo *pipeline* no las pide. Si la `version` de `api:{id}` ya no es la de las rutas en memoria, se vuelven a pedir en un viaje más: solo pasa en la primera petición después de una publicación.

- Hay una conexión multiplexada a Redis (StackExchange.Redis) y un solo invocador HTTP de YARP (`HttpMessageInvoker` con `SocketsHttpHandler`), que agrupa las conexiones por destino (*pooling*). La conexión a Redis se abre al arrancar la compuerta, antes de atender peticiones. Si se abriera con la primera, varias peticiones simultáneas esperarían bloqueadas la conexión síncrona y agotarían el *pool* de hilos durante segundos. Si Redis no responde al arrancar, la compuerta espera `connectTimeout` (5 s por defecto), empieza a atender y responde 503 `servicio_no_disponible` mientras la conexión se reintenta sola.

- **Pruebas de aceptación:** RNF-01 y RNF-03 se miden con k6 contra `origen-envios` en el ambiente productivo simulado. Los resultados se documentan en el manual técnico.

09 · Cobros y suscripciones

1. Principios

- Hay **dos niveles**: la organización contrata con Shapi (plataforma) y el consumidor contrata con el proveedor (API). **Los dos niveles usan el mismo código**: la clase base `Suscripcion`, la misma máquina de estados y el mismo prorrateo.
- El único medio de pago es la **tarjeta**, procesada por la **pasarela simulada** a través de `IPasarelaPagos` (ADR-12).
- Shapi **nunca guarda** el número completo de la tarjeta ni el CVV. Guarda el **token** que emite la pasarela, la marca, los últimos 4 dígitos, el titular y el vencimiento (ADR-13).
- Toda la lógica de tiempo usa `IReloj`, para que las pruebas y el modo demostración puedan adelantar la fecha.

2. Pasarela simulada

```
public interface IPasarelaPagos
{
    Task<ResultadoTokenizacion> TokenizarAsync(DatosTarjeta tarjeta);                          // valida
y devuelve el token
    Task<ResultadoCobro> CobrarAsync(string token, decimal monto, string referencia, bool esRenovacion);
    Task<ResultadoReembolso> ReembolsarAsync(string referenciaCobro);
}
```

Reglas de la simulación (`PasarelaSimulada`)

Validación	Regla
Número	Sin contar espacios ni guiones, tiene entre 13 y 19 dígitos y pasa el **algoritmo de Luhn**. Si no, `numero_invalido`
Marca (según el BIN)	`4…` → Visa · `51–55…` o `2221–2720…` → Mastercard · `34…` o `37…` → American Express. Cualquier otro → `marca_no_soportada`
Vencimiento	Tiene que ser el mes actual o uno posterior, según la fecha en America/Guatemala. Si no, `tarjeta_vencida`
CVV	3 dígitos, o 4 si es American Express. Si no, `cvv_invalido`

Tarjetas de prueba

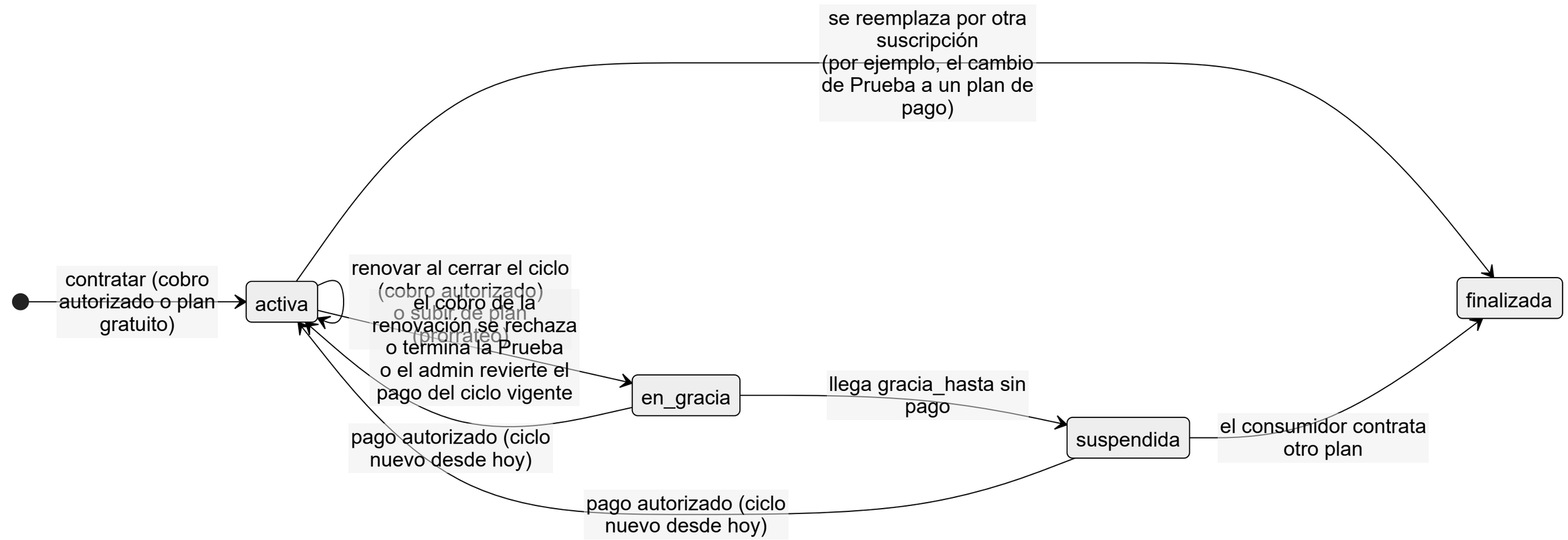
Número	Resultado
`4242 4242 4242 4242`	Visa · siempre se aprueba
`5555 5555 5555 4444`	Mastercard · siempre se aprueba
`4000 0000 0000 0002`	Visa · siempre se rechaza (`fondos_insuficientes`)

Número	Resultado
`4000 0000 0000 0069`	Visa · siempre se rechaza (`tarjeta_vencida`)
`4000 0000 0000 0341`	Visa · **se aprueba al contratar y se rechaza en las renovaciones** (`fondos_insuficientes`). Sirve para demostrar el periodo de gracia
Cualquier otro número válido según Luhn	Se aprueba. Por ejemplo, las tarjetas de los mockups: `4024 0071 2284 4821` y `5412 7534 1203 3057`

- Las tarjetas de prueba se reconocen por el **número completo**. Otra tarjeta que termine en `0002` , `0069` o `0341` se trata como cualquier otra.
- Con `SHAPI_PASARELA_FALLA=true` , todas las operaciones (tokenizar, cobrar y reembolsar) fallan con `pasarela_no_disponible` (B3 la muestra fuera de servicio).
- La pasarela simulada tarda entre 300 y 800 ms en responder, para que se note la espera en la interfaz. Con `Pagos:DemoraMs` se fija otra demora en milisegundos; las pruebas usan 0.
- El token tiene el formato `tok_sim_{uuid}` . El de una tarjeta de prueba especial lleva sus últimos 4 dígitos (`tok_sim_0002_{uuid}` , `tok_sim_0069_{uuid}` o `tok_sim_0341_{uuid}`), para que su comportamiento no se pierda si el proceso se reinicia y lo respete cualquier proceso que cobre, como el Trabajador en las renovaciones. La pasarela no guarda nada en memoria.
- Las referencias de cobro tienen el formato `ch_sim_{uuid}` y las de reembolso `re_sim_{uuid}` .
- Si se rechaza el cobro de una primera contratación de API, se registra `pago` con estado `rechazado` , monto, motivo, consumidor y API, sin medio de pago ni suscripción. Los rechazos de renovaciones se asocian a la suscripción existente. Un rechazo nunca activa claves ni una suscripción nueva.
- Si la pasarela responde `pasarela_no_disponible` al cobrar una contratación, no es un rechazo: se responde 503, como en la tokenización, y no se registra ningún pago.
- Si el cobro de una contratación se autorizó pero algo falla antes de confirmar la transacción, se reembolsa con `ReembolsarAsync` , para que no quede un cobro sin suscripción.

3. Máquina de estados de las suscripciones

Diagrama 18 · Vista general



Estado	¿Funciona la compuerta?	Qué muestra la interfaz
`activa`	Sí	"Activa", el periodo y la próxima renovación (B1.4, B2.3)
`en_gracia`	**Sí**	Aviso con los días que quedan y el botón "Pagar con otra tarjeta" (B1.4 gracia, B2.3 gracia)
`suspendida`	**No**: responde 403 (`api_no_disponible` si es la plataforma, `suscripcion_inactiva` si es la API)	"Su tráfico está detenido" y el botón para pagar (B1.4 suspendida). El consumidor ve "La API no está disponible temporalmente" si el suspendido es el proveedor (B2.3)
`finalizada`	No	Solo aparece en el historial

Parámetros (en `appsettings`, en la sección `Cobros`): `DiasGracia = 7`, `AvisoVencimientoPruebaDias = 7`, `ReintentosRenovacion = 0`. No hay reintentos automáticos: durante la gracia paga el usuario.

4. Ciclos

- `inicio` es el momento de la contratación, redondeado al **inicio del día en America/Guatemala**. `fin = inicio + vigencia_dias`.
- En pantalla se muestra `inicio – (fin – 1 día)` y "Próxima renovación: `fin`". Por ejemplo, un plan de 30 días contratado el 24 de agosto de 2026 se muestra como **24 ago 2026 – 22 sep 2026** y se renueva el **23 de septiembre de 2026**.
- Al renovarse, el ciclo nuevo empieza en el `fin` anterior. Al reactivarse después de gracia o suspensión, **empieza el día del pago**.
- La cuota (`cuota:susc:*` y `cuota:org:*`) se identifica por el `inicio` del ciclo, así que se reinicia sola cuando empieza un ciclo nuevo.

5. Cambio de plan

Qué cuenta como subir de plan: que el plan nuevo tenga un **precio diario** mayor (`precio / vigencia_dias`). Si el precio diario es igual o menor, es una bajada.

Subir de plan (se aplica de inmediato)

Sea `d = días completos que faltan del ciclo = ceil((fin – ahora) / 1 día)`:

```
credito = round(precio_actual × d / vigencia_actual, 2)
si vigencia_nueva == vigencia_actual:
    cargo   = round(precio_nuevo × d / vigencia_nueva, 2)     → el ciclo se mantiene (inicio y fin no
cambian)
si no (por ejemplo, de mensual a anual):
    cargo   = precio_nuevo                                    → empieza un ciclo nuevo hoy
a_pagar = max(cargo – credito, 0)
```

Ejemplo de A2.5: un cambio de Lanzamiento (Q 199) a Producto (Q 599) con 13 días restantes de 30 da un crédito de 199 × 13 / 30 = **Q 86.23**, un cargo de 599 × 13 / 30 = **Q 259.57** y un monto a pagar de **Q 173.34**.

- Si `a_pagar > 0`, se cobra al medio de pago registrado o a una tarjeta nueva. Si el cobro se rechaza, **no cambia nada**.
- El pago queda con `concepto = cambio_plan` y una descripción del tipo "Lanzamiento → Producto · diferencia prorrateada".
- Un cambio **desde un plan gratuito o desde Prueba** crea una suscripción nueva (la anterior queda `finalizada`), cobra el precio completo y empieza el ciclo hoy (A2.2).
- `POST /api/suscripcion/pagar` solo reactiva planes de pago. Prueba (`es_prueba = true`) y los planes de precio cero responden **422** `requiere_plan_de_pago`, sin tokenizar, cobrar ni modificar la suscripción. En gracia o suspensión, B1.4 ofrece **«Contratar un plan de pago»**, que lleva a A2.1; desde allí se contrata por A2.2.
- Después de aplicar una subida o programar una bajada en A2.5, se vuelve a B1.4 con los datos actualizados: el plan nuevo o «A partir del {fin} su plan será {X}».
- Si el plan de pago está en gracia o suspendido, A2.1 ofrece **«Reactivar suscripción»** y lleva a B1.4 para pagarlo antes de cambiar de plan.
- Los errores de plataforma son `suscripcion_existente` (409 al intentar contratar con un plan de pago vigente), `suscripcion_no_encontrada` (404 si no existe una suscripción en el estado que requiere la operación) y `plan_no_encontrado` (404 si el destino no existe, está inactivo o es Prueba).
- Cancelar un cambio programado inexistente devuelve 204 sin registrar una acción nueva en la bitácora.
- La consulta de suscripción conserva los datos y límites completos del plan vigente aunque esté desactivado. A2.5 toma de allí el plan actual, para permitir cambiar desde un plan que ya no aparece en el catálogo público.

Cambiar un plan de API entre gratuito y de pago

El proveedor no puede convertir un plan gratuito en uno de pago, ni al revés, mientras el plan tenga suscripciones que no estén finalizadas: se rechaza con **422** `plan_con_suscripciones`. Si no, quedarían renovaciones de pago sin tarjeta, o tarjetas guardadas que no se cobran. Los demás cambios (precio, vigencia, cuota y límite) sí se permiten; la cuota y el límite se publican de inmediato y el precio aplica desde la siguiente renovación.

Bajar de plan (se programa)

- Se guarda `plan_siguiente_id` y se aplica cuando se cierra el ciclo (CU-16). No se reembolsa nada.
- **Se rechaza** con `excede_limites_del_plan` si la organización tiene más APIs registradas o más miembros de los que permite el plan de destino.
- Si hay una bajada programada, la interfaz muestra "A partir del {fin} su plan será {X}" y permite cancelarla.

6. Límites del plan de plataforma (RF-43)

Límite	Dónde se aplica	Error
`max_apis`	Al registrar una API (se cuentan las APIs en cualquier estado)	`limite_del_plan` (`limite = "apis"`)

Límite	Dónde se aplica	Error
`max_miembros`	Al invitar a un miembro (se cuentan los miembros y las invitaciones vigentes)	`limite_del_plan` (`limite = "miembros"`)
`dominio_propio`	Al conectar un dominio	`plan_sin_dominio_propio`
`cuota_peticiones`	En la compuerta (filtro 6)	429 `cuota_plataforma_agotada`

Avisos de cuota: la API de control lee `cuota:org:*` y muestra un aviso en el panel al llegar al **80 %** y otro al **100 %**. El trabajador envía un correo al propietario la primera vez que se cruza cada umbral en el ciclo.

7. Reversión de pagos (RF-24)

1. Solo se pueden revertir los pagos **autorizados** de la plataforma.
2. Se llama a `ReembolsarAsync`, el pago queda `revertido` con `revertido_en` y `revertido_por`, y se registra en la bitácora.
3. Si ese pago cubría el ciclo vigente (`periodo_inicio ≤ ahora < periodo_fin`), la suscripción pasa a `en_gracia` con `gracia_hasta = ahora + 7 días`.

8. Quién paga a quién

- Los pagos de plataforma son ingresos de Shapi.
- Los pagos de API los cobra Shapi **por cuenta del proveedor**, sin comisión. El proveedor los ve en B1.2 (la facturación de cada consumidor).
- Liquidarle ese dinero al proveedor **queda fuera del alcance** (01 §7).
- Los pagos de API **no** aparecen en A6.3, que muestra solo los de plataforma, y el administrador no los revierte.

B1.2 (RF-36). Se muestra una fila por suscripción no finalizada de la API, incluida la gracia y la suspensión. El consumidor se identifica por su empresa. El consumo suma las llamadas realmente descontadas y consolidadas de producción, de todas las rutas (también las retiradas), desde el inicio del ciclo hasta su último día inclusivo en Guatemala; si el ciclo venció sin renovarse, incluye el consumo hasta hoy, igual que B2.1. Puede ir hasta diez segundos detrás de Redis. La cuota y el nombre corresponden al plan actual; el porcentaje se presenta redondeado al entero más cercano, sin limitarlo a 100 %.

La facturación suma los pagos de API autorizados cuyo inicio y fin de periodo coinciden con el ciclo vigente, incluidos los cambios prorrateados. No se calcula a partir del precio actual del plan y no incluye cobros rechazados ni ciclos anteriores. El estado del último cobro corresponde al último intento de la suscripción (fecha e ID descendentes), aunque fuera rechazado al renovar. Sin pagos se muestran Q 0.00 y «Sin cobros». Los totales abarcan todas las suscripciones mostradas. El pie dice «con suscripción activa» cuando todas están activas; en los demás casos dice «con suscripción vigente». Las filas se ordenan por empresa y luego por ID de suscripción. Solo propietario y editor ven el enlace para invitar; el lector puede consultar la tabla.

9. Modo demostración

Con `SHAPI_MODO_DEMO=true` se habilita el endpoint `POST /api/admin/demo/reloj` (`{ "adelantarDias": 30 }`), que solo puede usar el administrador. Adelanta el `IReloj` de la API y del trabajador y dispara el cierre de ciclo. Así se pueden mostrar en vivo la renovación, la gracia y la suspensión sin esperar 30 días. **Nunca está habilitado cuando `SHAPI_MODO_DEMO=false`.**

El desplazamiento total se guarda en segundos en `demo:reloj:desplazamiento`. El endpoint incrementa la llave atómicamente, aplica el total devuelto al reloj de su proceso y publica **ese total en segundos**, no el incremento en días, en el canal del mismo nombre. Cada proceso mantiene el desplazamiento en memoria: leer `IReloj.Ahora` no hace operaciones de red. Al arrancar se suscribe y lee la llave; cada cinco segundos vuelve a leerla como respaldo y reintenta la suscripción si falló. Los mensajes actualizan la hora en menos de un segundo con Redis disponible, sin volver a consultar la llave. Los duplicados y los mensajes anteriores no suman de nuevo ni hacen retroceder el reloj; una lectura iniciada antes de un mensaje no sobrescribe el desplazamiento nuevo. La lectura periódica también recoge el borrado de la llave al reiniciar la demostración.

La siembra base inicializa el reloj mediante una lectura asíncrona antes de usarlo, sin abrir suscripciones ni iniciar el bucle: tanto el arranque de la API como `sembrar-demo` ejecutan esa siembra antes de arrancar el host. Así, las fechas y la resincronización posterior del comando respetan un desplazamiento ya guardado, incluidas las expiraciones de las claves rotadas.

El trabajador recibe el aviso de cierre **después** de actualizar su reloj en memoria y procesa los cierres en serie. Si hay varios avisos mientras cierra un ciclo, conserva una solicitud pendiente para volver a comprobar la hora más reciente. Un fallo de Redis, incluido un timeout o una cancelación ajena a la detención del servicio, conserva el último desplazamiento conocido (cero hasta la primera sincronización) y registra una advertencia con el tipo de excepción; se reintenta en el siguiente intervalo. Fuera del modo demostración se usa la hora UTC real sin suscripciones ni lecturas del desplazamiento, y no se registra el servicio de sincronización.

10 · Identidad y seguridad

1. Autenticación

Aspecto	Decisión
Mecanismo	**Sesiones del lado del servidor** con una cookie, sin JWT (ADR-06). La cookie guarda un valor aleatorio de 32 bytes, y la tabla `sesion` guarda su SHA-256
Cookie	Personal: `shapi_sesion`, en el host `shapi.localhost`. Consumidor: `portal_sesion`, en el host de cada portal. Las dos son `HttpOnly`, `Secure`, `SameSite=Lax` y `Path=/`, sin `Domain`. La API elige el esquema por la ruta, no por la cookie: `/api/portal/*` solo lee `portal_sesion` y todo lo demás solo `shapi_sesion`, aunque la petición traiga las dos. La política por defecto exige una sesión del ámbito personal (04)
Vencimiento	A las 8 h de inactividad (`Sesion:InactividadHoras`) o a los 7 días desde el inicio de sesión. Cada petición actualiza `ultimo_uso_en`, como máximo una vez por minuto
CSRF	`SameSite=Lax`, más la cabecera `X-Requested-With: shapi` obligatoria en todo método no seguro (los distintos de GET, HEAD, OPTIONS y TRACE), más la comprobación de `Origin`: si llega, debe tener el mismo esquema, host y puerto que la petición. Los frontends llaman a la API **en su mismo host**, así que no hace falta CORS entre el panel y la API
Contraseñas	Tienen entre 10 y 128 caracteres y no pueden ser iguales al correo. Se guardan con `PasswordHasher<T>` de ASP.NET Core Identity (PBKDF2-HMAC-SHA512, 100,000 iteraciones o más). Si el hash guardado se hizo con parámetros más débiles que los actuales, se recalcula al iniciar sesión
Bloqueo	Tras 5 intentos fallidos seguidos, la cuenta se bloquea 15 minutos (`bloqueado_hasta`). Los mensajes de error son genéricos: `401 credenciales_invalidas` si la cuenta no existe o la contraseña no coincide, con el mismo tiempo de respuesta en los dos casos. Durante el bloqueo responde `423 cuenta_bloqueada`. Una cuenta desactivada responde `403 cuenta_desactivada` (CU-02 2b), solo si la contraseña es correcta. Cada intento fallido se cuenta en una sola sentencia de la base, para que los intentos simultáneos no se pierdan. También cuenta una contraseña actual incorrecta al cambiarla en Mi perfil (A8.1), que responde `400 datos_invalidos` con `errores.contrasenaActual`, para que una sesión robada no pueda probar contraseñas sin límite
Recuperación	Siempre se responde lo mismo, exista o no la cuenta, con el mismo trabajo en la base (§8). Se limita a 3 solicitudes por cuenta en una hora; al pasarse responde lo mismo, pero no envía nada. El enlace es de un solo uso y vence a los 60 minutos. Al usarlo **se revocan todas las sesiones** de la cuenta (RF-03), se invalidan los demás enlaces de recuperación pendientes y se reinicia el bloqueo por intentos fallidos
Verificación de correo	El enlace vence a las 24 horas. El reenvío se limita a 3 por cuenta en una hora, sin contar el enlace del registro. Al pasarse responde lo mismo, pero no envía nada, para no revelar si la cuenta existe. Mientras no se confirme, el proveedor no puede **publicar** APIs y el consumidor no puede **contratar** planes (RF-02)
Cuentas de plataforma	Las crea el administrador y reciben un enlace `definir_contrasena` que vence a los 7 días (RF-42)

Aspecto	Decisión
Limitación de peticiones	La API de control limita a 10 peticiones por minuto por IP (`RateLimiter` de ASP.NET Core) los endpoints de `/api/auth/*` que reciben credenciales o tokens: `registro`, `verificar-correo`, `reenviar-verificacion`, `entrar`, `recuperar` y `restablecer`. Con el mismo límite, en el portal del consumidor, `/api/portal/auth/*`: `registro`, `verificar-correo`, `reenviar-verificacion`, `entrar`, `recuperar`, `restablecer` y la consulta y aceptación de `invitacion`. Al pasarse responde `429 demasiadas_peticiones` con `Retry-After`. `GET /api/auth/sesion` y `POST /api/auth/salir` no se limitan, porque no sirven para adivinar credenciales y el panel consulta la sesión en cada carga. La IP del cliente se toma de `X-Forwarded-For`, y el esquema de `X-Forwarded-Proto`, solo si la conexión viene de las redes del borde, que se configuran en `SHAPI_REDES_BORDE` (CIDR separados por comas). Sin esa variable se confía en la máquina, desde donde llega Caddy con Docker Desktop, y en la red `shapi` de `infra/compose.yml`, que tiene la subred fija `172.30.0.0/24` y desde la que llega en Linux. Una red privada cualquiera no basta
Cierre de sesión	`POST /api/auth/salir` no exige una sesión vigente: si la cookie corresponde a una sesión, la revoca, y siempre borra la cookie, aunque la sesión ya haya vencido

Los enlaces enviados por correo llevan a la pantalla del ámbito de la cuenta. Para el personal, el host es el dominio base configurado: la verificación lleva a `https://{dominio_base}/verificar-correo?token={token}` (A1.2) y la recuperación a `https://{dominio_base}/restablecer?token={token}` (A1.4b). Para un consumidor, quien encola el correo agrega `hostPortal` a los datos, con el host del portal de la API, `{sub}.{dominio_base}` (06 §4). Se arma con el subdominio de la API que resolvió `IResolutorPortal`, nunca copiando la cabecera `Host`, y el enlace usa ese host: `https://{hostPortal}/verificar-correo?token={token}` (A5.8) y `https://{hostPortal}/restablecer?token={token}` (A5.10). El dominio propio de una API (RF-12) no sirve, porque apunta a la compuerta y no al portal. Si `hostPortal` no es una sola etiqueta ASCII seguida del dominio base, el correo no se arma y cuenta como un intento fallido. El token se codifica como componente de la URL.

Enrutamiento después de iniciar sesión

El endpoint público `POST /api/auth/definir-contrasena` también aplica el límite de 10 peticiones por minuto por IP. El enlace inicial de una cuenta de plataforma apunta a `https://{dominio_base}/definir-contrasena?token={token}`. Solo acepta el token de tipo `definir_contrasena`, vigente y sin usar, para una cuenta activa de la organización de plataforma sin contraseña definida. Guardar la contraseña verifica el correo e inicia una sesión del personal. La desactivación revoca sus sesiones vigentes; la reactivación no las restaura.

Rol	Destino
administrador	`/admin/organizaciones`
soporte	`/admin/casos`
propietario, editor, lector	`/panel/apis`
consumidor	`/cuenta/suscripcion` o, si no tiene una suscripción vigente (no `finalizada`) a la API del portal, `/planes`. Lo indica `destino` en `GET /api/portal/auth/sesion`

2. Resolución de la organización (multi-tenancy)

- **Personal:** el `organizacion_id` sale de la membresía del usuario de la sesión.

- **Portal:** la API de control identifica la API con el `Host` de la petición (`{sub}.shapi.localhost` → `api.subdominio`), y de ahí saca el `organizacion_id`. Para cada petición, la sesión del consumidor debe pertenecer a esa misma organización y a ese mismo host. Los endpoints del portal lo vuelven a comprobar: si la organización de la sesión no es la de la API del host, responden 404. Un consumidor solo ve y administra lo de la API del portal en el que está (por ejemplo, sus claves de esa API).
- `ShapiDbContext` aplica un **filtro global** (`HasQueryFilter`) por `organizacion_id` a todas las entidades que pertenecen a una organización. Solo los servicios de administración, del trabajador y de la compuerta lo desactivan, y lo hacen de forma explícita (`IgnoreQueryFilters`), con una revisión obligatoria en el código.
  - Pertenecen a una organización las tablas con `organizacion_id` propio (`membresia`, `consumidor`, `api`, `suscripcion_plataforma`, `caso` y `bitacora`), `medio_pago` (de la organización o de uno de sus consumidores) y las que dependen de ellas: `usuario` (por su membresía); `ruta`, `dominio_propio`, `plan_api`, `suscripcion_api` y `consumo_diario` (por la API); `clave` y `pago` (por la suscripción), y `caso_mensaje` (por el caso).
  - No tienen filtro las tablas globales o que se usan antes de conocer la organización: `organizacion`, `plan_plataforma`, `token`, `sesion`, `correo_saliente`, `registro_dns_simulado` y `lote_consolidado`.
  - Sin organización en el contexto, las tablas filtradas no devuelven nada. Las entradas de la bitácora sin organización solo las ve la administración.
- Pedir un recurso de otra organización devuelve **404** (04 §4).

3. Secretos y datos sensibles

Dato	Cómo se guarda	Se muestra
Contraseña	PBKDF2 (Identity v3)	Nunca
Clave de API	SHA-256 en hex + prefijo + últimos 4	Completa **una sola vez**. Después, `shp_prod_••••7c2e`
Token de correo y sesión	SHA-256. El token de un correo va además en claro en `correo_saliente.datos` solo mientras el correo está `pendiente`, y se borra al quedar `enviado` o `fallido`	Nunca (solo viaja en el enlace o en la cookie)
Secreto de origen	Cifrado con **ASP.NET Data Protection** (`IProtectorSecretoOrigen`, propósito `Shapi.SecretoOrigen`). El anillo de llaves persiste en el directorio `SHAPI_DPKEYS_DIR` (el volumen `dpkeys`), compartido por la API y el trabajador con el mismo nombre de aplicación, `Shapi`, para que el trabajador descifre lo que cifró la API al resincronizar. En Redis va en claro, porque Redis solo es accesible desde la red interna	Completo **una sola vez**, al generarlo o regenerarlo
Tarjeta	Solo el token de la pasarela, la marca, los últimos 4, el titular y el vencimiento	La marca y los últimos 4
Credenciales de infraestructura	Variables de entorno en `.env`, que no se versiona. En el repositorio va un `.env.example`	—

4. Protección del origen (SSRF)

Cuando se registra o se edita una API (RF-08), y **también en cada conexión** que abre la compuerta (`ConnectCallback`):

1. Se exige el esquema `http` o `https`, que no haya credenciales en la URL y que el puerto sea cualquiera entre 1 y 65535.
2. Se resuelve el host y se **rechaza** si **alguna** de las direcciones cae en: `127.0.0.0/8`, `::1`, `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`, `169.254.0.0/16`, `fe80::/10`, `fc00::/7`, `100.64.0.0/10`, `0.0.0.0/8`, multidifusión o las redes de Docker.
3. **Excepción del modo demostración:** con `SHAPI_MODO_DEMO=true`, la lista `SHAPI_ORIGENES_PERMITIDOS`, con entradas `host:puerto` (por defecto `localhost:5101,localhost:5102,origen-envios:8080,origen-agro:8080`), permite los orígenes de demostración en desarrollo y en el ambiente productivo simulado.
4. En la compuerta, la conexión se abre **contra la dirección ya validada**, para evitar ataques de *DNS rebinding*.
5. No se siguen redirecciones del origen: la respuesta 3xx se devuelve tal cual al consumidor.

5. Encabezados y protección de los frontends

- La **política de seguridad de contenido** (CSP) del panel y del portal es `default-src 'self'; img-src 'self' data:; style-src 'self' 'unsafe-inline' https://fonts.googleapis.com; font-src 'self' https://fonts.gstatic.com; connect-src 'self' https://*.api.shapi.localhost; frame-ancestors 'none'`.
- Los logotipos SVG se sirven con `Content-Type: image/svg+xml`, `Content-Security-Policy: sandbox` y `X-Content-Type-Options: nosniff`. El portal los muestra **solo** con `<img>`, nunca incrustados en el HTML.
- Todos los textos que escribe el proveedor (nombre, bienvenida y descripciones de la especificación) se muestran escapados. Las descripciones en Markdown de la especificación se convierten a HTML con una lista de etiquetas permitidas.
- Caddy agrega `Strict-Transport-Security`, `X-Content-Type-Options: nosniff` y `Referrer-Policy: strict-origin-when-cross-origin`.

6. Correos (RF-46)

Se envían desde `no-responder@{dominio_base}`. Los correos de un portal usan como remitente visible el nombre de ese portal. Las plantillas están en `Shapi.Infraestructura/Correo/Plantillas`:

Plantilla	Destinatario	Disparador
`verificacion_correo`	Proveedor o consumidor	Registro, o pedir de nuevo el enlace
`recuperacion`	Cualquiera	Solicitud de recuperación
`invitacion_miembro`	Persona invitada	CU-04
`invitacion_consumidor`	Consumidor invitado	CU-23
`definir_contrasena`	Cuenta de plataforma nueva	CU-21
`pago_rechazado`	Propietario o consumidor	Un cobro de renovación rechazado
`suscripcion_en_gracia`	Propietario o consumidor	Entrada en gracia
`suscripcion_suspendida`	Propietario o consumidor	Suspensión

Plantilla	Destinatario	Disparador
`organizacion_suspendida`	Propietario	Suspensión administrativa
`prueba_por_vencer`	Propietario	7 días antes de que termine la Prueba
`aviso_cuota_plataforma`	Propietario	Al cruzar el 80 % y el 100 % de la cuota
`respuesta_caso`	La otra parte del caso	Mensaje nuevo en un caso

Cada plantilla recibe estos datos para su contenido, además de los datos de marca del portal cuando el destinatario es un consumidor:

Plantilla	Datos requeridos
`verificacion_correo`	`nombre`, `token`
`recuperacion`	`nombre`, `token`
`invitacion_miembro`	`nombre`, `nombreOrganizacion`, `token`
`invitacion_consumidor`	`nombre`, `nombreApi`, `enlace`
`definir_contrasena`	`nombre`, `token` (el enlace se construye al renderizar y el token se elimina de la cola después del envío o fallo definitivo)
`pago_rechazado`	`nombre`, `nombrePlan`, `motivo`, `enlace`
`suscripcion_en_gracia`	`nombre`, `nombrePlan`, `fechaSuspension`, `enlace`
`suscripcion_suspendida`	`nombre`, `nombrePlan`, `enlace`
`organizacion_suspendida`	`nombre`, `nombreOrganizacion`, `motivo`, `enlace`
`prueba_por_vencer`	`nombre`, `fechaFin`, `enlace`
`aviso_cuota_plataforma`	`nombre`, `porcentaje`, `enlace`
`respuesta_caso`	`nombre`, `numeroCaso` (por ejemplo, `CAS-104`), `asunto`, `enlace`

Las fechas llegan ya formateadas en español y los enlaces son absolutos. El motor arma por sí mismo el `enlace` de `verificacion_correo` y `recuperacion` a partir del `token` y del ámbito de la cuenta, y el de `invitacion_consumidor` a partir del `token` y `hostPortal`. En los tres casos el token se elimina de los datos de la cola al enviar el correo o agotar los reintentos.

Reintentos: el trabajador revisa `correo_saliente` cada 5 segundos. Si un envío falla, reintenta hasta 5 veces, con esperas de 5 s, 30 s, 2 min, 10 min y 1 h antes de cada reintento. Si falla el quinto reintento (el sexto intento), el correo queda `fallido`, con `intentos = 6` y `ultimo_error`. Un error al armar el correo, por ejemplo un `hostPortal` inválido, también cuenta como intento fallido.

Varios trabajadores: cada correo se toma en su propia transacción con `SELECT … FOR UPDATE SKIP LOCKED`, así que dos trabajadores nunca envían el mismo correo. En cuanto el servidor SMTP acepta el mensaje, el correo se guarda como `enviado`, aunque falle el cierre de la sesión (`QUIT`) o el trabajador se esté deteniendo.

Enlaces de un portal: si los `datos` traen `hostPortal`, debe ser `{sub}.{dominio_base}`, con una sola etiqueta ASCII que no sea un subdominio reservado (06 §4).

Cifrado: con `SHAPI_SMTP_TLS=true`, el cifrado es obligatorio: SMTPS implícito si `SHAPI_SMTP_PUERTO` es 465, y STARTTLS en cualquier otro puerto. Si el servidor no ofrece STARTTLS, el envío falla y nunca sale en texto plano. Con `false` no se cifra, como en Mailpit.

Los correos de los consumidores llevan la marca del portal: el nombre, el color y el logotipo como enlace, y nunca la marca de Shapi en el cuerpo. Los correos del personal (proveedores, administración y soporte) llevan la marca de Shapi (variante 4) y salen con «Shapi» como nombre visible del remitente; los de un portal, con el nombre del portal (`nombrePortal`). Quien encola un correo de consumidor pasa en `datos` `nombrePortal`, `hostPortal`, `colorPortal` (el `api.portal_color`, solo `#RRGGBB`) y, si el portal tiene logotipo, `logoPortal: "true"`; el enlace del logotipo es `https://{hostPortal}/api/portal/logo`.

7. Bitácora

La bitácora (RF-41) registra las acciones que cambian **el acceso, el cobro o el estado** de una organización. Cada entrada guarda el actor, la organización, la acción, el objetivo, una descripción que se puede leer (la que muestra B3.2) y la IP.

B3.2 consulta la bitácora con permiso de administrador o soporte. `desde` y `hasta` son días calendario de `America/Guatemala`, ambos inclusivos; si se omiten, el periodo comprende el día actual y los seis anteriores. Los resultados se ordenan del más reciente al más antiguo y se paginan desde la página 1, con tamaño predeterminado de 20 y máximo de 100; B3.2 muestra un paginador («Página X de Y», «Anterior» y «Siguiente») solo si hay más de una página. Un periodo o una paginación inválidos responden `400` `datos_invalidos`, con el error de cada parámetro en `errores`.

Cada entrada muestra el nombre que se guardó al registrarla y, debajo, el **rol y la organización actuales** del actor, que salen de su membresía al consultar (no se guardan en la entrada):

- personal de plataforma: su rol y «Plataforma Shapi» (así lo muestra B3.2, aunque la organización de plataforma se llame «Shapi»);
- miembro de un proveedor: su rol (`propietario`, `editor` o `lector`) y el nombre de la organización;
- consumidor: `consumidor` y la organización del proveedor dueño del portal;
- sistema: `sistema`, sin organización;
- usuario que ya no tiene membresía: `usuario`, sin organización.

Acción (`accion`)	Descripción de ejemplo
`organizacion.suspendida` / `organizacion.reactivada`	Suspendió la organización Datos Chapines, S.A.
`pago.revertido`	Revirtió el cobro de Q 199.00 de Datos Chapines, S.A.
`plan_plataforma.creado` / `.editado` / `.desactivado`	Editó el plan de plataforma Lanzamiento
`cuenta_plataforma.creada` / `.desactivada` / `.activada`	Creó la cuenta de plataforma de Lucía Ramírez Pineda con el rol de administrador
`suscripcion_plataforma.contratada` / `.cambiada`	Cambió su suscripción de plataforma de Lanzamiento a Producto
`miembro.invitado` / `.rol_cambiado` / `.quitado`	Invitó a karla.batres@enviosxelaju.com con el rol de lector
`api.registrada` / `.publicada` / `.despublicada`	Publicó la API de Cotización de Envíos

Acción (`accion`)	Descripción de ejemplo
`ruta.expuesta` / `ruta.ocultada`	Ocultó la ruta GET /tarifas de la API de Cotización de Envíos
`dominio.conectado` / `.verificado`	Verificó el dominio api.enviosxelaju.localhost
`secreto_origen.regenerado`	Regeneró el secreto de origen de la API de Cotización de Envíos
`plan_api.creado` / `.editado` / `.desactivado`	Editó el plan Básico de la API de Cotización de Envíos
`clave.revocada_por_proveedor`	Revocó la clave de pruebas de Tienda Sololá en la API de Cotización de Envíos
`clave.rotada` / `clave.revocada_por_consumidor`	Boutique Cayalá rotó su clave de producción en la API de Cotización de Envíos
`caso.abierto` / `caso.cerrado`	Abrió el caso CAS-104 de Envíos Xelajú, S.A.
`suscripcion.suspendida` (actor: sistema)	Suspendió por falta de pago la suscripción de Transportes Petén, S.A.

8. Amenazas consideradas

Amenaza	Mitigación
Robo de la base de datos	Las claves, los tokens y las contraseñas solo están como hash; el secreto de origen está cifrado; no hay datos de tarjetas
Uso de una clave filtrada	El consumidor la rota o la revoca, y la revocación se aplica en menos de 10 s
Saltarse la compuerta llamando directo al origen	La URL de origen nunca aparece en el portal. El proveedor puede validar `X-Shapi-Secreto` o limitar las IP que acepta su origen. Es una limitación documentada
SSRF desde la URL de origen	§4
Fuerza bruta en el inicio de sesión	Bloqueo tras 5 intentos y limitación por IP
Enumeración de cuentas	Mensajes genéricos al iniciar sesión y al recuperar la contraseña, con el mismo tiempo de respuesta exista o no la cuenta. Se acepta que el registro (`409 correo_ya_registrado` , CU-01 2a) y el bloqueo (`423 cuenta_bloqueada`) revelen que una cuenta existe: la limitación por IP frena la enumeración masiva
XSS a través de la marca o de la especificación	Escape de textos, sanitización del Markdown, SVG solo como `<img>` y CSP
Acceso a datos de otra organización	Filtro global, 404 y pruebas automatizadas de aislamiento
Abuso de peticiones	Límites por minuto y cuotas en la compuerta, y limitación en `/api/auth/*`

11 · Interfaz y mockups

1. Sistema visual oficial: variante 4, "Plano azul"

La variante 4 es la oficial (ADR-32). Las otras cinco quedaron en `mockups/_descartadas/` solo como historial. Los tokens se implementan como variables CSS y como tema de Tailwind en `frontend/packages/ui`.

Tipografía

Uso	Familia	Pesos
Títulos, cifras destacadas y precios	**Sora**	300 (titular del inicio), 400 y 500
Interfaz y datos	**IBM Plex Sans**	400, 500 y 600. Cifras tabulares (`font-variant-numeric: tabular-nums`) en toda tabla

Escala: título de página 32/1.2 (Sora 400) · título de tarjeta 22/1.3 · cuerpo 16/1.6 · dato de tabla 15 · etiqueta 12, en mayúsculas, con `letter-spacing .16em`, color `--tinta-suave` · encabezado de tabla 11, en mayúsculas, con `letter-spacing .12em`. En Tailwind son `text-titulo`, `text-titulo-tarjeta`, `text-cuerpo`, `text-dato`, `text-etiqueta` y `text-encabezado`.

Colores

Token	Superficie clara (aplicación y portal)	Superficie oscura (inicio y barra lateral)
`--principal`	`#3B6FF0` (cursor encima: `#2C57C9`)	`#7FA6FF`
`--tinta`	`#0B1220`	`#E8EDF7`
`--tinta-suave`	`#5A6884`	`#8B98B0`
`--fondo`	`#F4F6FA`	`#060910`
`--panel`	`#FFFFFF`	`#0C1220`
`--borde`	`#DCE3EE` (campos: `#C9D2E1`, filas: `#EBEFF5`)	`#1B2436` (`#2A3550` en los controles)
`--correcto`	base `#1F8A5B` · etiqueta: texto `#146542`, fondo `#E4F3EC`, borde `#B6DCC9`	`#3FBF88`
`--alerta`	base `#C2481F` · etiqueta: texto `#8E3315`, fondo `#FBE9E3`, borde `#EDC3B4`	`#F08A5F`
`--neutro`	texto `#5A6884` · fondo `#F4F6FA` · borde `#DCE3EE`	—
`--tinta-inactiva`	`#A3AEC2` (texto del botón deshabilitado y texto guía de los campos)	—
`--borde-inactivo`	`#E4E9F1` (borde del botón deshabilitado)	—
`--anillo-foco`	`#DEE7FC` (anillo de 3 px del campo enfocado)	—

Token	Superficie clara (aplicación y portal)	Superficie oscura (inicio y barra lateral)
`--borde-barra`	—	`#131B2B` (separadores de la barra superior y la lateral)
`--tinta-rotulo`	—	`#7F8DA8` (rótulos de grupo de la barra lateral)
`--tinta-navegacion`	—	`#B9C4D8` (enlaces de la barra lateral y botón de salir)
`--fondo-activo`	—	`#0E1830` (enlace activo de la barra lateral)

En la superficie oscura solo cambian los tokens que tienen valor en su columna; los demás conservan el de la superficie clara. Correcto y alerta tienen allí un solo tono, que vale para la base y para la etiqueta. En Tailwind, los colores se declaran con `@theme inline`, para que la clase `.dark` los cambie. Se precisó el 27 de septiembre (auditoría, H-86). Los cuatro últimos tokens solo existen en la superficie oscura y salen de las barras de N.1, A6 y B3 (H-98).

Cada color de estado tiene dos tonos, como en la lámina y los mockups:

- **Base** (`--correcto-base` y `--alerta-base`): iconos de estado, muestras de la paleta, códigos HTTP (por ejemplo, el "200" de la consola) y texto de estado dentro de una tabla ("Expuesta").
- **Etiqueta** (`--correcto` y `--alerta`, con su fondo y su borde): el texto sobre fondo de color, en las etiquetas `eti-c` y `eti-a` y en los avisos.

Se precisó el 26 de septiembre: la tabla solo tenía el tono de etiqueta, mientras que la lámina y unos 20 mockups usan también el tono base.

Solo hay tres colores de estado: correcto (activa, pagado, en servicio), alerta (suspendida, rechazado, en gracia, degradado) y neutro (despublicada, pendiente, cerrado).

Componentes base

- **Radio único:** 8 px. **Espaciado:** 4 · 8 · 12 · 16 · 20 · 24 · 32 · 44 · 64 · 80.
- **Botón principal:** 46 px de alto, texto de 15 px con peso 500, fondo `--principal`. **Botón secundario:** borde `#C9D2E1`. **Botón deshabilitado:** borde `#E4E9F1` y texto `#A3AEC2`.
- **Contraseña:** los diez campos de contraseña de A1.1, A1.3, A1.4b, A8.1 (actual y nueva), A8.2, A5.3, A5.3b, A5.7 y A5.10 usan `CampoContrasena`, con un ojo a la derecha. El ojo abierto muestra el valor y el tachado lo oculta; el botón alterna «Mostrar contraseña» / «Ocultar contraseña» y `aria-pressed`, funciona con teclado y no envía el formulario.
- **Tarjeta:** A2.2, A2.5, B1.4 y A5.6 vacían todos los datos de tarjeta (número, vencimiento, CVV y titular) cuando el pago resulta autorizado o rechazado. Un error de validación, de red o de servicio los conserva para corregir y reintentar.
- **Cursor:** las acciones habilitadas (botones, enlaces, selectores, casillas y opciones de navegación) muestran la mano; las deshabilitadas muestran `not-allowed`. La regla es común en `packages/ui/src/style.css`.
- **Campo:** 48 px de alto, texto de 15 px, relleno horizontal de 14 px y borde `#C9D2E1`. Enfocado: borde `--principal` y anillo de 3 px `--anillo-foco`. Texto guía en `--tinta-inactiva`. La etiqueta va encima, en 13 px seminegra.
- **Etiqueta de estado:** clases `eti-c` (correcto), `eti-a` (alerta) y `eti-n` (neutro).

- **Tabla:** filas de 42 px, encabezado de 11 px en mayúsculas, seminegro, con 10 px debajo y borde `#DCE3EE`, y separador `#EBEFF5` en todas las filas. Las celdas tienen 16 px de relleno a la derecha y ninguno a la izquierda.
- **Tarjeta:** fondo blanco, borde de 1 px `#DCE3EE`, radio de 8 px y relleno de 20 px, **sin sombra**.
- **Estructura del panel:** barra superior oscura de 76 px, barra lateral oscura de 272 px con grupos *Publicación · API · Organización* y contenido con un margen de 44 px. Se diseña para 1440 × 900 y funciona desde 1280 px. Desde un área visible de 1366 × 650 px (pantalla de 1366 × 768 con Chrome al 100 %), todas las opciones y el pie de cada barra lateral quedan visibles sin desplazamiento propio. Se compactan filas y grupos sin quitar opciones ni cambiar su orden; este espaciado se aparta de N.1 por decisión de Jordin del 9 de octubre de 2026 (JG-20).
- **Portal:** usa la misma estructura clara, pero el color principal es `--marca-principal`, el de la API. En el portal no aparece la marca de Shapi.

2. Formato de los mockups

- **Fuente editable:** `mockups/<Tanda>/*.dc.html` (HTML estático de 1440 × 900) y `mockups/<Tanda>/canvas.json`, que tiene el orden y los títulos de las mesas de trabajo.
- **Para verlos:** los `.html` de la raíz de `mockups/` (por ejemplo `a3-publicacion-api.html`) son **paquetes autocontenidos**; se abren con doble clic en el navegador.
- **Después de editar** cualquier `.dc.html` o `canvas.json`, hay que volver a generar los paquetes:

```
python mockups/herramientas/empaquetar.py
```

- Los mockups muestran **datos de ejemplo** con las mismas cifras que la siembra de demostración (07 §6). Los frontends tienen que verse igual a los mockups con esos datos.

3. Catálogo de pantallas

Estado: **=** sin cambios · ✎ corregida el 22 de septiembre de 2026 · ★ nueva.

> Todas las pantallas del panel del proveedor (A2, A3, A4, B1 y N.1) llevan además el nuevo elemento **Casos de soporte** en la barra lateral. No se marca en cada fila.

A0 · Sitio público (`mockups/A0`, paquete `a0-inicio.html`)

ID	Pantalla	Archivo	Ruta	RF	Estado
A0.1	Inicio de Shapi con planes	`Main.dc.html`	`shapi.localhost/`	RF-19	✎ (peticiones, miembros, Escala mensual/anual, texto de las claves)
A0.2	Lámina de estilo oficial	`Lamina.dc.html`	`shapi.localhost/_ui` (solo en desarrollo)	—	✎ (subdominio de ejemplo)

> A0.2 lleva el host completo a propósito: la prueba del catálogo (`frontend/apps/panel/src/tests/rutas.test.tsx`) exige que cada ruta que empieza con `/` muestre su ID, y la lámina no lo muestra. La ruta `/_ui` se prueba aparte.

A1 · Acceso del proveedor (`a1-acceso-proveedor.html`)

ID	Pantalla	Archivo	Ruta	RF	Estado
A1.1	Registro del proveedor	`Main.dc.html`	`/registro`	RF-01	=
A1.2	Aviso de verificación de correo	`Verificacion.dc.html`	`/verificar-correo`	RF-02	✎ (peticiones, vence en 24 h, reenviar)
A1.3	Inicio de sesión	`Sesion.dc.html`	`/entrar`	RF-04	=
A1.4a	Recuperación: pedir el enlace	`Recuperacion.dc.html`	`/recuperar`	RF-03	=
A1.4b	Recuperación: contraseña nueva	`NuevaContrasena.dc.html`	`/restablecer?token=`	RF-03	✎ (se cierran las demás sesiones; el correo no se muestra — ver §4, «Comportamiento de la recuperación y de Mi perfil (EM-04)»)

A2 · Plan de plataforma (`a2-contratacion-plan.html`)

ID	Pantalla	Archivo	Ruta	RF	Estado
A2.1	Planes de plataforma	`Main.dc.html`	`/panel/suscripcion/planes`	RF-19	✎ (peticiones, miembros, Escala mensual/anual)
A2.2	Contratar un plan superior	`Contratacion.dc.html`	`/panel/suscripcion/contratar/:plan`	RF-20	✎ (fechas de 30 días, texto del token)
A2.3	Confirmación	`Confirmacion.dc.html`	—	RF-20	✎ (fechas)
A2.4	Tarjeta rechazada	`Rechazo.dc.html`	—	RF-20	=
A2.5	Cambio de plan con prorrateo	`CambioPlan.dc.html`	`/panel/suscripcion/cambiar/:plan`	RF-25	✎ (fechas)

A3 · Publicación de una API (`a3-publicacion-api.html`)

ID	Pantalla	Archivo	Ruta	RF	Estado
A3.1	APIs de la organización (y su variante vacía)	`Main.dc.html`, `ListaVacia.dc.html`	`/panel/apis`	RF-14, RF-43	✎ (uso del límite del plan)

ID	Pantalla	Archivo	Ruta	RF	Estado
A3.2	Registrar una API	`Registro.dc.html`	`/panel/apis/nueva`	RF-08	✎ (subdominio y prueba de conexión)
A3.3	Cargar especificación	`Especificacion.dc.html`	`/panel/apis/:id/especificacion`	RF-09	✎ (las rutas nuevas quedan ocultas)
A3.4	Rutas expuestas	`Rutas.dc.html`	`/panel/apis/:id/rutas`	RF-10	=
A3.5	Configuración por ruta	`ConfigRutas.dc.html`	`/panel/apis/:id/configuracion-rutas`	RF-13	✎ (peso en llamadas)
A3.6	Direcciones, dominio propio y secreto de origen	`Dominio.dc.html`	`/panel/apis/:id/dominios`	RF-11, RF-12, RF-47	✎ (hosts `.localhost`, DNS simulado, secreto)
A3.7	Personalización del portal	`Portal.dc.html`	`/panel/apis/:id/portal`	RF-15	✎ (host del portal, peticiones por minuto)
A3.7r	Referencia: inicio del portal a tamaño real	`InicioPortal.dc.html`	—	RF-16	✎ (peticiones por minuto)

A4 · Planes, claves y equipo (`a4-planes-claves-equipo.html`)

ID	Pantalla	Archivo	Ruta	RF	Estado
A4.1	Planes de la API (vacía, editar, nuevo)	`Main`, `PlanesVacia`, `EditarPlan`, `NuevoPlan`	`/panel/apis/:id/planes`	RF-18	✎ (límite en peticiones)
A4.2	Miembros y roles (vacía, quitar)	`Miembros`, `MiembrosVacia`, `QuitarMiembro`	`/panel/miembros`	RF-06, RF-43	✎ (uso del límite de miembros)
A4.3	Claves de los consumidores (vacía)	`Claves`, `ClavesVacia`	`/panel/apis/:id/claves`	RF-26, RF-28	✎ (**solo revocar**; ya no hay "Rotar")
A4.3b	Revocar una clave	`RevocarClave.dc.html`	—	RF-28	=
~~A4.3r~~	~~Rotar una clave (el proveedor)~~	eliminada → `_descartadas/A4-RotarClave.dc.html`	—	—	Reemplazada por B2.4

A5 · Portal de marca blanca (`a5-portal-marca-blanca.html`)

ID	Pantalla	Archivo	Ruta (`{sub}.shapi.localhost`)	RF	Estado
A5.0	Inicio del portal	`Main.dc.html`	`/`	RF-15, RF-16	✎ (peticiones por minuto)
A5.1	Documentación	`Documentacion.dc.html`	`/documentacion` y `/documentacion/:ruta`	RF-16	✎ (URL de la API en `.api.`)

ID	Pantalla	Archivo	Ruta (`{sub}.shapi.localhost`)	RF	Estado
A5.2	Consola de pruebas	`Consola.dc.html`	`/consola`	RF-16, RF-45	✎ (la clave de pruebas se pega)
A5.3	Registro del consumidor	`Registro.dc.html`	`/registro`	RF-05	=
A5.3b	Registro por invitación	`RegistroInvitacion.dc.html`	`/invitacion?token=`	RF-05	=
A5.4	Planes de la API (vacía)	`Planes` , `PlanesVacia`	`/planes`	RF-19	✎ (peticiones por minuto)
A5.4b	Plan contratado y claves (única vez)	`Confirmacion.dc.html`	—	RF-20, RF-26	✎ (aviso de que no se volverán a mostrar)
A5.5	El mismo portal con otra marca (inicio y documentación)	`InicioAgro` , `DocumentacionAgro`	`agro.shapi.localhost`	RF-15	✎ (URL y peticiones)
A5.6	Pago en el portal	`Pago.dc.html`	`/contratar/:plan`	RF-20	✎ (texto del token)
A5.7	Inicio de sesión del consumidor	`Acceso.dc.html`	`/entrar`	RF-04	✎ (enlace para recuperar la contraseña)
A5.8	Verificación de correo del consumidor	`Verificacion.dc.html`	`/verificar-correo`	RF-02	★
A5.9	Recuperación: pedir el enlace	`Recuperacion.dc.html`	`/recuperar`	RF-03	★
A5.10	Recuperación: contraseña nueva	`NuevaContrasena.dc.html`	`/restablecer?token=`	RF-03	★ (correo no visible — ver §4, «Comportamiento de la recuperación y de Mi perfil (EM-04)» y «Comportamiento del portal y de sus pantallas de acceso (DC-03 y DC-08)»)

A6 · Administración y soporte (`a6-administracion-soporte.html`)

ID	Pantalla	Archivo	Ruta	RF	Estado
A6.1	Planes de plataforma (nuevo y editar)	`Main` , `NuevoPlan` , `EditarPlan`	`/admin/planes`	RF-17	✎ (peticiones, miembros, Escala anual)
A6.2	Organizaciones (vacía)	`Organizaciones` , `OrganizacionesVacia`	`/admin/organizaciones`	RF-38	✎ (propietario en vez de subdominio; fechas)
A6.2b	Suspender una organización	`Suspender.dc.html`	—	RF-38	✎ (sin subdominio; antes de confirmar se captura el motivo administrativo requerido por RF-38)

ID	Pantalla	Archivo	Ruta	RF	Estado
A6.3	Pagos de plataforma (vacía)	`Pagos`, `PagosVacia`	`/admin/pagos`	RF-24	=
A6.3b	Revertir un pago	`RevertirPago.dc.html`	—	RF-24	=
A6.4	Casos de soporte (vacía)	`Casos`, `CasosVacia`	`/admin/casos`	RF-40	✎ (casos abiertos por el proveedor)
A6.4b	Atención de un caso	`Caso.dc.html`	`/admin/casos/:numero`	RF-40	✎ (conversación, API afectada, fechas)
A6.5	Cuentas de administración y soporte (vacía)	`Cuentas`, `CuentasVacia`	`/admin/cuentas`	RF-42	=

A7 · Soporte del proveedor ★ (`a7-soporte-proveedor.html`)

ID	Pantalla	Archivo	Ruta	RF	Estado
A7.1	Casos de soporte de la organización y formulario para abrir uno	`Casos.dc.html`	`/panel/soporte`	RF-40	★
A7.2	Conversación de un caso	`Caso.dc.html`	`/panel/soporte/:numero`	RF-40	★

A8 · Cuenta del personal ★ (`a8-cuenta.html`)

ID	Pantalla	Archivo	Ruta	RF	Estado
A8.1	Mi perfil (nombre, contraseña, cerrar sesión)	`Perfil.dc.html`	`/panel/perfil` y `/admin/perfil`	RF-04	★
A8.2	Aceptar la invitación de un miembro	`Invitacion.dc.html`	`/invitacion?token=`	RF-06	★

B1 · Salidas del proveedor (`b1-salidas-proveedor.html`)

ID	Pantalla	Archivo	Ruta	RF	Estado
B1.1	Consumo, latencia y errores (vacía)	`Main`, `ConsumoVacia`	`/panel/apis/:id/consumo`	RF-35	✎ (peticiones y llamadas, bytes)
B1.2	Consumo y facturación por consumidor (vacía)	`Consumidores`, `ConsumidoresVacia`	`/panel/apis/:id/consumidores`	RF-36	✎ (fechas y botón para invitar)
B1.3	Historial de pagos (vacía)	`Pagos`, `PagosVacia`	`/panel/pagos`	RF-21	✎ (fechas)
B1.4	Suscripción de plataforma: vigente, en gracia y suspendida	`Suscripcion*`	`/panel/suscripcion`	RF-22, RF-23	✎ (fechas de 30 días y 7 días de gracia)

ID	Pantalla	Archivo	Ruta	RF	Estado
B1.5	Invitar consumidores	`InvitarConsumidores.dc.html`	`/panel/apis/:id/consumidores/invitar`	RF-05	★

B2 · Salidas del consumidor (`b2-salidas-consumidor.html`)

ID	Pantalla	Archivo	Ruta (`{sub}.shapi.localhost`)	RF	Estado
B2.1	Consumo del ciclo por ruta (vacía)	`Main`, `ConsumoVacia`	`/cuenta/consumo`	RF-37	=
B2.2	Historial de pagos (vacía)	`Pagos`, `PagosVacia`	`/cuenta/pagos`	RF-21	=
B2.3	Suscripción y claves: vigente, en gracia, no disponible y vacía	`Suscripcion*`	`/cuenta/suscripcion`	RF-22, RF-23, RF-26 a RF-28	✎ (claves enmascaradas; **Rotar** y **Revocar**; cambiar de plan)
B2.4	Rotar una clave	`RotarClave.dc.html`	—	RF-27	★
B2.5	Clave nueva (única vez)	`ClaveNueva.dc.html`	—	RF-27	★
B2.6	Revocar una clave propia	`RevocarClave.dc.html`	—	RF-28	★
B2.7	Cambio de plan del consumidor	`CambioPlan.dc.html`	`/cuenta/cambiar-plan/:plan`	RF-25	★

Precisión de B2.3: en `SuscripcionGracia.dc.html` el rango visual termina el 8 de octubre, igual que la fecha de suspensión. La implementación sigue la convención temporal de 12 §ADR-30: muestra como periodo de gracia los días con servicio (1–7 oct) y la suspensión aparte (8 oct). Los días restantes los calcula el servidor con `IReloj`, no el navegador, para respetar el reloj de demostración.

B3 · Salidas del administrador y del soporte (`b3-salidas-administracion-soporte.html`)

ID	Pantalla	Archivo	Ruta	RF	Estado
B3.1	Estado de los componentes: normal, degradado y vista de soporte	`Main`, `Degradado`, `Soporte`	`/admin/estado`	RF-39	✎ (los 7 componentes de la arquitectura)
B3.2	Bitácora (vacía)	`Bitacora`, `BitacoraVacia`	`/admin/bitacora`	RF-41	✎ (acción de un consumidor)

Navegación

ID	Pantalla	Archivo	Estado
N.1	Barra lateral del proveedor, aplicada a A4.1	`Navegacion/Main.dc.html`	✎ (se agregó "Casos de soporte")

Barra lateral del proveedor:

- *Publicación*: APIs.
- *API* (con selector): Especificación · Rutas expuestas · Configuración por ruta · Dominios · Portal · Planes · Claves · Consumo · Consumidores.
- *Organización*: Miembros y roles · Suscripción de plataforma · Historial de pagos · **Casos de soporte**.
- Al pie: el nombre y el rol del usuario (lleva a **Mi perfil**) y el botón para cerrar sesión.

4. Estados que toda pantalla debe cubrir

Estos estados no tienen mockup propio. Se implementan con los componentes base:

- **Cargando:** esqueleto con la forma de la tabla o la tarjeta.
- **Error de red o del servidor:** un aviso con los colores de `eti-a` y el botón secundario "Reintentar". El aviso no copia de la etiqueta las mayúsculas ni el texto en una sola línea.
- **Error de validación:** el mensaje va debajo del campo, en `--alerta`, y el campo lleva un borde de ese color.
- **Sin permiso:** el menú oculta la opción. Si se entra directo por la URL, se muestra "No tiene permiso para ver esta página".
- **Acción confirmada:** un aviso breve (4 s) en la esquina superior derecha.

Cuentas de plataforma (JZ-09)

- A6.5 permite crear cuentas de administración y soporte, desactivarlas y activarlas. Los selectores de las cuentas existentes muestran el rol en solo lectura: RF-42 no incluye cambiar roles. La cuenta propia se identifica con «(usted)» y no ofrece desactivación.
- La variante vacía conserva la cuenta inicial y se muestra cuando no hay otras cuentas.
- `/definir-contrasena?token=` es pública y usa el diseño de A1.4b. Su rótulo es «Cuenta de plataforma», el enlace dura **7 días**, es de un solo uso e inicia sesión al guardar. Si falta, venció o ya se usó, muestra «El enlace ya no sirve» y «El enlace venció o ya se usó. Comuníquese con el administrador de Shapi.».
- Después de definir la contraseña se consulta la sesión y se navega al destino del rol (10 §1). Si solo falla esa consulta, «Reintentar» no vuelve a consumir el enlace.

Comportamiento de la estructura de navegación (DC-02)

- La API seleccionada se representa en `/panel/apis/:id/...`. Al elegir desde una página sin ID, se abre su especificación; al cambiar de API desde una sección, se conserva esa sección. Sin selección, se indica que debe elegir una API y no se crean enlaces a un ID ficticio.

- Mientras no exista el listado de DC-04, una respuesta 404 o 501 se presenta como «Sin APIs». Los demás errores muestran el aviso recuperable con «Reintentar».
- Una sesión ausente (401) redirige a `/entrar`; un error de red o servidor mantiene la dirección y ofrece reintentar. Al cerrar sesión se espera la revocación del servidor antes de salir y limpiar los datos privados del cliente; si falla, se conserva la pantalla con un aviso y reintento.
- El encabezado del proveedor muestra el nombre de su organización, que `GET /api/auth/sesion` devuelve en `organizacion.nombre`.
- `/panel` y `/admin` llevan al destino del rol (10 §1), y `/panel/apis/:id` lleva a la especificación de esa API. A0.1 (`/`) tiene su propio encabezado; el de A1 solo va en las pantallas de acceso.
- Quien entra al área de otro rol (un proveedor a `/admin`, o el personal de Shapi a `/panel`) ve "No tiene permiso para ver esta página" sin barra lateral, con "Ir a su panel" (su destino) y "Cerrar sesión". Dentro de su área, el 403 y el 404 conservan la barra lateral.
- Un error que no es 404 (por ejemplo, una página que no se pudo descargar) muestra el aviso de error con "Reintentar", que vuelve a cargar la página.
- La sesión consultada se considera vigente 5 minutos. Entrar la vuelve a pedir y salir borra la caché.
- La estructura ocupa la altura de la ventana: solo se desplaza el contenido, con su barra a la derecha, y la barra lateral completa y su pie quedan fijos, sin desplazamiento propio, en panel, administración, soporte y portal. Desde 1366 × 650 px visibles caben todas las opciones y el pie, incluso con una API seleccionada. En documentación caben las rutas de la siembra; si hay más rutas de las que caben, solo su lista interna puede desplazarse, conservando visibles el resto de enlaces y el pie.

Comportamiento de las pantallas de acceso (EM-03)

- **A1.1 y A1.3:**
  - en A1.1, los errores de validación de la API (`errores` por campo) se muestran debajo de cada campo;
  - `correo_ya_registrado` se muestra debajo del correo, con los enlaces «Entrar» (`/entrar`) y «Recuperar la contraseña» (`/recuperar`) (CU-01 2a);
  - los demás errores con `codigo` (credenciales incorrectas, cuenta bloqueada o desactivada, demasiadas peticiones) se muestran en un aviso de alerta arriba del formulario, con el mensaje de la API;
  - si falla la red o el servidor (un 5xx, aunque venga como ProblemDetails sin `codigo` del contrato), el aviso dice «No se pudo completar la solicitud. Revise su conexión e intente de nuevo.» y ofrece «Reintentar», que vuelve a enviar el formulario. Si la sesión no quedó iniciada después de entrar, se avisa lo mismo.
- **A1.2 sin `token`:** es el aviso «Revise su correo» del mockup, con el correo de `?correo=`. «Enviar el enlace otra vez» pide un enlace nuevo y confirma con «Si su correo todavía no está confirmado, le llegará un enlace nuevo en unos minutos.», un mensaje que no revela si la cuenta existe (10 §1).
- **A1.2 con `?token=`**, el destino del enlace del correo (10 §1):
  - muestra «Confirmando su correo» mientras llama a la API, una sola vez por token;
  - si la verificación funciona, consulta la sesión y lleva al destino según el rol;
  - si el enlace venció o ya se usó (`token_invalido`), muestra «Enlace no válido» y un campo de correo para pedir otro, porque el enlace no trae la dirección;
  - cualquier otro error muestra «No se pudo confirmar su correo» con el mensaje. Si es de red o del servidor, ofrece «Reintentar», y si el correo ya se confirmó y solo falló la consulta de la sesión, el

reintento no vuelve a enviar el token.

Comportamiento de la recuperación y de Mi perfil (EM-04)

- **A1.4a, después de enviar:** muestra «Revise su correo», con el rótulo «Recuperación · revise su correo» y el mensaje «Si el correo existe, le enviamos un enlace para definir una contraseña nueva. El enlace es de un solo uso y vence en 60 minutos.». Mantiene la respuesta neutral de CU-03, paso 2; «paso 2 de 2» corresponde a A1.4b.
- **A1.4b y A5.10, sin el correo:** el enlace del correo es `/restablecer?token={token}` (10 §1) y no incluye el correo. Por eso, A1.4b **no muestra el correo** de la cuenta y usa el texto fijo «Defina una contraseña nueva para su cuenta.» en lugar del texto del mockup («Está definiendo la contraseña de {correo}.»). Mostrar el correo enmascarado exigiría un endpoint que lo devuelva a partir del token sin consumirlo, que no está en el alcance.
- **A1.4b con el enlace vencido o usado:** muestra el rótulo «Recuperación · enlace no válido», el título «El enlace ya no sirve» y «Solicitar un enlace nuevo», que lleva a `/recuperar`. Si la contraseña ya se restableció pero falla la consulta de la sesión, muestra el error y permite reintentar esa consulta sin volver a enviar el token consumido.
- **A8.1, Mi perfil:**
  - los errores por campo van debajo de cada campo, también «La contraseña actual no es correcta.» (`errores.contrasenaActual`, 10 §1);
  - la confirmación es el aviso breve de 4 s: «Nombre actualizado.» y «Contraseña actualizada. Se cerraron sus sesiones en otros equipos.»;
  - los demás errores van en un aviso arriba del formulario; si son de red o del servidor, con «Reintentar», como en A1.1.

Comportamiento de la publicación de una API (DC-04 y DC-05)

- **A3.2, origen inaccesible o no permitido:** el aviso muestra el título del error y el motivo de la prueba de conexión (`detalle.motivo`).
- **A3.3, al volver a la pantalla:** si la API ya tiene especificación, se muestra la tarjeta del archivo con «Cargado». Como el nombre del archivo no se guarda, en su lugar va el título de la especificación, y debajo «OpenAPI {versión} · {tamaño}». Justo después de cargar un archivo se muestra su nombre, como en el mockup.
- **A3.3, especificación inválida:** el aviso muestra el título, la ubicación y el motivo en español. Si el error lo dio la biblioteca que lee el documento, se agrega «Detalle técnico: {mensaje original}», que está en inglés.
- **A3.4 y A3.3 para el lector:** el lector puede ver las rutas (04 §3.1), pero guardar cambios responde 403.

Comportamiento de las claves del proveedor (JG-10)

- **A4.3:** conserva el orden del contrato de claves: consumidores por fecha de contratación y, dentro de cada consumidor, producción antes de pruebas, con los estados activa, rotada y revocada. Se pagina por consumidor, en páginas de 20; «Anterior» y «Siguiente» aparecen cuando hay más de una página.
- Las fechas de «Vigente hasta» y «El» se muestran en Guatemala, con el formato del mockup (`11 sep 2026, 09:15`). El lector no ve la columna «Acciones». Las claves revocadas no tienen acción.

- **A4.3b:** es una confirmación dentro de la ruta de A4.3. «Cancelar» vuelve a la lista sin enviar la revocación. Mientras se revoca, no se puede confirmar otra vez ni cancelar; los errores conservan la confirmación y permiten reintentar.
- Al revocar, se actualiza la lista y aparece «Clave revocada.» durante 4 segundos. Cambiar de API cierra la confirmación, vuelve a la primera página y consulta las claves de la nueva API.

Comportamiento del portal y de sus pantallas de acceso (DC-03 y DC-08)

- **Marca:** la raíz del portal redefine `--principal`, `--principal-hover` y `--anillo-foco` con `--marca-principal`, así que los botones, los campos y el anillo de foco de todas las pantallas usan la marca y nunca el azul de Shapi (§1).
- **Pie de las páginas públicas:** muestra la insignia de 24 px (radio de 6 px) con las iniciales de la organización, su nombre (`nombreOrganizacion` de la configuración) y los enlaces «Documentación», «Planes» y «Entrar».
- **Enlaces de «Documentación»:** llevan a `/documentacion`. Hasta DC-07, esa ruta muestra A5.1; DC-07 la redirige a la primera ruta expuesta.
- **Barra del portal:** A5.1, A5.4, A5.6 y A5.4b muestran la barra lateral con las secciones «API» y «Mi cuenta» de sus mockups; A5.1 identifica además cada ruta por método y patrón. Estas páginas siguen siendo públicas: sin sesión muestran los enlaces de cuenta sin datos del consumidor; si existe una sesión, muestran al pie el nombre y la empresa del consumidor y la acción de cerrar sesión, igual que el área autenticada.
- **Barra de la cuenta:** el enlace activo tiene un fondo teñido con la marca (`color-mix(in srgb, var(--marca-principal) 8%, #FFFFFF)`) y el texto en la marca; los demás enlaces van en `#2B3547`. `/cuenta` lleva a `/cuenta/suscripcion`.
- **Errores:** un error de render muestra el aviso de error con «Reintentar» en español; una ruta que no existe, «Página no encontrada». Si la configuración o la sesión responden 5xx, se muestra el aviso con «Reintentar», no «API no disponible» ni la pantalla de acceso.
- **Cerrar sesión:** borra de la caché los datos del consumidor (solo queda la configuración del portal) y, si falla, avisa «No se pudo cerrar la sesión.» con «Reintentar», sobre el contenido junto al pie, sin aumentar la altura de la barra lateral ni ocultar sus opciones. El panel usa la misma disposición del error (JG-20). También se borran si la sesión venció (al ir a `/entrar`) y cada vez que un consumidor entra, para que nunca vea en la caché datos de otro consumidor del mismo navegador.
- **Encabezado de las pantallas de acceso:** solo la marca del portal, sin la navegación pública ni el pie.
- **A5.3b:** si el enlace venció o ya se usó, muestra el rótulo «Invitación», el título «El enlace ya no sirve» y «La invitación venció, ya se usó o no pertenece a este portal.». Si el correo ya tiene una cuenta en el portal (`correo_ya_registrado`), muestra el error con los enlaces «Entrar» y «Recuperar la contraseña» (CU-11 2a).
- **A5.7:** «Contraseña» es el rótulo del campo, así que hacer clic en él enfoca el campo.
- **A5.8 con el enlace inválido:** muestra el estado de enlace vencido y permite pedir uno nuevo con el correo.
- **A5.9, después de enviar:** el rótulo «Recuperación · revise su correo», el título «Revise su correo» y, una sola vez, la respuesta neutral «Si el correo tiene una cuenta en este portal, recibirá el enlace.», en el recuadro con el vencimiento de 60 minutos.
- **A5.10:** «Defina una contraseña nueva para su cuenta en el portal de {nombrePortal}.» (sin el correo, como A1.4b). Con el enlace vencido o usado, muestra «El enlace ya no sirve» y «Solicitar un enlace

nuevo».

12 · Registro de decisiones de arquitectura (ADR)

Estas decisiones salieron de la auditoría del 22 de septiembre de 2026, que encontró contradicciones entre la propuesta, el documento de requisitos y diseño y los mockups. El coordinador las aprobó el mismo día. El número de cada ADR coincide con el de la decisión (D1 a D34) de esa revisión. ADR-35 se agregó después, con las versiones del stack.

Formato: **Contexto** → **Decisión** → **Alternativas descartadas** → **Consecuencias**.

ADR-01 · Las claves de API se guardan con hash y se muestran una sola vez

- **Contexto:** el RNF-07 decía "cifradas", el MER decía "hash", una secuencia decía "se muestra una vez" y los mockups las mostraban siempre.
- **Decisión:** se guarda el **SHA-256** de la clave, su prefijo y sus últimos 4 caracteres. La clave completa se muestra solo al emitirla o rotarla. En la consola, el consumidor pega su clave de pruebas.
- **Alternativas descartadas:** el cifrado reversible, porque obliga a guardar además un hash para buscar la clave y a administrar llaves de cifrado, y es menos seguro. Tampoco se usa bcrypt o PBKDF2, porque la clave ya tiene unos 154 bits aleatorios y un hash lento castigaría la latencia de la compuerta.
- **Consecuencias:** si el consumidor pierde una clave, la **rota**. Cambian A5.4b, B2.3 y A5.2. Es la práctica de Stripe, GitHub y AWS.

ADR-02 · El consumidor rota; el consumidor y el proveedor revocan; el soporte no toca claves

- **Contexto:** el RF-27 decía que rotaba el consumidor, los mockups decían que el proveedor y el RF-28 incluía a soporte.
- **Decisión:** solo el consumidor rota (la clave anterior vive 24 h más). El consumidor revoca sus claves y el proveedor revoca las de sus consumidores. El soporte es de solo lectura.
- **Alternativas descartadas:** que rote el proveedor. Con ADR-01 nadie vería la clave nueva, y el dueño de la credencial es el consumidor.
- **Consecuencias:** A4.3 pierde "Rotar". B2.3 gana "Rotar" y "Revocar". Nuevas pantallas B2.4, B2.5 y B2.6. Bloquear a un consumidor completo queda fuera del alcance.

ADR-03 · El consumidor es una identidad separada, aislada por organización

- **Contexto:** el MER relacionaba al consumidor con un `usuario` global, pero todos los textos decían que "su cuenta pertenece al portal".
- **Decisión:** tabla `consumidor` con credenciales propias y `UNIQUE(organizacion_id, correo)`. Hay dos ámbitos de sesión: `personal` y `consumidor`.
- **Alternativas descartadas:** un usuario global compartido. Si se filtrara un portal afectaría a los demás, el mismo correo tendría la misma contraseña en todos los portales, y contradice el modelo de marca blanca.

- **Consecuencias:** hay dos flujos de autenticación que comparten el mismo servicio, con un parámetro para el ámbito. La cuenta sirve en todos los portales de esa organización.

ADR-04 · El consumidor también verifica su correo y recupera su contraseña

- **Contexto:** los lineamientos lo exigen, el RF-03 decía "Todos" y no había pantallas.
- **Decisión:** el consumidor también verifica su correo (y no puede contratar hasta hacerlo) y recupera su contraseña. Pantallas nuevas A5.8, A5.9 y A5.10.
- **Consecuencias:** se evitan cuentas falsas y se cumple el punto 4.1 de los lineamientos para todos los usuarios.

ADR-05 · Matriz de permisos explícita y una sola organización por usuario

- **Decisión:** la matriz de 04, con denegación por defecto. Un solo propietario por organización. Un usuario pertenece a **una sola** organización.
- **Alternativas descartadas:** usuarios en varias organizaciones, porque exigiría un selector de organización y más pruebas de aislamiento, sin que ningún requisito lo pida.
- **Consecuencias:** no se puede invitar a alguien que ya pertenece a otra organización (`correo_en_otra_organizacion`).

ADR-06 · Sesiones del lado del servidor con cookie, PBKDF2 y sin verificación en dos pasos

- **Decisión:** la sesión se guarda en la tabla `sesion` y la cookie es `HttpOnly` con `SameSite=Lax`. El hash de las contraseñas es el PBKDF2 de Identity. Se agregan bloqueo tras intentos fallidos y cierre de las demás sesiones al recuperar la contraseña. **Se quita la verificación en dos pasos** que mencionaba la propuesta.
- **Alternativas descartadas:** JWT, porque no permite revocar sesiones al recuperar la contraseña sin montar una lista de bloqueo, y en el navegador suma riesgo de XSS.
- **Consecuencias:** como el frontend y la API comparten host, no hace falta CORS entre ellos.

ADR-07 · El subdominio, el portal y la marca son de cada API

- **Contexto:** la propuesta y A3 los ponían por API, pero A6.2 y el MER los tenían por organización.
- **Decisión:** cada API tiene su subdominio (elegido por el proveedor), su dominio propio, su portal y su marca.
- **Alternativas descartadas:** un portal por organización con varias APIs, porque necesita un selector de API en el portal y rediseñar A5 y B2.
- **Consecuencias:** cada API es un "entorno web", lo que **refuerza el argumento de la categoría** del enunciado. Salen de `organizacion` los campos `subdominio`, `color_marca` y `logo_url`. En A3.2 se agrega el campo Subdominio. En A6.2 se muestra el propietario.

ADR-08 · Hosts distintos para el portal y la API

- **Contexto:** `envios.shapi.com` servía el portal y también `envios.shapi.com/cotizaciones`, así que las rutas chocaban.

- **Decisión:** `{sub}.shapi.localhost` para el portal, `{sub}.api.shapi.localhost` para la API y `shapi.localhost` para el sitio, el panel y la administración.
- **Alternativas descartadas:** un prefijo de ruta (`/api/…`), porque cambia las URL del proveedor. Un segundo dominio, porque cuesta dinero.
- **Consecuencias:** el borde enruta solo por nombre de host. Hay que manejar CORS en la compuerta para la consola de pruebas (08 §6).

ADR-09 · Caddy en el borde y dominios simulados bajo `*.localhost` (plan B, costo cero)

- **Contexto:** Let's Encrypt con un certificado por subdominio tiene límites de emisión, y Nginx necesita un cliente ACME y scripts. **El equipo decidió no gastar dinero**: no hay dominio ni servidor público.
- **Decisión: Caddy**, con su autoridad certificadora interna (`local_certs`). Emite al instante los comodines `*.shapi.localhost` y `*.api.shapi.localhost`, y los certificados *on-demand* de los dominios propios verificados (pregunta a `/interno/tls/autorizar`). Los navegadores resuelven `*.localhost` a 127.0.0.1 sin configuración (RFC 6761).
- **Alternativas descartadas:**
  - Nginx con Certbot, que requiere un dominio real y es más complejo.
  - `nip.io` o `sslip.io`, que dependen de un servicio externo y producen nombres difíciles de leer.
  - Editar el archivo `hosts`, que no admite comodines.
- **Consecuencias:** cada equipo confía una vez en la raíz de Caddy. El dominio base se configura con `SHAPI_DOMINIO_BASE`, así que el código no cambia si algún día hay un dominio real. Encaja con el requisito de "gestión de dominios simulados" del enunciado.

ADR-10 · El dominio propio es solo para la API y se verifica con DNS simulado

- **Decisión:** el proveedor conecta su dominio a `{sub}.api.shapi.localhost` con un CNAME. `IResolutorDns` tiene dos implementaciones: `simulado` (tabla `registro_dns_simulado`, con el botón "Simular la creación del registro" en A3.6) y `real` (DnsClient.NET). En el entorno simulado, los dominios deben terminar en `.localhost`.
- **Por qué solo la API:** la URL de la API es la que el consumidor escribe en su código. Si es del proveedor, puede irse de Shapi sin romperle nada a sus clientes. El portal ya es de marca blanca en su subdominio.
- **Consecuencias:** requiere un plan con dominio propio (desde Producto). El certificado se emite con la primera petición.

ADR-11 · Protección contra SSRF y secreto de origen

- **Decisión:** la URL de origen no puede resolver a direcciones internas. Se valida al guardar **y** en cada conexión (`ConnectCallback`), para cubrir el *DNS rebinding*. Cada API tiene un **secreto de origen**, guardado cifrado, que se muestra una vez y viaja en `X-Shapi-Secreto`. En el modo demostración se permiten los orígenes de demostración, que están en la red de Docker.
- **Consecuencias:** sin esto, un proveedor podría usar la compuerta para llegar a Redis o a PostgreSQL. El proveedor puede validar el secreto (es opcional), así que "sin modificar el servidor" sigue siendo cierto.

ADR-12 · Pasarela de pagos simulada detrás de una interfaz

- **Decisión:** `IPasarelaPagos` tiene una implementación simulada, con validación de Luhn, detección de la marca, tarjetas de prueba (incluida una que falla solo en las renovaciones) y autorización inmediata. Se quitan "Confirmar pagos" y la rama "pendiente de confirmación". El RF-24 queda como "revisar y revertir".
- **Consecuencias:** la propuesta de valor ("conectar una pasarela local") queda como un punto de extensión real de la arquitectura.

ADR-13 · Se guarda el token de la tarjeta, no la tarjeta

- **Contexto:** los textos decían que "no se almacena la tarjeta", pero la renovación se cobraba "a la misma tarjeta".
- **Decisión:** se guardan el token de la pasarela, la marca, los últimos 4 dígitos, el titular y el vencimiento. Nunca el número completo ni el CVV.
- **Consecuencias:** las renovaciones automáticas funcionan como en una pasarela real (PCI). Cambian los textos de A2.2 y A5.6.

ADR-14 · Historia de negocio y cobro por cuenta del proveedor

- **Decisión:** Shapi cobra a los consumidores por cuenta del proveedor y sin comisión. **Liquidarle ese dinero al proveedor queda fuera del alcance.** Se corrige la justificación de la propuesta, incluida la frase sin terminar.

ADR-15 · Una sola máquina de estados para las suscripciones, con 7 días de gracia

- **Decisión:** `activa → en_gracia (7 días) → suspendida → activa` (al pagar, con un ciclo nuevo desde ese día), más `finalizada`. Es la misma para la plataforma y para las APIs. La compuerta solo mira el estado, nunca las fechas.
- **Consecuencias:** hay un solo conjunto de reglas y de pruebas. Los 7 días que ya mostraban los mockups pasan a ser un requisito.

ADR-16 · El plan Prueba vence

- **Decisión:** la Prueba dura 30 días y **no se renueva**. Al terminar entra en gracia y después se suspende. Se avisa 7 días antes.

ADR-17 · Reglas para cambiar de plan

- **Decisión:** subir de plan se cobra prorrateado y se aplica de inmediato. Bajar se aplica en la siguiente renovación, sin reembolso, y se bloquea si la organización excede los límites del plan de destino. La comparación se hace por **precio diario**.
- **Consecuencias:** no hay saldos a favor ni cobros negativos. Nueva pantalla B2.7 para el consumidor.

ADR-18 · Petición y llamada son unidades distintas

- **Decisión:**
  - *Petición* = una solicitud HTTP.

    - *Llamada* = la unidad de la cuota del consumidor, según el peso de cada ruta.
    - Los límites por minuto y la **cuota de plataforma** se cuentan en peticiones.
    - La **cuota del consumidor**, en llamadas.
- **Por qué:** si la cuota de plataforma se contara en llamadas, el proveedor podría evitarla poniéndoles peso 0 a sus rutas.
- **Consecuencias:** se corrigen los textos de A0, A2, A6, A3, A4, A5 y B1.

ADR-19 · Datos concretos de los planes y una suscripción por API

- **Decisión:**
    - Miembros por plan: Prueba 1, Lanzamiento 3, Producto 10 y Escala sin límite.
    - "Escala: 30 días o anual" se divide en **Escala mensual** (Q 1,500 y 10,000,000 peticiones cada 30 días) y **Escala anual** (Q 15,000 y 120,000,000 peticiones cada 365 días).
    - Cada consumidor tiene **una sola suscripción vigente por API**; si quiere otra, cambia de plan.

ADR-20 · Orden definitivo de la tubería de filtros

- **Decisión:** CORS → API → clave → organización → suscripción → ruta → límites y cuotas (incluida la **cuota de plataforma**, que es un filtro nuevo) → caché → reenvío → medición, esta última **después** de responder. El detalle está en 08 §3.
- **Consecuencias:** el diagrama de arquitectura y la secuencia coinciden. La cuota de plataforma sirve de ejemplo del RNF-13.

ADR-21 · Reserva atómica de la cuota con Lua en Redis

- **Decisión:** un script Lua hace `INCR` y `INCRBY`, y revierte los contadores si se pasa algún límite. Si el origen no se pudo conectar (502), la cuota se devuelve.
- **Alternativas descartadas:** contar en memoria y consolidar después. Así lo planteaba la propuesta, con el costo de que un consumidor podía pasarse de su cuota.
- **Consecuencias: la cuota nunca se excede**, y se elimina el párrafo del "margen mínimo".

ADR-22 · La compuerta sin datos en memoria, Redis persistente y resincronización

- **Decisión:** la compuerta consulta Redis en cada petición. Solo guarda en memoria, 5 segundos como máximo, las rutas de cada API, junto con su versión. Redis se configura con AOF `everysec`. El trabajador reconstruye las llaves de configuración desde PostgreSQL al arrancar y cada 5 minutos.
- **Consecuencias:** las revocaciones y las suspensiones se aplican al instante. Se cumple RNF-05 y existe un camino para recuperarse si Redis pierde sus datos.

ADR-23 · Histogramas de latencia, errores por código y bytes

- **Contexto:** con solo el promedio no se puede calcular el p95 que muestra B1.1.
- **Decisión:** `consumo_diario` guarda histogramas de 10 rangos (para la latencia total y la de la compuerta), contadores por código y bytes de entrada y de salida. La consolidación es por lotes y es idempotente (`lote_consolidado`).

- **Consecuencias:** el p95 se puede sumar entre días y rutas. Los bytes refuerzan el requisito de "monitorear consumo de recursos".

ADR-24 · Qué hace la clave de pruebas

- **Decisión:** llega al mismo origen con `X-Shapi-Entorno: pruebas`, no descuenta cuota ni se factura, y tiene un límite de 10 peticiones por minuto y 1,000 por día.

ADR-25 · Cabeceras de la compuerta

- **Decisión:** la clave va en `X-Api-Key` (nunca en la query string) y el origen no la recibe. Al origen se le envían `X-Shapi-Consumidor`, `X-Shapi-Entorno` y `X-Shapi-Secreto`. Al consumidor se le devuelven `X-Shapi-Plan`, `X-RateLimit-*` y `X-Cuota-*` (08 §5).

ADR-26 · OpenAPI y caché de respuestas

- **Decisión:**
  - Se aceptan OpenAPI 3.0 y 3.1, en JSON o YAML y de hasta 2 MB.
  - Si se vuelve a cargar la especificación, las rutas que ya existían conservan su configuración y las nuevas quedan **ocultas**.
  - La caché es opcional por ruta (solo GET), compartida entre consumidores, y un acierto sí descuenta cuota.
  - Es la funcionalidad de menor prioridad.

ADR-27 · Procesos y estilo arquitectónico

- **Decisión:**
  - **Procesos:** borde (Caddy), compuerta (YARP), API de control (tres capas), **trabajador aparte** (consolidación, cierre de ciclos, correo, dominios y resincronización), dos frontends React (panel y portal), Redis, PostgreSQL y Mailpit.
  - **Correo:** pasa por una bandeja de salida en la base de datos (`correo_saliente`).
- **Alternativas descartadas:**
  - **Un monolito**, porque un despliegue o una caída del panel cortaría el tráfico (incumple RNF-04).
  - **Microservicios por dominio**, porque su operación cuesta demasiado para 4 personas y 6 semanas.
  - **Kong o Tyk como compuerta**, porque el curso pide desarrollar la solución y la compuerta es el núcleo del producto.
- **Consecuencias:** si se cae la API de control, la medición sigue. El correo tiene su propio indicador de salud (B3.1).

ADR-28 · Modelo de datos completo

- **Decisión:** el MER de 07. Se agregan `consumidor` (con credenciales), `token`, `sesion`, `dominio_propio`, `registro_dns_simulado`, `medio_pago`, `caso_mensaje`, `bitacora`, `correo_saliente` y `lote_consolidado`. Se completan columnas que faltaban: descripciones, límites, peso, estados, concepto del pago y otras. Hay un diseño físico con restricciones e índices, y una estructura documentada de las llaves de Redis.

ADR-29 · Mockups corregidos y pantallas nuevas

- **Decisión:** se corrigen las pantallas que se contradecían con estas decisiones y se agregan las que faltaban: A5.8 a A5.10, A7.1, A7.2, A8.1, A8.2, B1.5 y B2.4 a B2.7. La lista está en 11 §3.

ADR-30 · Ciclos exactos de 30 días y cómo se muestran las fechas

- **Contexto:** los mockups mezclaban ciclos de 30 y de 31 días (por ejemplo "24 ago – 23 sep" con renovación el 24 de septiembre), aunque la vigencia decía "30 días".
- **Decisión:** `fin = inicio + vigencia_dias`. En pantalla se muestra el último día con servicio (`fin − 1 día`) y la renovación en `fin`. Por ejemplo, **24 ago 2026 – 22 sep 2026**, con renovación el **23 de septiembre de 2026**. El periodo de gracia se muestra igual (por ejemplo, 23 sep – 29 sep, con suspensión el 30 de septiembre).
- **Consecuencias:** se corrigieron las fechas de A2, A6 y B1. El prorrateo de A2.5 (13 días) no cambia.

ADR-31 · Las especificaciones viven en el repositorio y se verifican automáticamente

- **Decisión:** `docs/specs/*.md` es la **fuente de verdad**. Los diagramas están en Mermaid, que GitHub muestra directamente. Los documentos Word y PDF que se entregan se generan desde ahí. Nada se integra a `main` sin CI en verde (RNF-15).
- **Consecuencias:** los agentes de IA trabajan contra texto que pueden versionar y revisar. Desaparece la marca "Text is not SVG" de los diagramas exportados.

ADR-32 · La variante 4 es el sistema visual oficial

- **Decisión:** la variante 4, "Plano azul" (Sora, IBM Plex Sans y `#3B6FF0`), que es la que ya usaban todas las pantallas de la aplicación. Las demás quedan en `mockups/_descartadas/`.

ADR-33 · Orígenes de demostración y siembra igual a los mockups

- **Decisión:** el repositorio incluye dos APIs de origen de demostración (Envíos Xelajú y Agro Precios, cada una con su OpenAPI) y una siembra con los mismos datos que los mockups.
- **Consecuencias:** el "sistema funcionando" se puede demostrar de punta a punta sin depender de terceros.

ADR-34 · La categoría ya está aprobada

- **Decisión:** la docente aprobó la propuesta en la categoría *Servidores Web como Servicio*. La correspondencia con cada requisito del enunciado está en 01 §2.

ADR-35 · Versiones del stack

- **Decisión:**
  - **Backend: .NET 10 (LTS)** en vez de .NET 8, cuyo soporte termina en noviembre de 2026. Sigue siendo ASP.NET Core, así que no cambia nada respecto a lo aprobado.
  - **Frontend:** React 19, TypeScript, Vite y Tailwind 4, en un *workspace* pnpm con dos aplicaciones y un paquete de UI.
  - **Datos:** PostgreSQL 16 (por `NULLS NOT DISTINCT`) y Redis 7.4.

- **Consecuencias:** cada integrante instala el SDK de .NET 10, Node 24 o posterior, pnpm y Docker Desktop. (El 26 de septiembre se cambió de Node 22 a Node 24, que es lo que exigen `frontend/package.json` y la CI.)